Principia Softwarica: The Plan 9 Widget Library
`libpanel`

version 0.1

Yoann Padioleau
yoann.padioleau@gmail.com

with code from
Tom Duff

February 12, 2026

The text and figures are Copyright © 2014–2026 Yoann Padioleau.
All rights reserved.

The source code is Copyright © 2021 Plan 9 Foundation.
MIT license.

Contents

1 Introduction **7**
1.1 Motivations . . . 7
1.2 The Plan 9 widget library: `libpanel` . . . 7
1.3 Other widget libraries . . . 7
1.4 Getting started . . . 7
1.5 Requirements . . . 7
1.6 About this document . . . 7
1.7 Copyright . . . 8
1.8 Acknowledgments . . . 8

2 Overview **9**
2.1 Widget library principles . . . 9
2.1.1 The widget . . . 9
2.1.2 The event loop . . . 9
2.1.3 Layout . . . 9
2.1.4 Scrolling . . . 9
2.2 `hellopanel.c` . . . 9
2.3 Code organization . . . 10
2.4 Software architecture . . . 10
2.5 Book structure . . . 10

3 Core Data Structures **11**
3.1 `Panel` . . . 11
3.1.1 Widget flags . . . 12
3.1.2 A tree of widgets . . . 12
3.1.3 Widget-specific data . . . 13
3.1.4 Widget display state . . . 13
3.1.5 Main widget methods . . . 14
3.2 `Icon` . . . 14
3.3 Layout styles . . . 14
3.3.1 Placement . . . 14
3.3.2 Packing . . . 14
3.3.3 Filling . . . 15
3.3.4 Padding . . . 15
3.4 `Direction` . . . 15

4 Initialization **16**
4.1 `plinit()` . . . 16

5 Drawing **17**
5.1 `pldraw()` . . . 17
5.2 Drawing helpers . . . 17
5.3 Drawing boxes . . . 18

6 Events **19**
6.1 Mouse events . . . 19
6.1.1 Mouse leaving the widget . . . 20
6.1.2 `REMOUSE` . . . 20
6.1.3 Hitting priority . . . 20
6.2 Keyboard events . . . 21

7 Layout **22**
7.1 Layout fields . . . 22
7.2 Computing sizes: `pl_sizereq()` . . . 22
7.2.1 Packing . . . 23
7.2.2 Padding . . . 23
7.3 Computing rectangles: `pl_setrect()` . . . 24
7.3.1 Padding . . . 24
7.3.2 Filling . . . 24
7.3.3 Placing . . . 24
7.3.4 Packing the children . . . 25

8 Basic Widgets **27**
8.1 Label . . . 27
8.1.1 Initializing . . . 27
8.1.2 Drawing . . . 28
8.1.3 Reacting . . . 29
8.1.4 Packing methods . . . 29
8.2 Text entry . . . 30
8.2.1 Initializing . . . 30
8.2.2 Drawing . . . 31
8.2.3 Reacting . . . 32
8.2.4 Packing methods . . . 34
8.2.5 Other methods . . . 34
8.3 Button . . . 34
8.3.1 `Button` . . . 34
8.3.2 Basic button . . . 35
8.3.3 Check button . . . 37
8.3.4 Radio button . . . 39
8.4 Slider . . . 40
8.4.1 Initializing . . . 41
8.4.2 Drawing . . . 41
8.4.3 Reacting . . . 42
8.4.4 Packing methods . . . 43
8.4.5 Other methods . . . 43
8.5 Canvas . . . 43
8.5.1 Initializing . . . 44
8.5.2 Drawing . . . 44
8.5.3 Reacting . . . 44

8.5.4 Packing methods . . . 45

9 Composite Widgets **46**
9.1 Frame . . . 46
9.1.1 Initializing . . . 46
9.1.2 Drawing . . . 46
9.1.3 Reacting . . . 47
9.1.4 Packing methods . . . 47
9.2 Group . . . 47
9.2.1 Initializing . . . 47
9.2.2 Drawing . . . 48
9.2.3 Reacting . . . 48
9.2.4 Packing methods . . . 48

10 Scrollable Widgets **49**
10.1 Scrolling fields . . . 49
10.2 Scrolling methods . . . 49
10.3 Scrollable list . . . 50
10.3.1 Initializing . . . 50
10.3.2 Drawing . . . 51
10.3.3 Reacting . . . 52
10.3.4 Packing methods . . . 53
10.3.5 Scrollbar to scrollee . . . 53
10.4 Scroll bar . . . 55
10.4.1 Initializing . . . 55
10.4.2 Drawing . . . 56
10.4.3 Reacting . . . 56
10.4.4 Packing methods . . . 57
10.4.5 Scrollee to scrollbar . . . 58

11 Menu widgets **59**
11.1 Menu items . . . 59
11.1.1 Initializing . . . 59
11.1.2 Drawing . . . 60
11.1.3 Reacting . . . 60
11.2 Popup menu . . . 60
11.2.1 Initializing . . . 60
11.2.2 Reacting . . . 61
11.2.3 Drawing . . . 62
11.2.4 Packing methods . . . 63
11.3 Pull-down . . . 63
11.3.1 Initializing . . . 63
11.3.2 Drawing . . . 64
11.3.3 Reacting . . . 64
11.3.4 Packing methods . . . 66
11.4 Menu bar . . . 66
11.4.1 Initializing . . . 66

12 Text Widgets **68**
12.1 Message . . . 68
12.1.1 Initializing . . . 68
12.1.2 Drawing . . . 69
12.1.3 Reacting . . . 69
12.1.4 Packing methods . . . 69
12.2 Edit . . . 70
12.2.1 Initializing . . . 70
12.2.2 Drawing . . . 71
12.2.3 Reacting . . . 72
12.2.4 Packing methods . . . 73
12.2.5 Other methods . . . 73
12.3 Text view . . . 75
12.3.1 Initializing . . . 76
12.3.2 Drawing . . . 76
12.3.3 Reacting . . . 76
12.3.4 Packing methods . . . 78
12.4 Completion . . . 78
12.5 Rich text . . . 78
12.5.1 Initializing . . . 78
12.5.2 Drawing . . . 79
12.5.3 Reacting . . . 81
12.5.4 Packing methods . . . 81
12.5.5 Other methods . . . 81

13 Advanced Topics TODO **82**
13.1 copy/paste . . . 82
13.1.1 Widgets hooks . . . 83
13.2 Rich text . . . 84
13.3 Hooks . . . 84
13.4 Advanced layout . . . 84
13.4.1 `FIXEDX`, `FIXEDY` . . . 84
13.4.2 `MAXX`, `MAXY` . . . 85

14 Advanced Widgets TODO **86**

15 Conclusion **87**

A Debugging **88**

B Error Management **90**

C Utilities **91**
C.1 Memory management . . . 91

D Extra Code **92**
D.1 `include/gui/` . . . 92
D.1.1 `include/gui/panel.h` . . . 92
D.1.2 `include/gui/rtext.h` . . . 95
D.2 `lib_gui/libpanel/` . . . 96
D.2.1 `lib_gui/libpanel/pldefs.h` . . . 96

D.2.2 lib_gui/libpanel/init.c . . . 98
D.2.3 lib_gui/libpanel/mem.c . . . 98
D.2.4 lib_gui/libpanel/draw.c . . . 98
D.2.5 lib_gui/libpanel/event.c . . . 99
D.2.6 lib_gui/libpanel/print.c . . . 99
D.2.7 lib_gui/libpanel/label.c . . . 99
D.2.8 lib_gui/libpanel/button.c . . . 100
D.2.9 lib_gui/libpanel/entry.c . . . 100
D.2.10 lib_gui/libpanel/edit.c . . . 101
D.2.11 lib_gui/libpanel/slider.c . . . 101
D.2.12 lib_gui/libpanel/canvas.c . . . 102
D.2.13 lib_gui/libpanel/frame.c . . . 102
D.2.14 lib_gui/libpanel/group.c . . . 102
D.2.15 lib_gui/libpanel/list.c . . . 102
D.2.16 lib_gui/libpanel/message.c . . . 103
D.2.17 lib_gui/libpanel/pack.c . . . 103
D.2.18 lib_gui/libpanel/popup.c . . . 103
D.2.19 lib_gui/libpanel/pulldown.c . . . 104
D.2.20 lib_gui/libpanel/rtext.c . . . 104
D.2.21 lib_gui/libpanel/scroll.c . . . 108
D.2.22 lib_gui/libpanel/scrollbar.c . . . 108
D.2.23 lib_gui/libpanel/snarf.c . . . 108
D.2.24 lib_gui/libpanel/textview.c . . . 109
D.2.25 lib_gui/libpanel/textwin.c . . . 111
D.2.26 lib_gui/libpanel/utf.c . . . 119

Glossary **121**

Index **122**

References **122**

Chapter 1

Introduction

The goal of this book is to explain with full details the source code of a widget library.

1.1 Motivations

Here are a few questions I hope this book will answer:

- How does a mouse event get dispatched to the appropriate widget?
- How does scrolling work? How do widgets communicate with each other?

1.2 The Plan 9 widget library: `libpanel`

1.3 Other widget libraries

Here are a few widget libraries that I considered for this book, but which I ultimately discarded:

- Xlib
- Tcl/Tk
- GTK
- Qt

1.4 Getting started

1.5 Requirements

1.6 About this document

This document is a *literate program* [Knu92]. It derives from a set of files processed by a tool, `syncweb` [Pad09], generating either this book or the actual source code of the program. The code and its documentation are thus strongly connected.

1.7 Copyright

Most of this document is made of source code from Plan 9, so those parts are copyright by Plan 9 Foundation. The prose is mine and is licensed under the All Rights Reserved License.

1.8 Acknowledgments

I would like to acknowledge of course the author of `libpanel`, Tom Duff, who wrote in some sense most of this book.

Chapter 2

Overview

2.1 Widget library principles

2.1.1 The widget

2.1.2 The event loop

2.1.3 Layout

2.1.4 Scrolling

2.2 hellopanel.c

⟨lib_gui/libpanel/tests/hellopanel.c 9⟩≡

```
// source code from panel.pdf introduction
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <event.h>

#include <panel.h>

Panel *root;

⟨function done 10a⟩

⟨function eresized 10c⟩

void main(void){
    errorneg1 err;
    Event e;
    int i;

    err = initdraw(nil, nil, "hellopanel");
    ⟨main() sanity check err 10b⟩
    einit(Emouse);
    plinit(view->depth);

    root=plframe(nil, NOFLAG);
    pllabel(root, NOFLAG, "Hello, world!");
    plbutton(root, NOFLAG, "done", done);

    eresized(false);
```

```
    for(;;) {
        i=event(&e);
        plmouse(root, &e.mouse);
    }
}
```

⟨*function* done 10a⟩≡ (9)

```
void done(Panel *p, buttons buttons){
    USED(p, buttons);
    exits(nil);
}
```

⟨main() *sanity check* err 10b⟩≡ (9)

```
if(err < 0)
    sysfatal("initdraw: %r");
```

⟨*function* eresized 10c⟩≡ (9)

```
void eresized(bool new){
    ⟨eresized() if new get a new window 10d⟩
    plpack(root, view->r);
    pldraw(root, view);
}
```

⟨eresized() *if* new *get a new window* 10d⟩≡ (10c)

```
if(new && getwindow(display, Refnone) == ERROR_NEG1) {
    fprint(STDERR, "getwindow: %r\n");
    exits("getwindow");
}
```

2.3 Code organization

⟨libpanel *includes* 10e⟩≡ (120d 119a 110d 108 107 104a 103 102 101 100 99 98)

```
#include <u.h>
#include <libc.h>
#include <draw.h>
#include <event.h>

#include <panel.h>
#include "pldefs.h"
```

2.4 Software architecture

2.5 Book structure

Chapter 3

Core Data Structures

3.1 Panel

⟨*struct* Panel 11a⟩≡ (92)

```
struct Panel{
    // public

    Rectangle r;     /* where the Panel goes */

    ⟨Panel padding fields 15i⟩
    ⟨Panel fixed size field 85b⟩
    ⟨Panel user-specific fields 84e⟩

    /* private below */

    Image *b;     /* where we're drawn */
    ⟨Panel widget-specific data 13e⟩
    ⟨Panel layout fields 22b⟩
    ⟨Panel debugging fields 88a⟩
    ⟨Panel other fields 12c⟩

    // methods
    ⟨Panel main methods 14a⟩
    ⟨Panel packing methods 23a⟩
    ⟨Panel other methods 13f⟩

    // Extra
    ⟨Panel extra fields 12f⟩
};
```

⟨*function* pl_newpanel 11b⟩≡ (98b)

```
Panel *pl_newpanel(Panel *parent, int ndata){
    Panel *v;

    ⟨pl_newpanel() sanity check if can create child on parent 13d⟩
    v=pl_emalloc(sizeof(Panel));

    ⟨pl_newpanel() set tree fields 12j⟩
    ⟨pl_newpanel() set widget-specific data fields 13g⟩
    ⟨pl_newpanel() set other fields 12a⟩

    ⟨pl_newpanel() set default methods 14b⟩

    return v;
}
```

⟨pl_newpanel() *set other fields* 12a⟩≡ (11b) 12e ▷

```
v->r=Rect(0,0,0,0);
v->b=nil;
```

⟨*function* plfree 12b⟩≡ (98b)

```
void plfree(Panel *p){
    ⟨plfree locals 13a⟩

    if(p==nil)
        return;
    ⟨plfree() if plkbfocus 21f⟩
    ⟨plfree() free the children 13b⟩
    ⟨plfree() free the widget-specific data 13h⟩
    free(p);
}
```

3.1.1 Widget flags

⟨Panel *other fields* 12c⟩≡ (11a) 13i ▷

```
// LEAF | INVIS | ...
int flags;     /* position flags, see below */
```

⟨*constant* NOFLAG 12d⟩≡ (92)

```
#define NOFLAG 0
```

⟨pl_newpanel() *set other fields* 12e⟩+≡ (11b) ◁ 12a 15j ▷

```
v->flags=NOFLAG;
```

3.1.2 A tree of widgets

⟨Panel *extra fields* 12f⟩≡ (11a) 12g ▷

```
// option<list<ref_own<Panel>>> (next = Panel.next, tail = Panel.echild)
Panel *child;
```

⟨Panel *extra fields* 12g⟩+≡ (11a) ◁ 12f 12h ▷

```
// list<ref<Panel>> (head = Panel.child)
Panel *next;     /* It's a list! */
```

⟨Panel *extra fields* 12h⟩+≡ (11a) ◁ 12g 12i ▷

```
// option<ref<Panel>>
Panel *echild;
```

⟨Panel *extra fields* 12i⟩+≡ (11a) ◁ 12h

```
// option<ref<Panel>>
Panel *parent;    /* No, it's a tree! */
```

⟨pl_newpanel() *set tree fields* 12j⟩≡ (11b)

```
v->next=nil;
v->child=nil;
v->echild=nil;

v->parent=parent;
//add_list(v, parent->child)
if(parent){
    if(parent->child==nil)
        parent->child=v;
    else
        parent->echild->next=v;
    parent->echild=v;
}
```

⟨plfree *locals* 13a⟩≡ (12b)
```
Panel *cp, *ncp;
```

⟨plfree() *free the children* 13b⟩≡ (12b)
```
for(cp=p->child;cp;cp=ncp){
    ncp=cp->next;
    plfree(cp);
}
```

⟨*constant* LEAF 13c⟩≡ (96d)
```
#define LEAF 0x10000  /* newpanel will refuse to attach children */
```

⟨pl_newpanel() *sanity check if can create child on* parent 13d⟩≡ (11b)
```
if(parent && parent->flags&LEAF){
    fprint(STDERR, "newpanel: can't create child of %s %lux\n",
             parent->kind, (ulong)parent);
    exits("bad newpanel");
}
```

3.1.3 Widget-specific data

⟨Panel *widget-specific data* 13e⟩≡ (11a)
```
// union<ref_own<Label|Entry|Button|...>
void *data;     /* kind-specific data */
```

⟨Panel *other methods* 13f⟩≡ (11a) 20f ▷
```
void (*free)(Panel *);    /* free fields of data when done */
```

⟨pl_newpanel() *set widget-specific data fields* 13g⟩≡ (11b)
```
if(ndata)
    v->data=pl_emalloc(ndata);
else
    v->data=nil;
v->free=nil;
```

⟨plfree() *free the widget-specific data* 13h⟩≡ (12b)
```
if(p->free)
    p->free(p);
if(p->data)
    free(p->data);
```

3.1.4 Widget display state

⟨Panel *other fields* 13i⟩+≡ (11a) ◁ 12c 19b ▷
```
// enum<Style>
int state;     /* for hitting & drawing purposes */
```

⟨*enum* Style 13j⟩≡ (96d)
```
/*
 * States, also styles
 */
enum{
    // for text entries, buttons
    UP,
    DOWN,
    ⟨Style other cases 28c⟩
};
```

3.1.5 Main widget methods

⟨Panel *main methods* 14a⟩≡ (11a)
```
  void (*draw)(Panel *);    /* draw panel and children */

  bool (*hit)(Panel *, Mouse *);   /* process mouse event */
  void (*type)(Panel *, Rune);   /* process keyboard event */
```

⟨pl_newpanel() *set default methods* 14b⟩≡ (11b) 23b ▷
```
  v->draw=pl_drawerror;
  v->hit=pl_hiterror;
  v->type=pl_typeerror;
```

3.2 Icon

⟨*type* Icon 14c⟩≡ (92)
```
  typedef void Icon;   /* Always used as Icon * -- Image or char */
```

3.3 Layout styles

3.3.1 Placement

⟨*constant* PLACE 14d⟩≡ (92)
```
  #define PLACE 0x01e0  /* which side of its space should the Panel adhere to? */
```

⟨*constant* PLACECEN 14e⟩≡ (92)
```
  #define  PLACECEN 0x0000
```

⟨*constant* PLACES 14f⟩≡ (92)
```
  #define  PLACES 0x0020
```

⟨*constant* PLACEE 14g⟩≡ (92)
```
  #define  PLACEE 0x0040
```

⟨*constant* PLACEW 14h⟩≡ (92)
```
  #define  PLACEW 0x0060
```

⟨*constant* PLACEN 14i⟩≡ (92)
```
  #define  PLACEN 0x0080
```

⟨*constant* PLACENE 14j⟩≡ (92)
```
  #define  PLACENE 0x00a0
```

⟨*constant* PLACENW 14k⟩≡ (92)
```
  #define  PLACENW 0x00c0
```

⟨*constant* PLACESE 14l⟩≡ (92)
```
  #define  PLACESE 0x00e0
```

⟨*constant* PLACESW 14m⟩≡ (92)
```
  #define  PLACESW 0x0100
```

3.3.2 Packing

⟨*constant* PACK 14n⟩≡ (92)
```
  #define PACK 0x0007  /* which side of the parent is the Panel attached to? */
```

⟨*constant* PACKN 15a⟩≡ (92)
```
#define  PACKN 0x0000
```

⟨*constant* PACKE 15b⟩≡ (92)
```
#define  PACKE 0x0001
```

⟨*constant* PACKS 15c⟩≡ (92)
```
#define  PACKS 0x0002
```

⟨*constant* PACKW 15d⟩≡ (92)
```
#define  PACKW 0x0003
```

⟨*constant* PACKCEN 15e⟩≡ (92)
```
#define  PACKCEN 0x0004 /* only used by pulldown */
```

3.3.3 Filling

⟨*constant* FILLX 15f⟩≡ (92)
```
#define FILLX 0x0008  /* grow horizontally to fill the available space */
```

⟨*constant* FILLY 15g⟩≡ (92)
```
#define FILLY 0x0010  /* grow vertically to fill the available space */
```

⟨*constant* EXPAND 15h⟩≡ (92)
```
#define EXPAND 0x0200  /* use up all extra space in the parent */
```

3.3.4 Padding

⟨Panel *padding fields* 15i⟩≡ (11a)
```
Vector ipad;    /* extra space inside and outside */
Vector pad;
```

⟨pl_newpanel() *set other fields* 15j⟩+≡ (11b) ◁12e 19c▷
```
v->ipad=Pt(0,0);
v->pad=Pt(0,0);
```

3.4 Direction

⟨*enum* Direction 15k⟩≡ (96d)
```
/*
 * Scrollbar, slider orientations
 */
enum Direction {
    HORIZ,
    VERT
};
```

Chapter 4

Initialization

4.1 plinit()

⟨*function* plinit 16a⟩≡ (98a)

```
/*
 * Just a wrapper for all the initialization routines
 */
error0 plinit(int ldepth){
    return pl_drawinit(ldepth);
}
```

⟨*global* plldepth 16b⟩≡ (98c)

```
static int plldepth;
```

⟨*globals* pl_xxx 16c⟩≡ (98c)

```
static Image *pl_white, *pl_light, *pl_dark, *pl_black, *pl_hilit;
```

⟨*function* pl_drawinit 16d⟩≡ (98c)

```
error0 pl_drawinit(int ldepth){

    plldepth=ldepth;

    pl_black=allocimage(display, Rect(0,0,1,1), view->chan, 1, 0x000000FF);
    pl_white=allocimage(display, Rect(0,0,1,1), view->chan, 1, 0xFFFFFFFF);

    pl_dark =allocimage(display, Rect(0,0,1,1), view->chan, 1, DPurpleblue);
    pl_light=allocimagemix(display, DPalebluegreen, DWhite);

    pl_hilit=allocimage(display, Rect(0,0,1,1), CHAN1(CAlpha,8), 1, 0x80);

    if(pl_white==nil || pl_light==nil || pl_black==nil || pl_dark==nil)
        return ERROR_0;
    return OK_1;
}
```

Chapter 5

Drawing

5.1 pldraw()

⟨*function* pldraw 17a⟩≡ (98c)

```
void pldraw(Panel *p, Image *b){
    pl_draw1(p, b);
    flushimage(display, true);
}
```

⟨*function* pl_draw1 17b⟩≡ (98c)

```
void pl_draw1(Panel *p, Image *b){
    if(b!=nil)
        pl_drawall(p, b);
}
```

⟨*function* pl_drawall 17c⟩≡ (98c)

```
void pl_drawall(Panel *p, Image *b){
    ⟨pl_drawall() if invisible widget 63c⟩
    p->b=b;
    // widget-specific method
    p->draw(p);
    for(p=p->child;p;p=p->next)
        // recurse
        pl_draw1(p, b);
}
```

5.2 Drawing helpers

⟨*function* pl_relief 17d⟩≡ (98c)

```
void pl_relief(Image *b, Image *ul, Image *lr, Rectangle r, int wid){
    int x, y;

    draw(b, Rect(r.min.x, r.max.y-wid, r.max.x, r.max.y), lr, 0, ZP);/* bottom*/
    draw(b, Rect(r.max.x-wid, r.min.y, r.max.x, r.max.y), lr, 0, ZP);/* right */
    draw(b, Rect(r.min.x, r.min.y, r.min.x+wid, r.max.y), ul, 0, ZP);/* left */
    draw(b, Rect(r.min.x, r.min.y, r.max.x, r.min.y+wid), ul, 0, ZP);/* top */

    for(x=0;x!=wid;x++)
      for(y=wid-1-x;y!=wid;y++){
        draw(b, rectaddpt(Rect(0,0,1,1), Pt(x+r.max.x-wid, y+r.min.y)), lr, 0, ZP);
        draw(b, rectaddpt(Rect(0,0,1,1), Pt(x+r.min.x, y+r.max.y-wid)), lr, 0, ZP);
    }
}
```

5.3 Drawing boxes

⟨*function* pl_outline 18a⟩≡ (98c)
```
Rectangle pl_outline(Image *b, Rectangle r, int style){
    return pl_boxoutline(b, r, style, false);
}
```

⟨*function* pl_box 18b⟩≡ (98c)
```
Rectangle pl_box(Image *b, Rectangle r, int style){
    return pl_boxoutline(b, r, style, true);
}
```

⟨*function* pl_boxoutline 18c⟩≡ (98c)
```
Rectangle pl_boxoutline(Image *b, Rectangle r, int style, bool fill){
    ⟨pl_boxoutline() if plldepth is zero ??⟩
    else
    switch(style){
    ⟨pl_boxoutline() switch style cases 28d⟩
    }
    return insetrect(r, SPACE);
}
```

⟨*constant* SPACE 18d⟩≡ (98c)
```
#define SPACE 1 /* space inside relief of button or frame */
```

⟨*function* pl_boxsize 18e⟩≡ (98c)
```
Vector pl_boxsize(Vector interior, int state){
    switch(state){
    ⟨pl_boxsize() switch state cases 29h⟩
    }
    // else
    return Pt(0, 0);
}
```

⟨*function* pl_interior 18f⟩≡ (98c)
```
void pl_interior(int state, Point *ul, Vector *size){
    switch(state){
    ⟨pl_interior() switch state cases 18g⟩
    }
}
```

⟨pl_interior() *switch state cases* 18g⟩≡ (18f) 18h ▷
```
case UP:
case DOWN:
case DOWN1:
case DOWN2:
case DOWN3:
    *ul=addpt(*ul, Pt(BWID+SPACE, BWID+SPACE));
    *size=subpt(*size, Pt(2*(BWID+SPACE), 2*(BWID+SPACE)));
    break;
```

⟨pl_interior() *switch state cases* 18h⟩+≡ (18f) ◁ 18g 47g ▷
```
case PASSIVE:
    *ul=addpt(*ul, Pt(PWID+SPACE, PWID+SPACE));
    *size=subpt(*size, Pt(2*(PWID+SPACE), 2*(PWID+SPACE)));
    break;
```

Chapter 6

Events

6.1 Mouse events

⟨*function* plmouse 19a⟩≡ (99a)

```
void plmouse(Panel *g, Mouse *m){
    Panel* hit;
    Panel* last = g->lastmouse;
    bool remouse;

    ⟨plmouse() if REMOUSE set hit to last 20e⟩
    else{
        hit=pl_ptinpanel(m->xy, g);
        if(last && last!=hit){
            ⟨plmouse() when last!=hit send OUT mouse event 20b⟩
        }
    }
    if(hit){
        // widget-specific method
        remouse=hit->hit(hit, m);
        ⟨plmouse() handle remouse 20d⟩
        g->lastmouse=hit;
    }
    flushimage(display, true);
}
```

⟨Panel *other fields* 19b⟩+≡ (11a) ◁13i 49a▷

```
Panel *lastmouse;    /* who got the last mouse event? */
```

⟨pl_newpanel() *set other fields* 19c⟩+≡ (11b) ◁15j 20j▷

```
v->lastmouse=nil;
```

⟨*function* pl_ptinpanel 19d⟩≡ (99a)

```
/*
 * Return the most leafward, highest priority panel containing p
 */
Panel *pl_ptinpanel(Point p, Panel *g){
    Panel *v;

    for(;g;g=g->next)
      if(ptinrect(p, g->r)){
        //recurse
        v=pl_ptinpanel(p, g->child);

        if(v && v->pri(v, p) >= g->pri(g, p))
            return v;
```

```
        else
            return g;
      }
    return nil;
}
```

6.1.1 Mouse leaving the widget

⟨*constant* OUT 20a⟩≡ (92)

```
/*
 * An extra bit in Mouse.buttons
 */
#define OUT 8   /* Mouse.buttons bit, set when mouse leaves Panel */
```

⟨plmouse() *when* last!=hit *send* OUT *mouse event* 20b⟩≡ (19a)

```
m->buttons|=OUT;
last->hit(last, m);
m->buttons&=~OUT;
```

6.1.2 REMOUSE

⟨*constant* REMOUSE 20c⟩≡ (96d)

```
#define REMOUSE 0x40000  /* send next mouse event here, even if not inside */
```

⟨plmouse() *handle* remouse 20d⟩≡ (19a)

```
if(remouse)
    g->flags|=REMOUSE;
else
    g->flags&=~REMOUSE;
```

⟨plmouse() *if* REMOUSE *set* hit *to* last 20e⟩≡ (19a)

```
if(g->flags&REMOUSE)
    hit=last;
```

6.1.3 Hitting priority

⟨Panel *other methods* 20f⟩+≡ (11a) ◁13f 49g▷

```
int (*pri)(Panel *, Point);   /* priority for hitting */
```

⟨*constant* PRI_NORMAL 20g⟩≡ (92)

```
#define PRI_NORMAL 0  /* ordinary panels */
```

⟨*constant* PRI_POPUP 20h⟩≡ (92)

```
#define PRI_POPUP 1  /* popup menus */
```

⟨*constant* PRI_SCROLLBAR 20i⟩≡ (92)

```
#define PRI_SCROLLBAR 2  /* scroll bars */
```

⟨pl_newpanel() *set other fields* 20j⟩+≡ (11b) ◁19c 22c▷

```
v->pri=pl_prinormal;
```

⟨*function* pl_prinormal 20k⟩≡ (98b)

```
int pl_prinormal(Panel *, Point){
    return PRI_NORMAL;
}
```

⟨*function* pl_pripopup 21a⟩≡ (103c)

```
int pl_pripopup(Panel *, Point){
    return PRI_POPUP;
}
```

⟨*function* pl_priscrollbar 21b⟩≡ (108d)

```
int pl_priscrollbar(Panel *, Point){
    return PRI_SCROLLBAR;
}
```

6.2 Keyboard events

⟨*global* plkbfocus 21c⟩≡ (92)

```
Panel *plkbfocus;   /* the panel in keyboard focus */
```

⟨*function* plgrabkb 21d⟩≡ (99a)

```
void plgrabkb(Panel *g){
    plkbfocus=g;
}
```

⟨*function* plkeyboard 21e⟩≡ (99a)

```
void plkeyboard(Rune c){
    if(plkbfocus){
        // widget-specific callback
        plkbfocus->type(plkbfocus, c);
        flushimage(display, true);
    }
}
```

⟨plfree() *if* plkbfocus 21f⟩≡ (12b)

```
if(p==plkbfocus)
    plkbfocus=nil;
```

Chapter 7

Layout

⟨*function* `plpack` 22a⟩≡ (103b)

```
void plpack(Panel *p, Rectangle where){
    pl_sizereq(p);
    pl_setrect(p, where.min, subpt(where.max, where.min));
}
```

7.1 Layout fields

⟨`Panel` *layout fields* 22b⟩≡ (11a) 22d ▷

```
Vector sizereq;     /* size requested by this Panel */
Vector childreq;     /* total size needed by children */
```

⟨`pl_newpanel()` *set other fields* 22c⟩+≡ (11b) ◁ 20j 22e ▷

```
v->sizereq=Pt(0,0);
```

⟨`Panel` *layout fields* 22d⟩+≡ (11a) ◁ 22b

```
Vector size;     /* space for this Panel */
```

⟨`pl_newpanel()` *set other fields* 22e⟩+≡ (11b) ◁ 22c 49b ▷

```
v->size=Pt(0,0);
```

7.2 Computing sizes: `pl_sizereq()`

⟨*function* `pl_sizereq` 22f⟩≡ (103b)

```
/*
 * Compute the requested size of p and its descendants.
 */
void pl_sizereq(Panel *p){
    Panel *cp;
    ⟨pl_sizeref() other locals 85g⟩

    for(cp=p->child;cp;cp=cp->next){
        // recurse
        pl_sizereq(cp);
        ⟨pl_sizeref() when looping over children, adjust maxsize 85h⟩
    }
    ⟨pl_sizereq() if MAXX or MAXY 85i⟩
    p->childreq=pl_sizesibs(p->child);
    // widget-specific method
    p->sizereq=p->getsize(p, p->childreq);
    ⟨pl_sizereq() adjust size with padding 23h⟩
    ⟨pl_sizereq() if FIXEDX or FIXEDY 85c⟩
}
```

⟨Panel *packing methods* 23a⟩≡ (11a) 25c ▷

```
Vector (*getsize)(Panel *, Vector);  /* return size, given child size */
```

⟨pl_newpanel() *set default methods* 23b⟩+≡ (11b) ◁ 14b 25d ▷

```
v->getsize=pl_getsizeerror;
```

7.2.1 Packing

⟨*function* pl_sizesibs 23c⟩≡ (103b)

```
Vector pl_sizesibs(Panel *p){
    Vector s;

    ⟨pl_sizesibs() if no panel 23d⟩
    // recurse
    s=pl_sizesibs(p->next);

    switch(p->flags&PACK){
    ⟨pl_sizesibs() switch packing flags cases 23e⟩
    }
    return s;
}
```

⟨pl_sizesibs() *if no panel* 23d⟩≡ (23c)

```
if(p==nil)
    return Pt(0,0);
```

⟨pl_sizesibs() *switch packing flags cases* 23e⟩≡ (23c) 23f ▷

```
case PACKN:
case PACKS:
    s.x=pl_max(s.x, p->sizereq.x);
    s.y+=p->sizereq.y;
    break;
```

⟨pl_sizesibs() *switch packing flags cases* 23f⟩+≡ (23c) ◁ 23e

```
case PACKE:
case PACKW:
    s.x+=p->sizereq.x;
    s.y=pl_max(s.y, p->sizereq.y);
    break;
```

⟨*function* pl_max 23g⟩≡ (103b)

```
int pl_max(int a, int b){
    return a>b ? a : b;
}
```

7.2.2 Padding

⟨pl_sizereq() *adjust size with padding* 23h⟩≡ (22f)

```
p->sizereq=addpt(addpt(p->sizereq, p->ipad), p->pad);
```

7.3 Computing rectangles: pl_setrect()

⟨*function* pl_setrect 24a⟩≡ (103b)

```
/*
 * Set the sizes and rectangles of p and its descendants, given their requested sizes.
 */
void pl_setrect(Panel *p, Point ul, Vector avail){
    ⟨pl_setrect() locals 25a⟩

    // setting p->size
    p->size=p->sizereq;
    ⟨pl_setrect() reduce size for external padding 24b⟩
    ⟨pl_setrect() reduce size if bigger than available 24c⟩
    ⟨pl_setrect() expand size if fill or expand flags 24d⟩

    // setting ul
    ⟨pl_setrect() adjust origin with placement 24e⟩

    // setting p->r
    p->r=Rpt(ul, addpt(ul, p->size));

    // recurse
    ⟨pl_setrect() setting the rectangle of the children 25b⟩
}
```

7.3.1 Padding

⟨pl_setrect() *reduce size for external padding* 24b⟩≡ (24a)

```
p->size=subpt(p->sizereq, p->pad);
ul=addpt(ul, divpt(p->pad, 2));
avail=subpt(avail, p->pad);
```

7.3.2 Filling

⟨pl_setrect() *reduce size if bigger than available* 24c⟩≡ (24a)

```
if(p->size.x>avail.x)
    p->size.x = avail.x;
if(p->size.y>avail.y)
    p->size.y = avail.y;
```

⟨pl_setrect() *expand size if fill or expand flags* 24d⟩≡ (24a)

```
if(p->flags&(FILLX|EXPAND))
    p->size.x=avail.x;
if(p->flags&(FILLY|EXPAND))
    p->size.y=avail.y;
```

7.3.3 Placing

⟨pl_setrect() *adjust origin with placement* 24e⟩≡ (24a)

```
switch(p->flags&PLACE){
case PLACECEN: ul.x+=(avail.x-p->size.x)/2; ul.y+=(avail.y-p->size.y)/2; break;
case PLACES: ul.x+=(avail.x-p->size.x)/2; ul.y+= avail.y-p->size.y   ; break;
case PLACEE: ul.x+= avail.x-p->size.x   ; ul.y+=(avail.y-p->size.y)/2; break;
case PLACEW:                              ul.y+=(avail.y-p->size.y)/2; break;
case PLACEN: ul.x+=(avail.x-p->size.x)/2;                              break;
case PLACENE: ul.x+= avail.x-p->size.x   ;                              break;
case PLACENW:   /** Nothing **/                                          break;
```

```
  case PLACESE: ul.x+= avail.x-p->size.x   ; ul.y+= avail.y-p->size.y   ; break;
  case PLACESW:                               ul.y+= avail.y-p->size.y   ; break;
  }
```

7.3.4 Packing the children

⟨pl_setrect() *locals* 25a⟩≡ (24a) 26a ▷

```
  Panel *c;
  Vector space;
  Point newul;
  Vector newspace;
```

⟨pl_setrect() *setting the rectangle of the children* 25b⟩≡ (24a)

```
  space=p->size;
  // widget-specific adjustment method
  p->childspace(p, &ul, &space);

  ⟨pl_setrect() before looping over children, set locals 26b⟩
  for(c=p->child;c;c=c->next){
      ⟨pl_setrect() when looping over children, if EXPAND flag 26c⟩
      switch(c->flags&PACK){
      ⟨pl_setrect() when looping over children, switch packing cases 25e⟩
      }
      ul=newul;
      space=newspace;
  }
```

⟨Panel *packing methods* 25c⟩+≡ (11a) ◁ 23a

```
  /* child ul & size given our size */
  void (*childspace)(Panel *, Point * /**INOUT**/, Vector * /**INOUT**/);
```

⟨pl_newpanel() *set default methods* 25d⟩+≡ (11b) ◁ 23b 49h ▷

```
  v->childspace=pl_childspaceerror;
```

⟨pl_setrect() *when looping over children, switch packing cases* 25e⟩≡ (25b) 25f ▷

```
  case PACKN:
      newul=Pt(ul.x, ul.y+c->sizereq.y);
      newspace=Pt(space.x, space.y-c->sizereq.y);
      pl_setrect(c, ul, Pt(space.x, c->sizereq.y));
      break;
  case PACKS:
      newul=ul;
      newspace=Pt(space.x, space.y-c->sizereq.y);
      pl_setrect(c, Pt(ul.x, ul.y+space.y-c->sizereq.y),
          Pt(space.x, c->sizereq.y));
      break;
```

⟨pl_setrect() *when looping over children, switch packing cases* 25f⟩+≡ (25b) ◁ 25e

```
  case PACKW:
      newul=Pt(ul.x+c->sizereq.x, ul.y);
      newspace=Pt(space.x-c->sizereq.x, space.y);
      pl_setrect(c, ul, Pt(c->sizereq.x, space.y));
      break;
  case PACKE:
      newul=ul;
      newspace=Pt(space.x-c->sizereq.x, space.y);
      pl_setrect(c, Pt(ul.x+space.x-c->sizereq.x, ul.y),
          Pt(c->sizereq.x, space.y));
      break;
```

Expanding child

⟨pl_setrect() *locals* 26a⟩+≡ (24a) ◁25a

```
Vector slack, share;
int l;
```

⟨pl_setrect() *before looping over children, set locals* 26b⟩≡ (25b)

```
slack=subpt(space, p->childreq);
share=pl_getshare(p->child);
```

⟨pl_setrect() *when looping over children, if* EXPAND *flag* 26c⟩≡ (25b)

```
if(c->flags&EXPAND){
    switch(c->flags&PACK){
    case PACKN:
    case PACKS:
        c->sizereq.x+=slack.x;
        l=slack.y/share.y;
        c->sizereq.y+=l;
        slack.y-=l;
        --share.y;
        break;
    case PACKE:
    case PACKW:
        l=slack.x/share.x;
        c->sizereq.x+=l;
        slack.x-=l;
        --share.x;
        c->sizereq.y+=slack.y;
        break;
    }
}
```

⟨*function* pl_getshare 26d⟩≡ (103b)

```
Point pl_getshare(Panel *p){
    Point share;

    if(p==nil)
        return Pt(0,0);

    // recurse
    share=pl_getshare(p->next);

    if(p->flags&EXPAND) {
        switch(p->flags&PACK){
        case PACKN:
        case PACKS:
            if(share.x==0) share.x=1;
            share.y++;
            break;
        case PACKE:
        case PACKW:
            share.x++;
            if(share.y==0) share.y=1;
            break;
        }
    }
    return share;
}
```

Chapter 8

Basic Widgets

8.1 Label

⟨*struct* Label 27a⟩≡ (99c)

```
struct Label{
    // enum<Placement> (default = PLACECEN)
    int placement;
    // ref_own<Icon> = ref_own<Image|string> depending on Panel.flags&BITMAP
    Icon *icon;
};
```

⟨*constant* BITMAP 27b⟩≡ (92)

```
#define BITMAP 0x4000  /* text argument is a bitmap, not a string */
```

8.1.1 Initializing

⟨*function* pllabel 27c⟩≡ (99c)

```
Panel *pllabel(Panel *parent, int flags, Icon *icon){
    Panel *p;

    p=pl_newpanel(parent, sizeof(Label));
    plinitlabel(p, flags, icon);
    plplacelabel(p, PLACECEN);
    return p;
}
```

⟨*function* plinitlabel 27d⟩≡ (99c)

```
void plinitlabel(Panel *v, int flags, Icon *icon){
    Label* l = v->data;

    v->flags=flags|LEAF;

    v->draw=pl_drawlabel;
    v->hit=pl_hitlabel;
    v->type=pl_typelabel;

    v->getsize=pl_getsizelabel;
    v->childspace=pl_childspacelabel;

    l->icon=icon;
    //l->placement set in plplacelabel()

    v->kind="label";
}
```

⟨*function* plplacelabel 28a⟩≡ (99c)

```
void plplacelabel(Panel *p, int placement){
    Label* l = p->data;

    l->placement=placement;
}
```

8.1.2 Drawing

⟨*function* pl_drawlabel 28b⟩≡ (99c)

```
void pl_drawlabel(Panel *p){
    Label *l = p->data;

    pl_drawicon(p->b, pl_box(p->b, p->r, PASSIVE),
                l->placement, p->flags, l->icon);
}
```

⟨Style *other cases* 28c⟩≡ (13j) 46d ▷

```
// for labels
PASSIVE,
```

⟨pl_boxoutline() *switch style cases* 28d⟩≡ (18c) 31e ▷

```
case PASSIVE:
    if(fill)
        draw(b, r, pl_light, 0, ZP);
    r=insetrect(r, PWID);
    if(!fill)
        border(b, r, SPACE, pl_white, ZP);
    break;
```

⟨*constant* PWID 28e⟩≡ (98c)

```
#define PWID 1 /* width of label border */
```

⟨*function* pl_drawicon 28f⟩≡ (98c)

```
void pl_drawicon(Image *b, Rectangle r, int stick, int flags, Icon *s){
    Point ul;
    Vector offs;
    ⟨pl_drawicon() other locals 29b⟩

    ul=r.min;
    offs=subpt(subpt(r.max, r.min), pl_iconsize(flags, s));

    switch(stick){
    ⟨pl_drawicon() switch placement cases, adjust ul 29a⟩
    }

    ⟨pl_drawicon() save and adjust clip rectangle 29c⟩
    if(flags&BITMAP)
        draw(b, Rpt(ul, addpt(ul, pl_iconsize(flags, s))), s, nil, ZP);
    else
        string(b, ul, pl_black, ZP, font, s);
    ⟨pl_drawicon() restore saved clip rectangle 29d⟩
}
```

⟨*function* pl_iconsize 28g⟩≡ (98c)

```
Vector pl_iconsize(int flags, Icon *p){
    if(flags&BITMAP)
        return subpt(((Image *)p)->r.max, ((Image *)p)->r.min);
    else
        return stringsize(font, (char *)p);
}
```

⟨pl_drawicon() *switch placement cases, adjust* ul 29a⟩≡ (28f)

```
case PLACENW: /** Nothing **/                 break;
case PLACEN: ul.x+=offs.x/2;                 break;
case PLACENE: ul.x+=offs.x;                   break;
case PLACEW:                 ul.y+=offs.y/2; break;
case PLACECEN: ul.x+=offs.x/2; ul.y+=offs.y/2; break;
case PLACEE: ul.x+=offs.x;                   break;
case PLACESW:                 ul.y+=offs.y;   break;
case PLACES: ul.x+=offs.x/2; ul.y+=offs.y;   break;
case PLACESE: ul.x+=offs.x;   ul.y+=offs.y;   break;
```

⟨pl_drawicon() *other locals* 29b⟩≡ (28f)

```
Rectangle save;
```

⟨pl_drawicon() *save and adjust clip rectangle* 29c⟩≡ (28f)

```
save=b->clipr;
if(!rectclip(&r, save))
    return;
replclipr(b, b->repl, r);
```

⟨pl_drawicon() *restore saved clip rectangle* 29d⟩≡ (28f)

```
replclipr(b, b->repl, save);
```

8.1.3 Reacting

⟨*function* pl_hitlabel 29e⟩≡ (99c)

```
bool pl_hitlabel(Panel *p, Mouse *m){
    USED(p, m);
    return false;
}
```

⟨*function* pl_typelabel 29f⟩≡ (99c)

```
void pl_typelabel(Panel *p, Rune c){
    USED(p, c);
}
```

8.1.4 Packing methods

⟨*function* pl_getsizelabel 29g⟩≡ (99c)

```
Vector pl_getsizelabel(Panel *p, Vector children){
    USED(children);  /* shouldn't have any children */
    return pl_boxsize(pl_iconsize(p->flags, ((Label *)p->data)->icon),
                      PASSIVE);
}
```

⟨pl_boxsize() *switch state cases* 29h⟩≡ (18e) 34b ▷

```
case PASSIVE:
    return addpt(interior, Pt(2*(PWID+SPACE), 2*(PWID+SPACE)));
```

⟨*function* pl_childspacelabel 29i⟩≡ (99c)

```
void pl_childspacelabel(Panel *g, Point *ul, Vector *size){
    USED(g, ul, size);
}
```

8.2 Text entry

⟨*struct* Entry 30a⟩≡ (100b)

```
struct Entry{
    // ref_own<string>
    char *entry;
    ⟨Entry pointer in entry fields 30b⟩
    Vector minsize;

    void (*hit)(Panel *, char *);
};
```

⟨Entry *pointer in* entry *fields* 30b⟩≡ (30a)

```
// point to \0 in entry
char *entp;
// point to end of entry
char *eent;
```

8.2.1 Initializing

⟨*function* plentry 30c⟩≡ (100b)

```
Panel *plentry(Panel *parent, int flags, int wid, char *str, void (*hit)(Panel *, char *)){
    Panel *v;

    v=pl_newpanel(parent, sizeof(Entry));
    plinitentry(v, flags, wid, str, hit);
    return v;
}
```

⟨*function* plinitentry 30d⟩≡ (100b)

```
void plinitentry(Panel *v, int flags, int wid, char *str, void (*hit)(Panel *, char *)){
    Entry *ep = v->data;
    int elen;

    v->flags=flags|LEAF;

    v->draw=pl_drawentry;
    v->hit=pl_hitentry;
    v->type=pl_typeentry;

    v->getsize=pl_getsizeentry;
    v->childspace=pl_childspaceentry;

    ⟨plinitentry() set snarf methods 83c⟩
    ⟨plinitentry() set extra fields 31c⟩
    ⟨plinitentry() set fields in ep 31a⟩

    v->free=pl_freeentry;

    v->kind="entry";
}
```

⟨*function* pl_freeentry 30e⟩≡ (100b)

```
void pl_freeentry(Panel *p){
    Entry *ep = p->data;

    free(ep->entry);
    ep->entry = ep->eent = ep->entp = nil;
}
```

⟨plinitentry() *set fields in* ep 31a⟩≡ (30d)

```
ep->minsize=Pt(wid, font->height);

elen=100;
if(str)
    elen+=strlen(str);
ep->entry=pl_erealloc(ep->entry, elen+SLACK);

ep->eent=ep->entry+elen;
strecpy(ep->entry, ep->eent, str ? str : "");
ep->entp=ep->entry+strlen(ep->entry);

ep->hit=hit;
```

⟨*constant* SLACK*(*(lib_gui/libpanel/textwin.c)*)* 31b⟩≡ (119a)

```
#define SLACK 100
```

8.2.2 Drawing

⟨plinitentry() *set extra fields* 31c⟩≡ (30d)

```
v->state=UP;
```

⟨*function* pl_drawentry 31d⟩≡ (100b)

```
void pl_drawentry(Panel *p){
    Entry *ep = p->data;
    char *s = ep->entry;
    Rectangle r;

    r=pl_box(p->b, p->r, p->state);

    ⟨pl_drawentry if USERFL 32b⟩
    if(stringwidth(font, s) <= r.max.x-r.min.x)
        pl_drawicon(p->b, r, PLACEW, NOFLAG, s);
    else
        pl_drawicon(p->b, r, PLACEE, NOFLAG, s);
    ⟨pl_drawentry when USERFL if s changed 32c⟩
}
```

⟨pl_boxoutline() *switch style cases* 31e⟩+≡ (18c) ◁28d 31g▷

```
case UP:
    pl_relief(b, pl_white, pl_black, r, BWID);
    r=insetrect(r, BWID);
    if(fill)
        draw(b, r, pl_light, nil, ZP);
    else
        border(b, r, SPACE, pl_white, ZP);
    break;
```

⟨*constant* BWID 31f⟩≡ (98c)

```
#define BWID 1 /* width of button relief */
```

⟨pl_boxoutline() *switch style cases* 31g⟩+≡ (18c) ◁31e 46e▷

```
case DOWN:
case DOWN1:
case DOWN2:
case DOWN3:
    pl_relief(b, pl_black, pl_white, r, BWID);
    r=insetrect(r, BWID);
    if(fill)
```

```
        draw(b, r, pl_dark, 0, ZP);
    else
        border(b, r, SPACE, pl_black, ZP);
    break;
```

Drawing secret text

⟨*constant* USERFL 32a⟩≡ (92)

```
#define USERFL 0x100000 /* start of user flag */
```

⟨pl_drawentry *if* USERFL 32b⟩≡ (31d)

```
if(p->flags & USERFL){
    char *p;
    s=strdup(s);
    for(p=s; *p; p++)
        *p='*';
}
```

⟨pl_drawentry *when* USERFL *if* s *changed* 32c⟩≡ (31d)

```
if(s != ep->entry)
    free(s);
```

8.2.3 Reacting

⟨*function* pl_hitentry 32d⟩≡ (100b)

```
bool pl_hitentry(Panel *p, Mouse *m){
    if((m->buttons&7)==CLICK_LEFT){
        p->state=DOWN;
        plgrabkb(p);
        // redraw with changed state
        pldraw(p, p->b);

        while(m->buttons&CLICK_LEFT){
            int old = m->buttons;
            // next mouse event
            *m=emouse();
            ⟨pl_hitentry() handle copy/paste when middle or right click 84b⟩
        }
        // redraw with changed state
        p->state=UP;
        pldraw(p, p->b);
    }
    return false;
}
```

⟨*function* pl_typeentry 32e⟩≡ (100b)

```
void pl_typeentry(Panel *p, Rune c){
    Entry *ep = p->data;
    int n;

    switch(c){
    ⟨pl_typeentry() switch rune cases 33a⟩
    }
    // draw updated entry
    pldraw(p, p->b);
}
```

⟨pl_typeentry() *switch rune cases* 33a⟩≡ (32e) 33b ▷

```
case '\n':
case '\r':
    *ep->entp='\0';
    if(ep->hit)
        ep->hit(p, ep->entry);
    return;
```

⟨pl_typeentry() *switch rune cases* 33b⟩+≡ (32e) ◁ 33a 33e ▷

```
default:
    ⟨pl_typeentry() handle special characters 33d⟩
    ep->entp+=runetochar(ep->entp, &c);
    ⟨pl_typeentry() realloc entry if necessary 33c⟩
    *ep->entp='\0';
    break;
```

⟨pl_typeentry() *realloc entry if necessary* 33c⟩≡ (33b)

```
if(ep->entp>ep->eent){
    n=ep->entp-ep->entry;
    ep->entry=pl_erealloc(ep->entry, n+100+SLACK);
    ep->entp=ep->entry+n;
    ep->eent=ep->entp+100;
}
```

Special keys

⟨pl_typeentry() *handle special characters* 33d⟩≡ (33b)

```
if(c < 0x20 || (c & 0xFF00) == KF || (c & 0xFF00) == Spec)
    break;
```

⟨pl_typeentry() *switch rune cases* 33e⟩+≡ (32e) ◁ 33b 33f ▷

```
case Kesc:
    plsnarf(p);
    /* no break */
case Kdel: /* clear */
case Kbs: /* ^H: erase character */
    while(ep->entp!=ep->entry && !pl_rune1st(ep->entp[-1]))
        *--ep->entp='\0';
    if(ep->entp!=ep->entry)
        *--ep->entp='\0';
    break;
```

⟨pl_typeentry() *switch rune cases* 33f⟩+≡ (32e) ◁ 33e

```
// case Knack: /* ^U: erase line */
//  ep->entp=ep->entry;
//  *ep->entp='\0';
//  break;
// case Ketb: /* ^W: erase word */
//  while(ep->entp!=ep->entry && !pl_idchar(ep->entp[-1]))
//   --ep->entp;
//  while(ep->entp!=ep->entry && pl_idchar(ep->entp[-1]))
//   --ep->entp;
//  *ep->entp='\0';
//  break;
```

8.2.4 Packing methods

⟨*function* pl_getsizeentry 34a⟩≡ (100b)

```
Vector pl_getsizeentry(Panel *p, Vector children){
    Entry* e = p->data;

    USED(children);
    return pl_boxsize(e->minsize, p->state);
}
```

⟨pl_boxsize() *switch state cases* 34b⟩+≡ (18e) ◁29h 47e▷

```
case UP:
case DOWN:
case DOWN1:
case DOWN2:
case DOWN3:
    return addpt(interior, Pt(2*(BWID+SPACE), 2*(BWID+SPACE)));
```

⟨*function* pl_childspaceentry 34c⟩≡ (100b)

```
void pl_childspaceentry(Panel *p, Point *ul, Vector *size){
    USED(p, ul, size);
}
```

8.2.5 Other methods

⟨*function* plentryval 34d⟩≡ (100b)

```
char *plentryval(Panel *p){
    Entry *ep = p->data;

    *ep->entp='\0';
    return ep->entry;
}
```

8.3 Button

8.3.1 Button

⟨*struct* Button 34e⟩≡ (100a)

```
struct Button{
    // enum<ButtonType>
    int btype;   /* button type */

    Icon *icon;   /* what to write on the button */

    ⟨Button other fields 35b⟩

    void (*pl_buttonhit)(Panel *, buttons); /* call back user code on button hit */
    void (*hit)(Panel *, buttons, bool); /* call back user code on check/radio hit */
    ⟨Button other methods 59b⟩
};
```

⟨*constant* BUTTON 34f⟩≡ (100a)

```
#define BUTTON 1
```

⟨*constant* CHECK 34g⟩≡ (100a)

```
#define CHECK 2
```

⟨*constant* RADIO 35a⟩≡ (100a)

```
#define RADIO 3
```

⟨Button *other fields* 35b⟩≡ (34e) 35c ▷

```
// state of buttons when hit
buttons buttons;
```

⟨Button *other fields* 35c⟩+≡ (34e) ◁35b 59a ▷

```
bool check;   /* for check/radio buttons */
```

⟨*function* pl_initbtype 35d⟩≡ (100a)

```
void pl_initbtype(Panel *v, int flags, Icon *icon, void (*hit)(Panel *, buttons, bool), int btype){
    Button *bp = v->data;

    v->flags=flags|LEAF;
    v->state=UP;

    v->draw=pl_drawbutton;
    v->hit=pl_hitbutton;
    v->type=pl_typebutton;

    v->getsize=pl_getsizebutton;
    v->childspace=pl_childspacebutton;

    bp->btype=btype;
    bp->check=false;
    bp->hit=hit;
    bp->icon=icon;

    switch(btype){
    case BUTTON: v->kind="button"; break;
    case CHECK:  v->kind="checkbutton"; break;
    case RADIO:  v->kind="radiobutton"; break;
    }
}
```

8.3.2 Basic button

Initializing

⟨*function* plbutton 35e⟩≡ (100a)

```
Panel *plbutton(Panel *parent, int flags, Icon *icon, void (*hit)(Panel *, buttons)){
    Panel *p;

    p=pl_newpanel(parent, sizeof(Button));
    plinitbutton(p, flags, icon, hit);
    return p;
}
```

⟨*function* plinitbutton 35f⟩≡ (100a)

```
void plinitbutton(Panel *p, int flags, Icon *icon, void (*hit)(Panel *, buttons)){
    Button* b = p->data;

    b->pl_buttonhit=hit;
    pl_initbtype(p, flags, icon, pl_buttonhit, BUTTON);
}
```

⟨*function* pl_buttonhit 36a⟩≡ (100a)

```
void pl_buttonhit(Panel *p, buttons buttons, bool check){
    Button* b = p->data;

    USED(check);
    if(b->pl_buttonhit)
        b->pl_buttonhit(p, buttons);
}
```

Drawing

⟨*function* pl_drawbutton 36b⟩≡ (100a)

```
void pl_drawbutton(Panel *p){
    Rectangle r;
    Button *bp = p->data;

    r=pl_box(p->b, p->r, p->state);
    switch(bp->btype){
    ⟨pl_drawbutton() switch button type cases 36c⟩
    }
    pl_drawicon(p->b, r, PLACECEN, p->flags, bp->icon);
}
```

⟨pl_drawbutton() *switch button type cases* 36c⟩≡ (36b) 37f▷

```
case BUTTON:
    break;
```

Reacting

⟨*function* pl_hitbutton 36d⟩≡ (100a)

```
bool pl_hitbutton(Panel *p, Mouse *m){
    Button *bp = p->data;
    int oldstate = p->state;
    bool hitme;
    ⟨pl_hitbutton() other locals 40b⟩

    if(m->buttons&OUT){ // mouse leaving the widget
        p->state=UP;
        hitme=false;
    }
    else if(m->buttons&7){ // mouse click inside
        p->state=DOWN;
        hitme=false;
        bp->buttons=m->buttons; // remember for the release event
    }
    else{ /* mouse inside, but no buttons down */ // possibly a release
        hitme=p->state==DOWN;
        p->state=UP;
    }

    if(hitme) {
        switch(bp->btype){
        ⟨pl_hitbutton() switch button type cases 38f⟩
        }
    }
    if(hitme || oldstate!=p->state)
        pldraw(p, p->b);
    if(hitme && bp->hit){
```

```
        // user callback
        bp->hit(p, bp->buttons, bp->check);
        p->state=UP;
    }
    return false;
}
```

⟨*function* pl_typebutton 37a⟩≡ (100a)

```
void pl_typebutton(Panel *g, Rune c){
    USED(g, c);
}
```

Packing methods

⟨*function* pl_getsizebutton 37b⟩≡ (100a)

```
Vector pl_getsizebutton(Panel *p, Vector children){
    Button *bp = p->data;
    Vector s;
    ⟨pl_getsizebutton() other locals 38g⟩

    USED(children);  /* shouldn't have any children */
    s=pl_iconsize(p->flags, bp->icon);
    ⟨pl_getsizebutton() if not a BUTTON 38h⟩
    return pl_boxsize(s, p->state);
}
```

⟨*function* pl_childspacebutton 37c⟩≡ (100a)

```
void pl_childspacebutton(Panel *g, Point *ul, Vector *size){
    USED(g, ul, size);
}
```

8.3.3 Check button

Initializing

⟨*function* plcheckbutton 37d⟩≡ (100a)

```
Panel *plcheckbutton(Panel *parent, int flags, Icon *icon, void (*hit)(Panel *, buttons, bool)){
    Panel *p;

    p=pl_newpanel(parent, sizeof(Button));
    plinitcheckbutton(p, flags, icon, hit);
    return p;
}
```

⟨*function* plinitcheckbutton 37e⟩≡ (100a)

```
void plinitcheckbutton(Panel *p, int flags, Icon *icon, void (*hit)(Panel *, buttons, bool)){
    pl_initbtype(p, flags, icon, hit, CHECK);
}
```

Drawing

⟨pl_drawbutton() *switch button type cases* 37f⟩+≡ (36b) ◁36c 39f▷

```
case CHECK:
    r=pl_check(p->b, r, bp->check);
    break;
```

⟨*function* pl_check 38a⟩≡ (98c)

```
Rectangle pl_check(Image *b, Rectangle r, bool val){
    Rectangle remainder = r;

    r.max.x=r.min.x+r.max.y-r.min.y;
    remainder.min.x=r.max.x;
    r=insetrect(r, CKINSET);
    ⟨pl_check() if null plldepth part1 ??⟩
    else
        pl_relief(b, pl_black, pl_white, r, CKWID);
    r=insetrect(r, CKWID);
    ⟨pl_check() if null plldepth part2 ??⟩
    else
        draw(b, r, pl_light, nil, ZP);
    r=insetrect(r, CKBORDER);
    if(val){
        ⟨pl_check() if checked button 38e⟩
    }
    return remainder;
}
```

⟨*constant* CKINSET 38b⟩≡ (98c)

```
#define CKINSET 1 /* space around check mark frame */
```

⟨*constant* CKWID 38c⟩≡ (98c)

```
#define CKWID 1 /* width of frame around check mark */
```

⟨*constant* CKBORDER 38d⟩≡ (98c)

```
#define CKBORDER 2 /* space around X inside frame */
```

⟨pl_check() *if checked button* 38e⟩≡ (38a)

```
line(b, Pt(r.min.x,   r.min.y+1), Pt(r.max.x-1, r.max.y  ), Endsquare, Endsquare, 0, pl_black, ZP);
line(b, Pt(r.min.x,   r.min.y  ), Pt(r.max.x,   r.max.y  ), Endsquare, Endsquare, 0, pl_black, ZP);
line(b, Pt(r.min.x+1, r.min.y  ), Pt(r.max.x,   r.max.y-1), Endsquare, Endsquare, 0, pl_black, ZP);
line(b, Pt(r.min.x  , r.max.y-2), Pt(r.max.x-1, r.min.y-1), Endsquare, Endsquare, 0, pl_black, ZP);
line(b, Pt(r.min.x,   r.max.y-1), Pt(r.max.x,   r.min.y-1), Endsquare, Endsquare, 0, pl_black, ZP);
line(b, Pt(r.min.x+1, r.max.y-1), Pt(r.max.x,   r.min.y  ), Endsquare, Endsquare, 0, pl_black, ZP);
```

Reacting

⟨pl_hitbutton() *switch button type cases* 38f⟩≡ (36d) 40c ▷

```
case CHECK:
    if(hitme)
        bp->check=!bp->check;
    break;
```

Packing methods

⟨pl_getsizebutton() *other locals* 38g⟩≡ (37b)

```
int ckw;
```

⟨pl_getsizebutton() *if not a* BUTTON 38h⟩≡ (37b)

```
if(bp->btype!=BUTTON){
    ckw=pl_ckwid();
    if(s.y<ckw){
        s.x+=ckw;
        s.y=ckw;
    }
    else s.x+=s.y;
}
```

⟨*function* pl_ckwid 39a⟩≡ (98c)

```
int pl_ckwid(void){
    return 2*(CKINSET+CKSPACE+CKWID)+CKSIZE;
}
```

⟨*constant* CKSIZE 39b⟩≡ (98c)

```
#define CKSIZE 3 /* size of check mark */
```

Other methods

⟨*function* plsetbutton 39c⟩≡ (100a)

```
void plsetbutton(Panel *p, int val){
    Button* b = p->data;

    b->check=val;
}
```

8.3.4 Radio button

Initializing

⟨*function* plradiobutton 39d⟩≡ (100a)

```
Panel *plradiobutton(Panel *parent, int flags, Icon *icon, void (*hit)(Panel *, buttons, bool)){
    Panel *p;

    p=pl_newpanel(parent, sizeof(Button));
    plinitradiobutton(p, flags, icon, hit);
    return p;
}
```

⟨*function* plinitradiobutton 39e⟩≡ (100a)

```
void plinitradiobutton(Panel *p, int flags, Icon *icon, void (*hit)(Panel *, buttons, bool)){
    pl_initbtype(p, flags, icon, hit, RADIO);
}
```

Drawing

⟨pl_drawbutton() *switch button type cases* 39f⟩+≡ (36b) ◁37f

```
case RADIO:
    r=pl_radio(p->b, r, bp->check);
    break;
```

⟨*function* pl_radio 39g⟩≡ (98c)

```
/*
 * Place a check mark at the left end of r.  Return the unused space.
 * Caller must guarantee that r.max.x-r.min.x>=r.max.y-r.min.y!
 */
Rectangle pl_radio(Image *b, Rectangle r, bool val){
    Rectangle remainder = r;

    r.max.x=r.min.x+r.max.y-r.min.y;
    remainder.min.x=r.max.x;
    r=insetrect(r, CKINSET);
    ⟨pl_radio() if null plldepth part1 ??⟩
    else
        pl_relief(b, pl_black, pl_white, r, CKWID);
    r=insetrect(r, CKWID);
    ⟨pl_radio() if null plldepth part2 ??⟩
```

```
    else
        draw(b, r, pl_light, 0, ZP);
    if(val)
        draw(b, insetrect(r, CKSPACE), pl_black, nil, ZP);
    return remainder;
}
```

⟨*constant* CKSPACE 40a⟩≡ (98c)

```
#define CKSPACE 2 /* space around check mark */
```

Reacting

⟨pl_hitbutton() *other locals* 40b⟩≡ (36d)

```
Panel *sib;
```

⟨pl_hitbutton() *switch button type cases* 40c⟩+≡ (36d) ◁38f

```
case RADIO:
    if(bp->check)
        bp->check=false;
    else{
        if(p->parent){
            for(sib=p->parent->child;sib;sib=sib->next){
                if(sib->hit==pl_hitbutton
                && ((Button *)sib->data)->btype==RADIO
                && ((Button *)sib->data)->check){
                    ((Button *)sib->data)->check=false;
                    pldraw(sib, p->b);
                }
            }
        }
        bp->check=true;
    }
    break;
```

8.4 Slider

⟨*struct* Slider 40d⟩≡ (101b)

```
struct Slider{
    // enum<Direction>
    int dir;   /* HORIZ or VERT */
    Vector minsize; // range

    int val;   /* setting, in screen coordinates */
    ⟨Slider other fields 40e⟩

    void (*hit)(Panel *, buttons, int, int); /* call back to user when slider changes */
};
```

⟨Slider *other fields* 40e⟩≡ (40d)

```
buttons buttons;
```

8.4.1 Initializing

⟨*function* plslider 41a⟩≡ (101b)

```
Panel *plslider(Panel *parent, int flags, Vector size, void (*hit)(Panel *, buttons, int, int)){
    Panel *p;

    p=pl_newpanel(parent, sizeof(Slider));
    plinitslider(p, flags, size, hit);
    return p;
}
```

⟨*function* plinitslider 41b⟩≡ (101b)

```
void plinitslider(Panel *v, int flags, Vector size, void (*hit)(Panel *, buttons, int, int)){
    Slider *sp = v->data;

    v->flags=flags|LEAF;
    v->state=UP;

    v->draw=pl_drawslider;
    v->hit=pl_hitslider;
    v->type=pl_typeslider;

    v->getsize=pl_getsizeslider;
    v->childspace=pl_childspaceslider;

    v->r=Rect(0,0,size.x,size.y);

    sp->minsize=size;
    sp->dir=size.x>size.y?HORIZ:VERT;
    sp->hit=hit;

    v->kind="slider";
}
```

8.4.2 Drawing

⟨*function* pl_drawslider 41c⟩≡ (101b)

```
void pl_drawslider(Panel *p){
    Rectangle r;
    Slider *sp = p->data;

    r=pl_box(p->b, p->r, UP);
    switch(sp->dir){
    case HORIZ: pl_sliderupd(p->b, r, sp->dir, 0, sp->val); break;
    case VERT:  pl_sliderupd(p->b, r, sp->dir, r.max.y-sp->val, r.max.y); break;
    }
}
```

⟨*function* pl_sliderupd 41d⟩≡ (98c)

```
void pl_sliderupd(Image *b, Rectangle r1, int dir, int lo, int hi){
    Rectangle r2, r3;

    r2=r1;
    r3=r1;

    ⟨pl_sliderupd() sanitize lo and hi 42a⟩

    switch(dir){
    case HORIZ:
```

```
        r1.max.x=r1.min.x+lo;
        r2.min.x=r1.max.x;
        r2.max.x=r1.min.x+hi;
        if(r2.max.x>r3.max.x) r2.max.x=r3.max.x;
        r3.min.x=r2.max.x;
        break;
    case VERT:
        r1.max.y=r1.min.y+lo;
        r2.min.y=r1.max.y;
        r2.max.y=r1.min.y+hi;
        if(r2.max.y>r3.max.y) r2.max.y=r3.max.y;
        r3.min.y=r2.max.y;
        break;
    }
    draw(b, r1, pl_light, nil, ZP);
    draw(b, r2, pl_dark, nil, ZP);
    draw(b, r3, pl_light, nil, ZP);
}
```

⟨pl_sliderupd() *sanitize* lo *and* hi 42a⟩≡ (41d)

```
if(lo<0) lo=0;
if(hi<=lo) hi=lo+1;
```

8.4.3 Reacting

⟨*function* pl_hitslider 42b⟩≡ (101b)

```
bool pl_hitslider(Panel *p, Mouse *m){
    Slider *sp = p->data;
    int oldstate, oldval;
    int len;
    Point ul;
    Vector size;
    SET(len);

    ul=p->r.min;
    size=subpt(p->r.max, p->r.min);
    pl_interior(p->state, &ul, &size);

    oldstate=p->state;
    oldval=sp->val;

    if(m->buttons&OUT)
        p->state=UP;
    else if(m->buttons&7){
        p->state=DOWN;
        sp->buttons=m->buttons;
        ⟨pl_hitslider() when button, set sp->val and len 43a⟩
        ⟨pl_hitslider() sanitize sp->val 43b⟩
    }
    else /* mouse inside, but no buttons down */
        p->state=UP;

    if(oldval!=sp->val || oldstate!=p->state)
        pldraw(p, p->b);
    if(oldval!=sp->val && sp->hit)
        // user callback
        sp->hit(p, sp->buttons, sp->val, len);
    return false;
}
```

⟨pl_hitslider() *when button, set* sp->val *and* len 43a⟩≡ (42b)

```
if(sp->dir==HORIZ){
    sp->val=m->xy.x-ul.x;
    len=size.x;
}else{
    sp->val=ul.y+size.y-m->xy.y;
    len=size.y;
}
```

⟨pl_hitslider() *sanitize* sp->val 43b⟩≡ (42b)

```
if(sp->val<0)
    sp->val=0;
else
    if(sp->val>len)
        sp->val=len;
```

⟨*function* pl_typeslider 43c⟩≡ (101b)

```
void pl_typeslider(Panel *p, Rune c){
    USED(p, c);
}
```

8.4.4 Packing methods

⟨*function* pl_getsizeslider 43d⟩≡ (101b)

```
Vector pl_getsizeslider(Panel *p, Vector children){
    USED(children);
    return pl_boxsize(((Slider *)p->data)->minsize, p->state);
}
```

⟨*function* pl_childspaceslider 43e⟩≡ (101b)

```
void pl_childspaceslider(Panel *g, Point *ul, Vector *size){
    USED(g, ul, size);
}
```

8.4.5 Other methods

⟨*function* plsetslider 43f⟩≡ (101b)

```
void plsetslider(Panel *p, int value, int range){
    Slider *sp = p->data;

    ⟨plsetslider() sanitize value 43g⟩
    if(sp->dir==HORIZ)
        sp->val=value*(p->r.max.x-p->r.min.x)/range;
    else
        sp->val=value*(p->r.max.y-p->r.min.y)/range;
}
```

⟨plsetslider() *sanitize* value 43g⟩≡ (43f)

```
if(value<0) value=0;
else if(value>range) value=range;
```

8.5 Canvas

⟨*struct* Canvas 43h⟩≡ (102a)

```
struct Canvas{
    void (*draw)(Panel *);
    void (*hit)(Panel *, Mouse *);
};
```

8.5.1 Initializing

⟨*function* plcanvas 44a⟩≡ (102a)

```
Panel *plcanvas(Panel *parent, int flags, void (*draw)(Panel *), void (*hit)(Panel *, Mouse *)){
    Panel *p;

    p=pl_newpanel(parent, sizeof(Canvas));
    plinitcanvas(p, flags, draw, hit);
    return p;
}
```

⟨*function* plinitcanvas 44b⟩≡ (102a)

```
void plinitcanvas(Panel *v, int flags, void (*draw)(Panel *), void (*hit)(Panel *, Mouse *)){
    Canvas *c = v->data;;

    v->flags=flags|LEAF;

    v->draw=pl_drawcanvas;
    v->hit=pl_hitcanvas;
    v->type=pl_typecanvas;

    v->getsize=pl_getsizecanvas;
    v->childspace=pl_childspacecanvas;

    c->draw=draw;
    c->hit=hit;

    v->kind="canvas";
}
```

8.5.2 Drawing

⟨*function* pl_drawcanvas 44c⟩≡ (102a)

```
void pl_drawcanvas(Panel *p){
    Canvas *c = p->data;

    if(c->draw)
        c->draw(p);
}
```

8.5.3 Reacting

⟨*function* pl_hitcanvas 44d⟩≡ (102a)

```
int pl_hitcanvas(Panel *p, Mouse *m){
    Canvas *c =p->data;

    if(c->hit)
        c->hit(p, m);
    return false;
}
```

⟨*function* pl_typecanvas 44e⟩≡ (102a)

```
void pl_typecanvas(Panel *p, Rune c){
    USED(p, c);
}
```

8.5.4 Packing methods

⟨*function* pl_getsizecanvas 45a⟩≡ (102a)

```
Vector pl_getsizecanvas(Panel *p, Vector children){
    USED(p, children);
    return Pt(0,0);
}
```

⟨*function* pl_childspacecanvas 45b⟩≡ (102a)

```
void pl_childspacecanvas(Panel *p, Point *ul, Vector *size){
    USED(p, ul, size);
}
```

Chapter 9

Composite Widgets

9.1 Frame

9.1.1 Initializing

⟨*function* plframe 46a⟩≡ (102b)

```
Panel *plframe(Panel *parent, int flags){
    Panel *p;

    p=pl_newpanel(parent, 0); // no widget-specific data
    plinitframe(p, flags);
    return p;
}
```

⟨*function* plinitframe 46b⟩≡ (102b)

```
void plinitframe(Panel *v, int flags){
    v->flags=flags;

    v->draw=pl_drawframe;
    v->hit=pl_hitframe;
    v->type=pl_typeframe;

    v->getsize=pl_getsizeframe;
    v->childspace=pl_childspaceframe;

    v->kind="frame";
}
```

9.1.2 Drawing

⟨*function* pl_drawframe 46c⟩≡ (102b)

```
void pl_drawframe(Panel *p){
    pl_box(p->b, p->r, FRAME);
}
```

⟨Style *other cases* 46d⟩+≡ (13j) ◁28c 61a▷

```
// for frames
FRAME
```

⟨pl_boxoutline() *switch style cases* 46e⟩+≡ (18c) ◁31g

```
case FRAME:
    pl_relief(b, pl_white, pl_black, r, FWID);
    r=insetrect(r, FWID);
    pl_relief(b, pl_black, pl_white, r, FWID);
```

```
    r=insetrect(r, FWID);
    if(fill)
        draw(b, r, pl_light, nil, ZP);
    else
        border(b, r, SPACE, pl_white, ZP);
    break;
```

⟨*constant* FWID 47a⟩≡ (98c)

```
#define FWID 2 /* width of frame relief */
```

9.1.3 Reacting

⟨*function* pl_hitframe 47b⟩≡ (102b)

```
bool pl_hitframe(Panel *p, Mouse *m){
    USED(p, m);
    return false;
}
```

⟨*function* pl_typeframe 47c⟩≡ (102b)

```
void pl_typeframe(Panel *p, Rune c){
    USED(p, c);
}
```

9.1.4 Packing methods

⟨*function* pl_getsizeframe 47d⟩≡ (102b)

```
Vector pl_getsizeframe(Panel *p, Vector children){
    USED(p);
    return pl_boxsize(children, FRAME);
}
```

⟨pl_boxsize() *switch state cases* 47e⟩+≡ (18e) ◁34b

```
case FRAME:
    return addpt(interior, Pt(4*FWID+2*SPACE, 4*FWID+2*SPACE));
```

⟨*function* pl_childspaceframe 47f⟩≡ (102b)

```
void pl_childspaceframe(Panel *p, Point *ul, Vector *size){
    USED(p);
    pl_interior(FRAME, ul, size);
}
```

⟨pl_interior() *switch state cases* 47g⟩+≡ (18f) ◁18h

```
case FRAME:
    *ul=addpt(*ul, Pt(2*FWID+SPACE, 2*FWID+SPACE));
    *size=subpt(*size, Pt(4*FWID+2*SPACE, 4*FWID+2*SPACE));
```

9.2 Group

9.2.1 Initializing

⟨*function* plgroup 47h⟩≡ (102c)

```
Panel *plgroup(Panel *parent, int flags){
    Panel *p;

    p=pl_newpanel(parent, 0);
    plinitgroup(p, flags);
    return p;
}
```

⟨*function* plinitgroup 48a⟩≡ (102c)

```
void plinitgroup(Panel *v, int flags){
    v->flags=flags;

    v->draw=pl_drawgroup;
    v->hit=pl_hitgroup;
    v->type=pl_typegroup;

    v->getsize=pl_getsizegroup;
    v->childspace=pl_childspacegroup;

    v->kind="group";
}
```

9.2.2 Drawing

⟨*function* pl_drawgroup 48b⟩≡ (102c)

```
void pl_drawgroup(Panel *p){
    USED(p);
}
```

9.2.3 Reacting

⟨*function* pl_hitgroup 48c⟩≡ (102c)

```
bool pl_hitgroup(Panel *p, Mouse *m){
    USED(p, m);
    return false;
}
```

⟨*function* pl_typegroup 48d⟩≡ (102c)

```
void pl_typegroup(Panel *p, Rune c){
    USED(p, c);
}
```

9.2.4 Packing methods

⟨*function* pl_getsizegroup 48e⟩≡ (102c)

```
Vector pl_getsizegroup(Panel *p, Vector children){
    USED(p);
    return children;
}
```

⟨*function* pl_childspacegroup 48f⟩≡ (102c)

```
void pl_childspacegroup(Panel *p, Point *ul, Vector *size){
    USED(p, ul, size);
}
```

Chapter 10

Scrollable Widgets

10.1 Scrolling fields

⟨Panel *other fields* 49a⟩+≡ (11a) ◁19b 49d▷
```
// option<ref<Panel>>
Panel *scrollee;    /* pointer to scrolled window */
// option<ref<Panel>>
Panel *xscroller, *yscroller;   /* pointers to scroll bars */
```

⟨pl_newpanel() *set other fields* 49b⟩+≡ (11b) ◁22e 49f▷
```
v->scrollee=nil;
v->xscroller=nil;
v->yscroller=nil;
```

⟨*function* plscroll 49c⟩≡ (108c)
```
void plscroll(Panel *scrollee, Panel *xscroller, Panel *yscroller){
    scrollee->xscroller=xscroller;
    scrollee->yscroller=yscroller;
    if(xscroller) xscroller->scrollee=scrollee;
    if(yscroller) yscroller->scrollee=scrollee;
}
```

⟨Panel *other fields* 49d⟩+≡ (11a) ◁49a
```
Scroll scr;     /* scroll data */
```

⟨*struct* Scroll 49e⟩≡ (92)
```
struct Scroll{
    Point pos;
    Vector size;
};
```

⟨pl_newpanel() *set other fields* 49f⟩+≡ (11b) ◁49b
```
v->scr.pos=Pt(0,0);
v->scr.size=Pt(0,0);
```

10.2 Scrolling methods

⟨Panel *other methods* 49g⟩+≡ (11a) ◁20f 82a▷
```
void (*scroll)(Panel *, int, buttons, int, int); /* scroll bar to scrollee */
void (*setscrollbar)(Panel *, int, int, int); /* scrollee to scroll bar */
```

⟨pl_newpanel() *set default methods* 49h⟩+≡ (11b) ◁25d 82b▷
```
v->scroll=pl_scrollerror;
v->setscrollbar=pl_setscrollbarerror;
```

10.3 Scrollable list

⟨*struct* List 50a⟩≡ (102d)

```
struct List{
    int len;   /* # of items in list */
    Vector minsize;

    int lo;    /* indices of first, last items displayed */
    // option<int> (NONE = -1)
    int sel;   /* index of hilited item */
    ⟨List other fields 50b⟩

    char* (*gen)(Panel *, int); /* return text given index or 0 if out of range */
    void (*hit)(Panel *, int, int); /* call user back on hit */

};
```

⟨List *other fields* 50b⟩≡ (50a) 51d ▷

```
buttons buttons;
```

10.3.1 Initializing

⟨*function* pllist 50c⟩≡ (102d)

```
Panel *pllist(Panel *parent, int flags, char *(*gen)(Panel *, int), int nlist, void (*hit)(Panel *, int, int)){
    Panel *v;

    v=pl_newpanel(parent, sizeof(List));
    plinitlist(v, flags, gen, nlist, hit);
    return v;
}
```

⟨*function* plinitlist 50d⟩≡ (102d)

```
void plinitlist(Panel *v, int flags, char *(*gen)(Panel *, int), int nlist, void (*hit)(Panel *, int, int)){
    List *lp = v->data;
    ⟨plinitlist() other locals 50f⟩

    v->flags=flags|LEAF;
    v->state=UP;

    v->draw=pl_drawlist;
    v->hit=pl_hitlist;
    v->type=pl_typelist;

    v->getsize=pl_getsizelist;
    v->childspace=pl_childspacelist;

    ⟨plinitlist() set fields in lp 50e⟩
    ⟨plinitlist() set scrolling fields 53d⟩

    v->kind="list";
}
```

⟨plinitlist() *set fields in* lp 50e⟩≡ (50d) 51a ▷

```
lp->gen=gen;
lp->hit=hit;
```

⟨plinitlist() *other locals* 50f⟩≡ (50d)

```
int wid, max;
char *str;
```

⟨plinitlist() *set fields in* lp 51a⟩+≡ (50d) ◁50e 51c▷

```
⟨plinitlist() if FILLX or EXPAND 51b⟩
else{
    max=0;
    for(lp->len=0;str=gen(v, lp->len);lp->len++){
        wid=stringwidth(font, str);
        if(wid>max) max=wid;
    }
    lp->minsize=Pt(max, nlist*font->height);
}
```

⟨plinitlist() *if* FILLX *or* EXPAND 51b⟩≡ (51a)

```
if(flags&(FILLX|EXPAND)){
    for(lp->len=0;gen(v, lp->len);lp->len++)
       ;
    lp->minsize=Pt(0, nlist*font->height);
}
```

⟨plinitlist() *set fields in* lp 51c⟩+≡ (50d) ◁51a

```
lp->lo=0;
lp->sel=-1;
```

10.3.2 Drawing

⟨List *other fields* 51d⟩+≡ (50a) ◁50b

```
Rectangle listr;
```

⟨*function* pl_drawlist 51e⟩≡ (102d)

```
void pl_drawlist(Panel *p){
    List *lp = p->data;

    lp->listr=pl_box(p->b, p->r, UP);
    pl_liststrings(p, lp->lo, lp->lo+(lp->listr.max.y-lp->listr.min.y)/font->height,
        lp->listr);
}
```

⟨*function* pl_liststrings 51f⟩≡ (102d)

```
void pl_liststrings(Panel *p, int lo, int hi, Rectangle r){
    List *lp = p->data;
    char *s;
    int i;
    ⟨pl_liststrings() other locals 52d⟩

    for(i=lo;i!=hi && (s=lp->gen(p, i));i++){
        r.max.y=r.min.y+font->height;
        pl_drawicon(p->b, r, PLACEW, NOFLAG, s);
        r.min.y+=font->height;
    }
    ⟨pl_liststrings() draw the selection 51g⟩
    ⟨pl_liststrings() set the scrollbar 52e⟩
}
```

⟨pl_liststrings() *draw the selection* 51g⟩≡ (51f)

```
if(lo<=lp->sel && lp->sel<hi)
    pl_listsel(p, lp->sel, true);
```

⟨*function* pl_listsel 52a⟩≡ (102d)

```
void pl_listsel(Panel *p, int sel, bool on){
    List *lp = p->data;
    int hi;
    Rectangle r;

    hi=lp->lo+(lp->listr.max.y-lp->listr.min.y)/font->height;
    if(lp->lo>=0 && lp->lo<=sel && sel<hi && sel<lp->len){
        r=lp->listr;
        r.min.y+=(sel-lp->lo)*font->height;
        r.max.y=r.min.y+font->height;
        if(on)
            pl_highlight(p->b, r);
        else{
            pl_fill(p->b, r);
            pl_drawicon(p->b, r, PLACEW, NOFLAG, lp->gen(p, sel));
        }
    }
}
```

⟨*function* pl_highlight 52b⟩≡ (98c)

```
void pl_highlight(Image *b, Rectangle r){
    draw(b, r, pl_dark, pl_hilit, ZP);
}
```

⟨*function* pl_fill 52c⟩≡ (98c)

```
void pl_fill(Image *b, Rectangle r){
    draw(b, r, plldepth==0? pl_white : pl_light, 0, ZP);
}
```

Scrollee to scrollbar

⟨pl_liststrings() *other locals* 52d⟩≡ (51f)

```
Panel *sb = p->yscroller;
```

⟨pl_liststrings() *set the scrollbar* 52e⟩≡ (51f)

```
if(sb && sb->setscrollbar)
    sb->setscrollbar(sb, lp->lo,
        lp->lo+(lp->listr.max.y-lp->listr.min.y)/font->height, lp->len);
```

10.3.3 Reacting

⟨*function* pl_hitlist 52f⟩≡ (102d)

```
int pl_hitlist(Panel *p, Mouse *m){
    int oldsel;
    bool hitme;
    Point ul;
    Vector size;
    List *lp = p->data;

    hitme=false;
    ul=p->r.min;
    size=subpt(p->r.max, p->r.min);
    pl_interior(p->state, &ul, &size);
    oldsel=lp->sel;

    if(m->buttons&OUT){
        p->state=UP;
```

```
        if(m->buttons&~OUT) lp->sel=-1;
    }
    else if(p->state==DOWN || m->buttons&7){
        lp->sel=(m->xy.y-ul.y)/font->height+lp->lo;
        if(m->buttons&7){
            lp->buttons=m->buttons;
            p->state=DOWN;
        }
        else{ // release
            hitme=true;
            p->state=UP;
        }
    }

    if(oldsel!=lp->sel){
        pl_listsel(p, oldsel, false);
        pl_listsel(p, lp->sel, true);
    }
    if(hitme && 0<=lp->sel && lp->sel<lp->len && lp->hit)
        // user callback
        lp->hit(p, lp->buttons, lp->sel);
    return false;
}
```

⟨*function* pl_typelist 53a⟩≡ (102d)

```
void pl_typelist(Panel *g, Rune c){
    USED(g, c);
}
```

10.3.4 Packing methods

⟨*function* pl_getsizelist 53b⟩≡ (102d)

```
Vector pl_getsizelist(Panel *p, Vector children){
    USED(children);
    return pl_boxsize(((List *)p->data)->minsize, p->state);
}
```

⟨*function* pl_childspacelist 53c⟩≡ (102d)

```
void pl_childspacelist(Panel *g, Point *ul, Vector *size){
    USED(g, ul, size);
}
```

10.3.5 Scrollbar to scrollee

⟨plinitlist() *set scrolling fields* 53d⟩≡ (50d)

```
v->scroll=pl_scrolllist;
v->scr.pos=Pt(0,0);
v->scr.size=Pt(0,lp->len);
```

⟨*function* pl_scrolllist 53e⟩≡ (102d)

```
void pl_scrolllist(Panel *p, int dir, buttons buttons, int val, int len){
    List *lp = p->data;
    Point ul;
    Vector size;
    int nlist, nline;
    int oldlo, hi;
    Rectangle r;
    int y;
```

```
    oldlo=lp->lo;

    ul=p->r.min;
    size=subpt(p->r.max, p->r.min);
    pl_interior(p->state, &ul, &size);

    nlist=size.y/font->height;

    if(dir==VERT) switch(buttons){
    case CLICK_LEFT:    lp->lo-=nlist*val/len; break;
    case CLICK_MIDDLE:  lp->lo=lp->len*val/len; break;
    case CLICK_RIGHT: lp->lo+=nlist*val/len; break;
    }
    ⟨pl_scrolllist() sanitize lp->lo 54a⟩
    if(lp->lo==oldlo)
        return;
    // else

    p->scr.pos.y=lp->lo;
    r=lp->listr;
    nline=(r.max.y-r.min.y)/font->height;
    hi=lp->lo+nline;

    if(hi<=oldlo || lp->lo>=oldlo+nline){
        // =~ pl_drawlist(), complete redraw
        pl_box(p->b, r, PASSIVE); //BUG? forgot r=pl_box(...)
        pl_liststrings(p, lp->lo, hi, r);
    }
    else
        ⟨pl_scrolllist() optimized drawing by copying 54b⟩
}
```

⟨pl_scrolllist() *sanitize* lp->lo 54a⟩≡ (53e)

```
if(lp->lo<0) lp->lo=0;
if(lp->lo>=lp->len) lp->lo=lp->len-1;
```

⟨pl_scrolllist() *optimized drawing by copying* 54b⟩≡ (53e)

```
if(lp->lo<oldlo){
    y=r.min.y+(oldlo-lp->lo)*font->height;
    pl_cpy(p->b, Pt(r.min.x, y),
        Rect(r.min.x, r.min.y, r.max.x, r.min.y+(hi-oldlo)*font->height));
    r.max.y=y;
    pl_box(p->b, r, PASSIVE);
    pl_liststrings(p, lp->lo, oldlo, r);
}
else{
    pl_cpy(p->b, r.min, Rect(r.min.x, r.min.y+(lp->lo-oldlo)*font->height,
        r.max.x, r.max.y));
    r.min.y=r.min.y+(oldlo+nline-lp->lo)*font->height;
    pl_box(p->b, r, PASSIVE);
    pl_liststrings(p, oldlo+nline, hi, r);
}
```

⟨*function* pl_cpy 54c⟩≡ (98c)

```
void pl_cpy(Image *b, Point dst, Rectangle src){
    draw(b, Rpt(dst, addpt(dst, subpt(src.max, src.min))), b, nil, src.min);
}
```

10.4 Scroll bar

⟨*struct* Scrollbar 55a⟩≡ (108d)

```
struct Scrollbar{
    // enum<Direction>
    int dir;  /* HORIZ or VERT */
    Point minsize;

    int lo, hi;  /* setting, in screen coordinates */

    ⟨Scrollbar other fields 55b⟩
};
```

⟨Scrollbar *other fields* 55b⟩≡ (55a) 56c ▷

```
buttons buttons;  /* saved mouse buttons for transmittal to scrollee */
```

10.4.1 Initializing

⟨*function* plscrollbar 55c⟩≡ (108d)

```
Panel *plscrollbar(Panel *parent, int flags){
    Panel *v;

    v=pl_newpanel(parent, sizeof(Scrollbar));
    plinitscrollbar(v, flags);
    return v;
}
```

⟨*function* plinitscrollbar 55d⟩≡ (108d)

```
void plinitscrollbar(Panel *v, int flags){
    Scrollbar *sp = v->data;

    v->flags=flags|LEAF;
    v->state=UP;

    v->draw=pl_drawscrollbar;
    v->hit=pl_hitscrollbar;
    v->type=pl_typescrollbar;

    v->getsize=pl_getsizescrollbar;
    v->childspace=pl_childspacescrollbar;


    ⟨plinitscrollbar() set extra fields 55e⟩

    v->kind="scrollbar";
}
```

⟨plinitscrollbar() *set extra fields* 55e⟩≡ (55d) 55f ▷

```
v->pri=pl_priscrollbar; // !!
```

⟨plinitscrollbar() *set extra fields* 55f⟩+≡ (55d) ◁ 55e 56b ▷

```
switch(flags&PACK){
case PACKN:
case PACKS:
    sp->dir=HORIZ;
    sp->minsize=Pt(0, SBWID);
    v->flags|=FILLX;
    break;
case PACKE:
```

```
  case PACKW:
      sp->dir=VERT;
      sp->minsize=Pt(SBWID, 0);
      v->flags|=FILLY;
      break;
  }
```

⟨*constant* SBWID 56a⟩≡ (108d)

```
  #define SBWID 15 /* should come from draw.c? */
```

⟨plinitscrollbar() *set extra fields* 56b⟩+≡ (55d) ◁55f 58a▷

```
  sp->lo=0;
  sp->hi=0;
```

10.4.2 Drawing

⟨Scrollbar *other fields* 56c⟩+≡ (55a) ◁55b

```
  Rectangle interior;
```

⟨*function* pl_drawscrollbar 56d⟩≡ (108d)

```
  void pl_drawscrollbar(Panel *p){
      Scrollbar *sp = p->data;

      sp->interior=pl_outline(p->b, p->r, p->state);
      pl_sliderupd(p->b, sp->interior, sp->dir, sp->lo, sp->hi);
  }
```

10.4.3 Reacting

⟨*function* pl_hitscrollbar 56e⟩≡ (108d)

```
  int pl_hitscrollbar(Panel *g, Mouse *m){
      int oldstate;
      int pos, len, dy;
      Point ul;
      Vector size;
      Scrollbar *sp = g->data;

      ul=g->r.min;
      size=subpt(g->r.max, g->r.min);
      pl_interior(g->state, &ul, &size);
      oldstate=g->state;

      if(!(g->flags & USERFL) && (m->buttons&OUT || !ptinrect(m->xy, g->r))){
          m->buttons&=~OUT;
          g->state=UP;
          goto out;
      }

      if(sp->dir==HORIZ){
          pos=m->xy.x-ul.x;
          len=size.x;
      }
      else{
          pos=m->xy.y-ul.y;
          len=size.y;
      }
      ⟨pl_hitscrollbar() sanitize pos 57a⟩
```

```
    if(m->buttons&7){
        g->state=DOWN;
        sp->buttons=m->buttons;
        switch(m->buttons){
        ⟨pl_hitscrollbar() switch buttons cases 57b⟩
        }
    }
    else{
        if(!(sp->buttons&CLICK_MIDDLE) && g->state==DOWN &&
           g->scrollee && g->scrollee->scroll)
            // scroll hook
            g->scrollee->scroll(g->scrollee, sp->dir, sp->buttons,
                pos, len);
        g->state=UP;
    }
out:
    if(oldstate!=g->state) pldraw(g, g->b);
    return g->state==DOWN;
}
```

⟨pl_hitscrollbar() *sanitize* pos 57a⟩≡ (56e)

```
if(pos<0) pos=0;
else if(pos>len) pos=len;
```

⟨pl_hitscrollbar() *switch buttons cases* 57b⟩≡ (56e)

```
case CLICK_LEFT:
    dy=pos*(sp->hi-sp->lo)/len;
    pl_sliderupd(g->b, sp->interior, sp->dir, sp->lo-dy,
        sp->hi-dy);
    break;
case CLICK_MIDDLE:
    if(g->scrollee && g->scrollee->scroll)
        g->scrollee->scroll(g->scrollee, sp->dir,
            m->buttons, pos, len);
    break;
case CLICK_RIGHT:
    dy=pos*(sp->hi-sp->lo)/len;
    pl_sliderupd(g->b, sp->interior, sp->dir, sp->lo+dy,
        sp->hi+dy);
    break;
```

⟨*function* pl_typescrollbar 57c⟩≡ (108d)

```
void pl_typescrollbar(Panel *p, Rune c){
    USED(p, c);
}
```

10.4.4 Packing methods

⟨*function* pl_getsizescrollbar 57d⟩≡ (108d)

```
Point pl_getsizescrollbar(Panel *p, Point children){
    USED(children);
    return pl_boxsize(((Scrollbar *)p->data)->minsize, p->state);
}
```

⟨*function* pl_childspacescrollbar 57e⟩≡ (108d)

```
void pl_childspacescrollbar(Panel *p, Point *ul, Vector *size){
    USED(p, ul, size);
}
```

10.4.5 Scrollee to scrollbar

⟨plinitscrollbar() *set extra fields* 58a⟩+≡ (55d) ◁56b

```
v->setscrollbar=pl_setscrollbarscrollbar;
```

⟨*function* pl_setscrollbarscrollbar 58b⟩≡ (108d)

```
/*
 * Arguments lo, hi and len are in the scrollee's natural coordinates
 */
void pl_setscrollbarscrollbar(Panel *p, int lo, int hi, int len){
    Point ul;
    Vector size;
    int mylen;
    Scrollbar *sp = p->data;

    ul=p->r.min;
    size=subpt(p->r.max, p->r.min);
    pl_interior(p->state, &ul, &size);

    mylen=sp->dir==HORIZ?size.x:size.y;

    if(len==0) len=1;
    sp->lo=lo*mylen/len;
    sp->hi=hi*mylen/len;

    ⟨pl_setscrollbarscrollbar() sanitize sp->lo 58c⟩
    ⟨pl_setscrollbarscrollbar() sanitize sp->hi 58d⟩
    pldraw(p, p->b);
}
```

⟨pl_setscrollbarscrollbar() *sanitize* sp->lo 58c⟩≡ (58b)

```
if(sp->lo<0) sp->lo=0;
if(sp->lo>=mylen) sp->hi=mylen-1;
```

⟨pl_setscrollbarscrollbar() *sanitize* sp->hi 58d⟩≡ (58b)

```
if(sp->hi<=sp->lo) sp->hi=sp->lo+1;
if(sp->hi>mylen) sp->hi=mylen;
```

Chapter 11

Menu widgets

11.1 Menu items

⟨Button *other fields* 59a⟩+≡ (34e) ◁35c
```
int index;   /* arg to menuhit */
```

⟨Button *other methods* 59b⟩≡ (34e)
```
void (*menuhit)(buttons, int); /* call back user code on menu item hit */
```

11.1.1 Initializing

⟨*function* plmenu 59c⟩≡ (100a)
```
Panel *plmenu(Panel *parent, int flags, Icon **item, int cflags, void (*hit)(buttons, int)){
    Panel *v;

    v=plgroup(parent, flags);
    plinitmenu(v, flags, item, cflags, hit);
    return v;
}
```

⟨*function* plinitmenu 59d⟩≡ (100a)
```
void plinitmenu(Panel *v, int flags, Icon **item, int cflags, void (*hit)(buttons, int)){
    Panel *p;
    Button* b;
    int i;

    v->flags=flags;

    ⟨plinitmenu() free child widget if any 59e⟩

    for(i=0;item[i];i++){
        p=plbutton(v, cflags, item[i], pl_hitmenu);
        b = p->data;

        b->menuhit=hit;
        b->index=i;
    }
    v->kind="menu";
}
```

⟨plinitmenu() *free child widget if any* 59e⟩≡ (59d)
```
if(v->child){
    plfree(v->child);
    v->child=nil;
}
```

11.1.2 Drawing

11.1.3 Reacting

⟨*function* pl_hitmenu 60a⟩≡ (100a)

```
void pl_hitmenu(Panel *p, buttons buttons){
    Button* b = p->data;
    void (*hit)(int, int) = b->menuhit;

    if(hit)
        hit(buttons, b->index);
}
```

11.2 Popup menu

⟨*struct* Popup 60b⟩≡ (103c)

```
struct Popup{
    Panel *pop[3];   /* what to pop up */

    //option<ref_own<Image>>
    Image *save;   /* where to save what the popup covers */
};
```

11.2.1 Initializing

⟨*function* plpopup 60c⟩≡ (103c)

```
Panel *plpopup(Panel *parent, int flags, Panel *pop0, Panel *pop1, Panel *pop2){
    Panel *v;

    v=pl_newpanel(parent, sizeof(Popup));
    plinitpopup(v, flags, pop0, pop1, pop2);
    return v;
}
```

⟨*function* plinitpopup 60d⟩≡ (103c)

```
void plinitpopup(Panel *v, int flags, Panel *pop0, Panel *pop1, Panel *pop2){
    Popup *pp =v->data;

    v->flags=flags;
    v->state=UP;

    v->draw=pl_drawpopup;
    v->hit=pl_hitpopup;
    v->type=pl_typepopup;

    v->getsize=pl_getsizepopup;
    v->childspace=pl_childspacepopup;

    v->pri=pl_pripopup;

    pp->pop[0]=pop0;
    pp->pop[1]=pop1;
    pp->pop[2]=pop2;
    pp->save=nil;

    v->kind="popup";
}
```

11.2.2 Reacting

⟨Style *other cases* 61a⟩+≡ (13j) ◁46d

```
DOWN1,
DOWN2,
DOWN3,
```

⟨*function* pl_hitpopup 61b⟩≡ (103c)

```
bool pl_hitpopup(Panel *g, Mouse *m){
    Panel *p;
    Popup *pp = g->data;
    ⟨pl_hitpopup() other locals 62b⟩

    if(g->state==UP){
        switch(m->buttons&7){
        case CLICK_LEFT:   p=pp->pop[0]; g->state=DOWN1; break;
        case CLICK_MIDDLE: p=pp->pop[1]; g->state=DOWN2; break;
        case CLICK_RIGHT:  p=pp->pop[2]; g->state=DOWN3; break;
        ⟨pl_hitpopup() when UP, switch buttons other cases 61c⟩
        }
        ⟨pl_hitpopup() when UP, if no popup menu p 61e⟩
        else if(g->state!=UP){
            ⟨pl_hitpopup() when from UP to DOWNX and valid p 62c⟩
        }
    } else{ // a DOWNX state
        switch(g->state){
        case DOWN1: p=pp->pop[0]; break;
        case DOWN2: p=pp->pop[1]; break;
        case DOWN3: p=pp->pop[2]; break;
        case DOWN:  p=g->child;  break;
        default: SET(p); break;   /* can't happen! */
        }
        ⟨pl_hitpopup() when DOWNX state, if no buttons 62g⟩
    }
    // redispatch mouse event to appropriate menu or child
    plmouse(p, m);

    if((m->buttons&7)==0)
        g->state=UP;
    return (m->buttons&7)!=0;
}
```

⟨pl_hitpopup() *when* UP, *switch buttons other cases* 61c⟩≡ (61b) 61d▷

```
// just a mouse motion
case 0: p=g->child; break;
```

⟨pl_hitpopup() *when* UP, *switch buttons other cases* 61d⟩+≡ (61b) ◁61c

```
default: p=nil; break;
```

⟨pl_hitpopup() *when* UP, *if no popup menu* p 61e⟩≡ (61b)

```
if(p==nil){
    p=g->child;
    g->state=DOWN;
}
```

⟨*function* pl_typepopup 61f⟩≡ (103c)

```
void pl_typepopup(Panel *g, Rune c){
    USED(g, c);
}
```

11.2.3 Drawing

⟨*function* pl_drawpopup 62a⟩≡ (103c)

```
void pl_drawpopup(Panel *p){
    USED(p);
}
```

⟨pl_hitpopup() *other locals* 62b⟩≡ (61b)

```
Vector d; // delta
```

⟨pl_hitpopup() *when from* UP *to* DOWNX *and valid* p 62c⟩≡ (61b)

```
plpack(p, view->clipr);
⟨pl_hitpopup() when from UP to DOWNX, compute d 62e⟩
plmove(p, d);
pp->save=allocimage(display, p->r, g->b->chan, false, DNofill);
if(pp->save!=nil)
    draw(pp->save, p->r, g->b, nil, p->r.min);
pl_invis(p, false);
pldraw(p, g->b);
```

⟨*function* plmove 62d⟩≡ (103b)

```
/*
 * move an already-packed panel so that p->r=raddp(p->r, d)
 */
void plmove(Panel *p, Point d){
    ⟨plmove() if edit widget special case 62f⟩
    p->r=rectaddpt(p->r, d);
    for(p=p->child;p;p=p->next)
        plmove(p, d);
}
```

⟨pl_hitpopup() *when from* UP *to* DOWNX, *compute* d 62e⟩≡ (62c)

```
if(p->lastmouse)
    d=subpt(m->xy, divpt(addpt(p->lastmouse->r.min,
                  p->lastmouse->r.max), 2));
else
    d=subpt(m->xy, divpt(addpt(p->r.min, p->r.max), 2));
if(p->r.min.x+d.x<g->r.min.x) d.x=g->r.min.x-p->r.min.x;
if(p->r.max.x+d.x>g->r.max.x) d.x=g->r.max.x-p->r.max.x;
if(p->r.min.y+d.y<g->r.min.y) d.y=g->r.min.y-p->r.min.y;
if(p->r.max.y+d.y>g->r.max.y) d.y=g->r.max.y-p->r.max.y;
```

⟨plmove() *if edit widget special case* 62f⟩≡ (62d)

```
if(strcmp(p->kind, "edit") == 0) /* sorry */
    plemove(p, d);
```

Hiding

⟨pl_hitpopup() *when* DOWNX *state, if no buttons* 62g⟩≡ (61b)

```
if((m->buttons&7)==0){
    if(g->state!=DOWN){
        if(pp->save!=nil){
            // restore from saved image
            draw(g->b, p->r, pp->save, nil, p->r.min);
            flushimage(display, true);
            freeimage(pp->save);
            pp->save=nil;
        }
        pl_invis(p, true);
    }
    g->state=UP;
}
```

⟨*function* pl_invis 63a⟩≡ (98c)

```
void pl_invis(Panel *p, bool v){
    for(;p;p=p->next){
        if(v)
            p->flags|=INVIS;
        else
            p->flags&=~INVIS;
        pl_invis(p->child, v);
    }
}
```

⟨*constant* INVIS 63b⟩≡ (96d)

```
#define INVIS 0x20000  /* don't draw this */
```

⟨pl_drawall() *if invisible widget* 63c⟩≡ (17c)

```
if(p->flags&INVIS) return;
```

11.2.4 Packing methods

⟨*function* pl_getsizepopup 63d⟩≡ (103c)

```
Vector pl_getsizepopup(Panel *g, Vector children){
    USED(g);
    return children;
}
```

⟨*function* pl_childspacepopup 63e⟩≡ (103c)

```
void pl_childspacepopup(Panel *g, Point *ul, Vector *size){
    USED(g, ul, size);
}
```

11.3 Pull-down

⟨*struct* Pulldown 63f⟩≡ (104a)

```
struct Pulldown{
    Icon *icon;  /* button label */
    Panel *pull;  /* Panel to pull down */

    // enum<PackingDirection>
    int side;  /* which side of the button to put the panel on */
    //option<ref_own<Image>>
    Image *save;  /* where to save what we draw the panel on */
};
```

11.3.1 Initializing

⟨*function* plpulldown 63g⟩≡ (104a)

```
Panel *plpulldown(Panel *parent, int flags, Icon *icon, Panel *pullthis, int side){
    Panel *v;

    v=pl_newpanel(parent, sizeof(Pulldown));
    plinitpulldown(v, flags, icon, pullthis, side);
    return v;
}
```

⟨*function* plinitpulldown 64a⟩≡ (104a)

```
void plinitpulldown(Panel *v, int flags, Icon *icon, Panel *pullthis, int side){
    Pulldown *pp = v->data;

    v->flags=flags|LEAF;
    v->state=UP;

    v->draw=pl_drawpulldown;
    v->hit=pl_hitpulldown;
    v->type=pl_typepulldown;

    v->getsize=pl_getsizepulldown;
    v->childspace=pl_childspacepulldown;

    pp->icon=icon;
    pp->pull=pullthis;
    pp->side=side;
    pp->save=nil;

    v->kind="pulldown";
}
```

11.3.2 Drawing

⟨*function* pl_drawpulldown 64b⟩≡ (104a)

```
void pl_drawpulldown(Panel *p){
    Pulldown* pd = p->data;

    pl_drawicon(p->b, pl_box(p->b, p->r, p->state), PLACECEN,
        p->flags, pd->icon);
}
```

11.3.3 Reacting

⟨*function* pl_hitpulldown 64c⟩≡ (104a)

```
bool pl_hitpulldown(Panel *g, Mouse *m){
    Pulldown *pp = g->data;
    Panel *p = pp->pull;
    Panel *hitme = nil;
    int oldstate;
    Rectangle r;
    bool passon;

    oldstate=g->state;

    switch(g->state){
    case UP:
        ⟨pl_hitpulldown() when UP, if mouse outside widget 66a⟩
        else
            if(m->buttons&7){
                r=g->b->r;
                p->flags&=~PLACE;
                switch(pp->side){
                ⟨pl_hitpulldown() when UP and buttons, switch side cases 65⟩
                }
                plpack(p, r);
                pp->save=allocimage(display, p->r, g->b->chan, false, DNofill);
                if(pp->save!=nil)
```

```
                    draw(pp->save, p->r, g->b, nil, p->r.min);
                pl_invis(p, false);
                pldraw(p, g->b);
                g->state=DOWN;
            }
        break;
    case DOWN:
        if(!ptinrect(m->xy, g->r)){
            switch(pp->side){
            case PACKN: passon=m->xy.y<g->r.min.y; break;
            case PACKS: passon=m->xy.y>=g->r.max.y; break;
            case PACKE: passon=m->xy.x>=g->r.max.x; break;
            case PACKW: passon=m->xy.x<g->r.min.x; break;
            case PACKCEN: passon=true; break;
            default: SET(passon); break;  /* doesn't happen */
            }
            if(passon){
                hitme=p;
                if((m->buttons&7)==0)
                    g->state=UP;
            }
            else g->state=UP;
        }
        else if((m->buttons&7)==0) g->state=UP;
        else hitme=p;

        if(g->state!=DOWN && pp->save){
            draw(g->b, p->r, pp->save, nil, p->r.min);
            freeimage(pp->save);
            pp->save=nil;
            pl_invis(p, true);
            hitme=p;
        }
    }
    if(g->state!=oldstate)
        pldraw(g, g->b);
    if(hitme)
        plmouse(hitme, m);

    return g->state==DOWN;
}
```

⟨`pl_hitpulldown()` *when* `UP` *and buttons, switch side cases* 65⟩≡ (64c)

```
case PACKN:
    r.min.x=g->r.min.x;
    r.max.y=g->r.min.y;
    p->flags|=PLACESW;
    break;
case PACKS:
    r.min.x=g->r.min.x;
    r.min.y=g->r.max.y;
    p->flags|=PLACENW;
    break;
case PACKE:
    r.min.x=g->r.max.x;
    r.min.y=g->r.min.y;
    p->flags|=PLACENW;
    break;
case PACKW:
    r.max.x=g->r.min.x;
```

```
        r.min.y=g->r.min.y;
        p->flags|=PLACENE;
        break;
    case PACKCEN:
        r.min=g->r.min;
        p->flags|=PLACENW;
        break;
```

⟨pl_hitpulldown() *when* UP, *if mouse outside widget* 66a⟩≡ (64c)

```
    if(!ptinrect(m->xy, g->r))
        g->state=UP;
```

⟨*function* pl_typepulldown 66b⟩≡ (104a)

```
    void pl_typepulldown(Panel *p, Rune c){
        USED(p, c);
    }
```

11.3.4 Packing methods

⟨*function* pl_getsizepulldown 66c⟩≡ (104a)

```
    Vector pl_getsizepulldown(Panel *p, Vector children){
        USED(p, children);
        return pl_boxsize(pl_iconsize(p->flags, ((Pulldown *)p->data)->icon), p->state);
    }
```

⟨*function* pl_childspacepulldown 66d⟩≡ (104a)

```
    void pl_childspacepulldown(Panel *p, Point *ul, Vector *size){
        USED(p, ul, size);
    }
```

11.4 Menu bar

11.4.1 Initializing

⟨*function* plmenubar 66e⟩≡ (104a)

```
    Panel *plmenubar(Panel *parent, int flags, int cflags, Icon *l1, Panel *m1, Icon *l2, ...){
        Panel *v;
        va_list arg;
        Icon *s;
        int pulldir;

        ⟨plmenubar() set pulldir based on cflags 67⟩
        v=plgroup(parent, flags);

        va_start(arg, cflags);
        while((s=va_arg(arg, Icon *))!=nil)
            plpulldown(v, cflags, s, va_arg(arg, Panel *), pulldir);
        va_end(arg);

        USED(l1, m1, l2); // used for type checking at least the first arg
        v->kind="menubar";
        return v;
    }
```

⟨plmenubar() *set* pulldir *based on* cflags 67⟩≡ (66e)

```
switch(cflags&PACK){
case PACKE:
case PACKW:
    pulldir=PACKS;
    break;
case PACKN:
case PACKS:
    pulldir=PACKE;
    break;
default:
    SET(pulldir);
    break;
}
```

Chapter 12

Text Widgets

12.1 Message

⟨*struct* Message 68a⟩≡ (103a)

```
struct Message{
    // ref_own<string>
    char *text;

    Vector minsize;
};
```

12.1.1 Initializing

⟨*function* plmessage 68b⟩≡ (103a)

```
Panel *plmessage(Panel *parent, int flags, int wid, char *msg){
    Panel *v;

    v=pl_newpanel(parent, sizeof(Message));
    plinitmessage(v, flags, wid, msg);
    return v;
}
```

⟨*function* plinitmessage 68c⟩≡ (103a)

```
void plinitmessage(Panel *v, int flags, int wid, char *msg){
    Message *mp = v->data;

    v->flags=flags|LEAF;

    v->draw=pl_drawmessage;
    v->hit=pl_hitmessage;
    v->type=pl_typemessage;

    v->getsize=pl_getsizemessage;
    v->childspace=pl_childspacemessage;

    mp->text=msg;
    mp->minsize=Pt(wid, font->height);

    v->kind="message";
}
```

12.1.2 Drawing

⟨*function* pl_drawmessage 69a⟩≡ (103a)

```
void pl_drawmessage(Panel *p){
    pl_textmsg(p->b, pl_box(p->b, p->r, PASSIVE), font,
               ((Message *)p->data)->text);
}
```

⟨*function* pl_textmsg 69b⟩≡ (103a)

```
void pl_textmsg(Image *b, Rectangle r, Font *f, char *s){
    char *start, *end; /* of line */
    Point where;
    int lwid, c, wid;

    where=r.min;
    wid=r.max.x-r.min.x;
    do{
        start=s;
        lwid=0;
        end=s;
        do{
            for(;*s!=' ' && *s!='\0';s=pl_nextrune(s)) lwid+=pl_runewidth(f, s);
            if(lwid>wid) break;
            end=s;
            for(;*s==' ';s=pl_nextrune(s)) lwid+=pl_runewidth(f, s);
        }while(*s!='\0');
        if(end==start) /* can't even fit one word on line! */
            end=s;
        c=*end;
        *end='\0';
        string(b, where, display->black, ZP, f, start);
        *end=c;
        where.y+=font->height;
        s=end;
        while(*s==' ') s=pl_nextrune(s);
    }while(*s!='\0');
}
```

12.1.3 Reacting

⟨*function* pl_hitmessage 69c⟩≡ (103a)

```
bool pl_hitmessage(Panel *g, Mouse *m){
    USED(g, m);
    return false;
}
```

⟨*function* pl_typemessage 69d⟩≡ (103a)

```
void pl_typemessage(Panel *g, Rune c){
    USED(g, c);
}
```

12.1.4 Packing methods

⟨*function* pl_getsizemessage 69e⟩≡ (103a)

```
Vector pl_getsizemessage(Panel *p, Vector children){
    Message *mp = p->data;

    USED(children);
    return pl_boxsize(pl_foldsize(font, mp->text, mp->minsize.x), PASSIVE);
}
```

⟨*function* pl_foldsize 70a⟩≡ (103a)

```
Point pl_foldsize(Font *f, char *s, int wid){
    char *start, *end; /* of line */
    Point size;
    int lwid, ewid;
    size=Pt(0,0);
    do{
        start=s;
        lwid=0;
        end=s;
        ewid=lwid;
        do{
            for(;*s!=' ' && *s!='\0';s=pl_nextrune(s)) lwid+=pl_runewidth(f, s);
            if(lwid>wid) break;
            end=s;
            ewid=lwid;
            for(;*s==' ';s=pl_nextrune(s)) lwid+=pl_runewidth(f, s);
        }while(*s!='\0');
        if(end==start){ /* can't even fit one word on line! */
            ewid=lwid;
            end=s;
        }
        if(ewid>size.x) size.x=ewid;
        size.y+=font->height;
        s=end;
        while(*s==' ') s=pl_nextrune(s);
    }while(*s!='\0');
    return size;
}
```

⟨*function* pl_childspacemessage 70b⟩≡ (103a)

```
void pl_childspacemessage(Panel *p, Point *ul, Vector *size){
    USED(p, ul, size);
}
```

12.2 Edit

⟨*struct* Edit 70c⟩≡ (101a)

```
struct Edit{
    Point minsize;
    void (*hit)(Panel *);
    int sel0, sel1;
    Textwin *t;
    Rune *text;
    int ntext;
};
```

12.2.1 Initializing

⟨*function* pledit 70d⟩≡ (101a)

```
Panel *pledit(Panel *parent, int flags, Point minsize, Rune *text, int ntext, void (*hit)(Panel *)){
    Panel *v;

    v=pl_newpanel(parent, sizeof(Edit));
    ((Edit *)v->data)->t=0;
    plinitedit(v, flags, minsize, text, ntext, hit);
    return v;
}
```

⟨*function* plinitedit 71a⟩≡ (101a)

```
void plinitedit(Panel *v, int flags, Point minsize, Rune *text, int ntext, void (*hit)(Panel *)){
    Edit *ep = v->data;

    v->flags=flags|LEAF;
    v->state=UP;
    v->draw=pl_drawedit;
    v->hit=pl_hitedit;
    v->type=pl_typeedit;
    v->getsize=pl_getsizeedit;
    v->childspace=pl_childspaceedit;
    v->free=pl_freeedit;

    v->snarf=pl_snarfedit;
    v->paste=pl_pasteedit;

    v->kind="edit";
    ep->hit=hit;
    ep->minsize=minsize;
    ep->text=text;
    ep->ntext=ntext;
    if(ep->t!=0) twfree(ep->t);
    ep->t=0;
    ep->sel0=-1;
    ep->sel1=-1;
    v->scroll=pl_scrolledit;
    v->scr.pos=Pt(0,0);
    v->scr.size=Pt(ntext,0);
}
```

⟨*function* pl_freeedit 71b⟩≡ (101a)

```
void pl_freeedit(Panel *p){
    Edit *ep;
    ep=p->data;
    if(ep->t) twfree(ep->t);
    ep->t=0;
}
```

12.2.2 Drawing

⟨*function* pl_drawedit 71c⟩≡ (101a)

```
void pl_drawedit(Panel *p){
    Edit *ep;
    Panel *sb;
    ep=p->data;
    if(ep->t==0){
        ep->t=twnew(p->b, font, ep->text, ep->ntext);
        if(ep->t==0){
            fprint(2, "pl_drawedit: can't allocate\n");
            exits("no mem");
        }
    }
    ep->t->b=p->b;
    twreshape(ep->t, p->r);
    twhilite(ep->t, ep->sel0, ep->sel1, 1);
    sb=p->yscroller;
    if(sb && sb->setscrollbar)
        sb->setscrollbar(sb, ep->t->top, ep->t->bot, ep->t->etext-ep->t->text);
}
```

12.2.3 Reacting

⟨*function* pl_hitedit 72a⟩≡ (101a)

```
/*
 * Should do double-clicks:
 * If ep->sel0==ep->sel1 on entry and the
 * call to twselect returns the same selection, then
 * expand selections (| marks possible selection points, ... is expanded selection)
 * <|...|>   <> must nest
 * (|...|)   () must nest
 * [|...|]   [] must nest
 * {|...|}   {} must nest
 * '|...|'   no ' in ...
 * "|...|"   no " in ...
 * \n|...|\n  either newline may be the corresponding end of text
 *    include the trailing newline in the selection
 * ...|I...  I and ... are characters satisfying pl_idchar(I)
 * ...I|
 */
int pl_hitedit(Panel *p, Mouse *m){
    Edit *ep;
    ep=p->data;
    if(ep->t && m->buttons&1){
        plgrabkb(p);
        ep->t->b=p->b;
        twhilite(ep->t, ep->sel0, ep->sel1, 0);
        twselect(ep->t, m);
        ep->sel0=ep->t->sel0;
        ep->sel1=ep->t->sel1;
        if((m->buttons&7)==3){
            plsnarf(p);
            plepaste(p, 0, 0); /* cut */
        }
        else if((m->buttons&7)==5)
            plpaste(p);
        else if(ep->hit)
            (*ep->hit)(p);
    }
    return 0;
}
```

⟨*function* pl_typeedit 72b⟩≡ (101a)

```
void pl_typeedit(Panel *p, Rune c){
    Edit *ep;
    Textwin *t;
    int bot, scrolled;
    Panel *sb;
    ep=p->data;
    t=ep->t;
    if(t==0) return;
    t->b=p->b;
    twhilite(t, ep->sel0, ep->sel1, 0);
    switch(c){
    case Kesc:
        plsnarf(p);
        plepaste(p, 0, 0); /* cut */
        break;
    case Kdel: /* clear */
        ep->sel0=0;
        ep->sel1=plelen(p);
```

```
        plepaste(p, 0, 0); /* cut */
        break;
    case Kbs: /* ^H: erase character */
        if(ep->sel0!=0) --ep->sel0;
        twreplace(t, ep->sel0, ep->sel1, 0, 0);
        break;
// case Knack: /* ^U: erase line */
//  while(ep->sel0!=0 && t->text[ep->sel0-1]!='\n') --ep->sel0;
//  twreplace(t, ep->sel0, ep->sel1, 0, 0);
//  break;
// case Ketb: /* ^W: erase word */
//  while(ep->sel0!=0 && !pl_idchar(t->text[ep->sel0-1])) --ep->sel0;
//  while(ep->sel0!=0 && pl_idchar(t->text[ep->sel0-1])) --ep->sel0;
//  twreplace(t, ep->sel0, ep->sel1, 0, 0);
//  break;
    default:
        if((c & 0xFF00) == KF || (c & 0xFF00) == Spec)
            break;
        twreplace(t, ep->sel0, ep->sel1, &c, 1);
        ++ep->sel0;
        break;
    }
    ep->sel1=ep->sel0;
    /*
     * Scroll up until ep->sel0 is above t->bot.
     */
    scrolled=0;
    do{
        bot=t->bot;
        if(ep->sel0<=bot) break;
        twscroll(t, twpt2rune(t, Pt(t->r.min.x, t->r.min.y+font->height)));
        scrolled++;
    }while(bot!=t->bot);
    if(scrolled){
        sb=p->yscroller;
        if(sb && sb->setscrollbar)
            sb->setscrollbar(sb, t->top, t->bot, t->etext-t->text);
    }
    twhilite(t, ep->sel0, ep->sel1, 1);
}
```

12.2.4 Packing methods

⟨*function* pl_getsizeedit 73a⟩≡ (101a)

```
Point pl_getsizeedit(Panel *p, Point children){
    USED(children);
    return pl_boxsize(((Edit *)p->data)->minsize, p->state);
}
```

⟨*function* pl_childspaceedit 73b⟩≡ (101a)

```
void pl_childspaceedit(Panel *g, Point *ul, Vector *size){
    USED(g, ul, size);
}
```

12.2.5 Other methods

⟨*function* pl_scrolledit 73c⟩≡ (101a)

```
void pl_scrolledit(Panel *p, int dir, buttons buttons, int num, int den){
```

```
    Edit *ep;
    Textwin *t;
    Panel *sb;
    int index, nline;
    if(dir!=VERT) return;
    ep=p->data;
    t=ep->t;
    if(t==0) return;
    t->b=p->b;
    switch(buttons){
    default:
        return;
    case 1:  /* top line moves to mouse position */
        nline=(t->r.max.y-t->r.min.y)/t->hgt*num/den;
        index=t->top;
        while(index!=0 && nline!=0)
            if(t->text[--index]=='\n') --nline;
        break;
    case 2:  /* absolute */
        index=(t->etext-t->text)*num/den;
        break;
    case 4:  /* mouse points at new top line */
        index=twpt2rune(t,
            Pt(t->r.min.x, t->r.min.y+(t->r.max.y-t->r.min.y)*num/den));
        break;
    }
    while(index!=0 && t->text[index-1]!='\n') --index;
    if(index!=t->top){
        twhilite(ep->t, ep->sel0, ep->sel1, 0);
        twscroll(t, index);
        p->scr.pos.y=t->top;
        twhilite(ep->t, ep->sel0, ep->sel1, 1);
        sb=p->yscroller;
        if(sb && sb->setscrollbar)
            sb->setscrollbar(sb, t->top, t->bot, t->etext-t->text);
    }
}
```

⟨*function* plescroll 74a⟩≡ (101a)

```
void plescroll(Panel *p, int top){
    Textwin *t;
    t=((Edit*)p->data)->t;
    if(t) twscroll(t, top);
}
```

⟨*function* plegetsel 74b⟩≡ (101a)

```
void plegetsel(Panel *p, int *sel0, int *sel1){
    Edit *ep;
    ep=p->data;
    *sel0=ep->sel0;
    *sel1=ep->sel1;
}
```

⟨*function* plelen 74c⟩≡ (101a)

```
int plelen(Panel *p){
    Textwin *t;
    t=((Edit*)p->data)->t;
    if(t==0) return 0;
    return t->etext-t->text;
}
```

⟨*function* pleget 75a⟩≡ (101a)

```
Rune *pleget(Panel *p){
    Textwin *t;
    t=((Edit*)p->data)->t;
    if(t==0) return 0;
    return t->text;
}
```

⟨*function* plesel 75b⟩≡ (101a)

```
void plesel(Panel *p, int sel0, int sel1){
    Edit *ep;
    ep=p->data;
    if(ep->t==0) return;
    ep->t->b=p->b;
    twhilite(ep->t, ep->sel0, ep->sel1, 0);
    ep->sel0=sel0;
    ep->sel1=sel1;
    twhilite(ep->t, ep->sel0, ep->sel1, 1);
}
```

⟨*function* plepaste 75c⟩≡ (101a)

```
void plepaste(Panel *p, Rune *text, int ntext){
    Edit *ep;
    ep=p->data;
    if(ep->t==0) return;
    ep->t->b=p->b;
    twhilite(ep->t, ep->sel0, ep->sel1, 0);
    twreplace(ep->t, ep->sel0, ep->sel1, text, ntext);
    ep->sel1=ep->sel0+ntext;
    twhilite(ep->t, ep->sel0, ep->sel1, 1);
    p->scr.size.y=ep->t->etext-ep->t->text;
    p->scr.pos.y=ep->t->top;
}
```

⟨*function* plemove 75d⟩≡ (101a)

```
void plemove(Panel *p, Point d){
    Edit *ep;
    ep=p->data;
    if(ep->t && !eqpt(d, Pt(0,0))) twmove(ep->t, d);
}
```

12.3 Text view

⟨*struct* Textview 75e⟩≡ (110d)

```
struct Textview{
    void (*hit)(Panel *, int, Rtext *); /* call back to user on hit */
    Rtext *text;   /* text */
    int yoffs;   /* offset of top of screen */
    Rtext *hitword;   /* text to hilite */
    Rtext *hitfirst;  /* first word in range select */
    int twid;   /* text width */
    int thgt;   /* text height */
    Point minsize;   /* smallest acceptible window size */
    buttons buttons;
};
```

12.3.1 Initializing

⟨*function* pltextview 76a⟩≡ (110d)

```
Panel *pltextview(Panel *parent, int flags, Point minsize, Rtext *t, void (*hit)(Panel *, int, Rtext *)){
    Panel *v;
    v=pl_newpanel(parent, sizeof(Textview));
    plinittextview(v, flags, minsize, t, hit);
    return v;
}
```

⟨*function* plinittextview 76b⟩≡ (110d)

```
void plinittextview(Panel *v, int flags, Point minsize, Rtext *t, void (*hit)(Panel *, int, Rtext *)){
    Textview *tp = v->data;

    v->flags=flags|LEAF;
    v->state=UP;
    v->draw=pl_drawtextview;
    v->hit=pl_hittextview;
    v->type=pl_typetextview;
    v->getsize=pl_getsizetextview;
    v->childspace=pl_childspacetextview;
    v->kind="textview";
    v->pri=pl_pritextview;
    tp->hit=hit;
    tp->minsize=minsize;
    tp->text=t;
    tp->yoffs=0;
    tp->hitfirst=0;
    tp->hitword=0;
    v->scroll=pl_scrolltextview;
    v->snarf=pl_snarftextview;
    tp->twid=-1;
    v->scr.pos=Pt(0,0);
    v->scr.size=Pt(0,1);
}
```

12.3.2 Drawing

⟨*function* pl_drawtextview 76c⟩≡ (110d)

```
void pl_drawtextview(Panel *p){
    int twid;
    Rectangle r;
    Textview *tp;
    tp=p->data;
    r=pl_outline(p->b, p->r, UP);
    twid=r.max.x-r.min.x;
    if(twid!=tp->twid){
        tp->twid=twid;
        tp->thgt=pl_rtfmt(tp->text, tp->twid);
        p->scr.size.y=tp->thgt;
    }
    p->scr.pos.y=tp->yoffs;
    pl_rtdraw(p->b, r, tp->text, tp->yoffs);
    pl_setscrpos(p, tp, r);
}
```

12.3.3 Reacting

⟨*function* pl_hittextview 76d⟩≡ (110d)

```
int pl_hittextview(Panel *p, Mouse *m){
    Rtext *oldhitword, *oldhitfirst;
    int hitme, oldstate;
    Point ul, size;
    Textview *tp;

    tp=p->data;

    hitme=0;
    oldstate=p->state;
    oldhitword=tp->hitword;
    oldhitfirst=tp->hitfirst;
    if(oldhitword==oldhitfirst)
        pl_passon(oldhitword, m);
    if(m->buttons&OUT)
        p->state=UP;
    else if(m->buttons&7){
        p->state=DOWN;
        tp->buttons=m->buttons;
        if(oldhitword==0 || oldhitword->p==0 || (oldhitword->p->flags&REMOUSE)==0){
            ul=p->r.min;
            size=subpt(p->r.max, p->r.min);
            pl_interior(p->state, &ul, &size);
            tp->hitword=pl_rthit(tp->text, tp->yoffs, m->xy, ul);
            if(tp->hitword==0)
                if(oldhitword!=0 && oldstate==DOWN)
                    tp->hitword=oldhitword;
                else
                    tp->hitfirst=0;
            if(tp->hitword!=0 && oldstate!=DOWN)
                tp->hitfirst=tp->hitword;
        }
    }
    else{
        if(p->state==DOWN) hitme=1;
        p->state=UP;
    }
    if(tp->hitfirst!=oldhitfirst || tp->hitword!=oldhitword){
        plrtseltext(tp->text, tp->hitword, tp->hitfirst);
        pl_drawtextview(p);
        if(tp->hitword==tp->hitfirst)
            pl_passon(tp->hitword, m);
    }
    if(hitme && tp->hit && tp->hitword!=0 && tp->hitword==tp->hitfirst){
        plrtseltext(tp->text, 0, 0);
        pl_drawtextview(p);
        tp->hit(p, tp->buttons, tp->hitword);
        tp->hitword=0;
        tp->hitfirst=0;
    }
    return 0;
}
```

⟨*function* pl_typetextview 77⟩≡ (110d)

```
void pl_typetextview(Panel *g, Rune c){
    USED(g, c);
}
```

12.3.4 Packing methods

⟨*function* pl_getsizetextview 78a⟩≡ (110d)

```
Point pl_getsizetextview(Panel *p, Point children){
    USED(children);
    return pl_boxsize(((Textview *)p->data)->minsize, p->state);
}
```

⟨*function* pl_childspacetextview 78b⟩≡ (110d)

```
void pl_childspacetextview(Panel *g, Point *ul, Vector *size){
    USED(g, ul, size);
}
```

12.4 Completion

12.5 Rich text

⟨*struct* Rtext 78c⟩≡ (92)

```
struct Rtext{
    int flags;  /* responds to hits? text selection? */
    void *user;  /* user data */
    int space;  /* how much space before, if no break */
    int indent;  /* how much space before, after a break */
    Image *b;  /* what to display, if nonzero */
    Panel *p;  /* what to display, if nonzero and b==0 */
    Font *font;  /* font in which to draw text */
    char *text;  /* what to display, if b==0 and p==0 */
    Rtext *next;  /* next piece */
    /* private below */
    Rtext *nextline; /* links line to line */
    Rtext *last;  /* last, for append */
    Rectangle r;  /* where to draw, if origin were Pt(0,0) */
    int topy;  /* y coord of top of line */
    int wid;  /* not including space */
};
```

12.5.1 Initializing

⟨*function* pl_rtnew 78d⟩≡ (107)

```
Rtext *pl_rtnew(Rtext **t, int space, int indent, Image *b, Panel *p, Font *f, char *s, int flags, void *user){
    Rtext *new;
    new=pl_emalloc(sizeof(Rtext));
    new->flags=flags;
    new->user=user;
    new->space=space;
    new->indent=indent;
    new->b=b;
    new->p=p;
    new->font=f;
    new->text=s;
    new->next=0;
    new->nextline=0;
    new->r=Rect(0,0,0,0);
    if(*t)
        (*t)->last->next=new;
    else
        *t=new;
```

```
    (*t)->last=new;
    return new;
}
```

⟨*function* plrtpanel 79a⟩≡ (107)

```
Rtext *plrtpanel(Rtext **t, int space, int indent, Panel *p, void *user){
    return pl_rtnew(t, space, indent, 0, p, 0, 0, 1, user);
}
```

⟨*function* plrtstr 79b⟩≡ (107)

```
Rtext *plrtstr(Rtext **t, int space, int indent, Font *f, char *s, int flags, void *user){
    return pl_rtnew(t, space, indent, 0, 0, f, s, flags, user);
}
```

⟨*function* plrtbitmap 79c⟩≡ (107)

```
Rtext *plrtbitmap(Rtext **t, int space, int indent, Image *b, int flags, void *user){
    return pl_rtnew(t, space, indent, b, 0, 0, 0, flags, user);
}
```

⟨*function* plrtfree 79d⟩≡ (107)

```
void plrtfree(Rtext *t){
    Rtext *next;
    while(t){
        next=t->next;
        free(t);
        t=next;
    }
}
```

12.5.2 Drawing

⟨*constant* PL_HOT 79e⟩≡ (92)

```
#define PL_HOT  1
```

⟨*constant* PL_SEL 79f⟩≡ (92)

```
#define PL_SEL  2
```

⟨*function* pl_rtdraw 79g⟩≡ (107)

```
void pl_rtdraw(Image *b, Rectangle r, Rtext *t, int yoffs){
    static Image *backup;
    Point offs, lp;
    Rectangle dr;
    Image *bb;

    bb = b;
    if(backup==0 || backup->chan!=b->chan || rectinrect(r, backup->r)==0){
        freeimage(backup);
        backup=allocimage(display, bb->r, bb->chan, 0, DNofill);
    }
    if(backup)
        b=backup;
    pl_clr(b, r);
    lp=ZP;
    offs=subpt(r.min, Pt(0, yoffs));
    for(;t;t=t->next) if(!eqrect(t->r, Rect(0,0,0,0))){
        dr=rectaddpt(t->r, offs);
        if(dr.max.y>r.min.y
        && dr.min.y<r.max.y){
            if(t->b){
```

```
                draw(b, insetrect(dr, BORD), t->b, 0, t->b->r.min);
                if(t->flags&PL_HOT) border(b, dr, 1, display->black, ZP);
                if(t->flags&PL_SEL)
                    pl_highlight(b, dr);
            }
            else if(t->p){
                plmove(t->p, subpt(dr.min, t->p->r.min));
                pldraw(t->p, b);
                if(b!=bb)
                    pl_stuffbitmap(t->p, bb);
            }
            else{
                string(b, dr.min, display->black, ZP, t->font, t->text);
                if(t->flags&PL_SEL)
                    pl_highlight(b, dr);
                if(t->flags&PL_HOT){
                    if(lp.y+1 != dr.max.y)
                        lp = Pt(dr.min.x, dr.max.y-1);
                    line(b, lp, Pt(dr.max.x, dr.max.y-1),
                        Endsquare, Endsquare, 0,
                        display->black, ZP);
                    lp = Pt(dr.max.x, dr.max.y-1);
                    continue;
                }
            }
            lp=ZP;
        }
    }
    if(b!=bb)
        draw(bb, r, b, 0, r.min);
}
```

⟨*function* pl_clr 80a⟩≡ (98c)

```
void pl_clr(Image *b, Rectangle r){
    draw(b, r, display->white, 0, ZP);
}
```

⟨*function* pl_rtredraw 80b⟩≡ (107)

```
/*
 * Rectangle r of Image b contains an image of Rtext t, offset by oldoffs.
 * Redraw the text to have offset yoffs.
 */
void pl_rtredraw(Image *b, Rectangle r, Rtext *t, int yoffs, int oldoffs){
    int dy, size;
    dy=oldoffs-yoffs;
    size=r.max.y-r.min.y;
    if(dy>=size || -dy>=size)
        pl_rtdraw(b, r, t, yoffs);
    else if(dy<0){
        pl_reposition(t, b, r.min,
            Rect(r.min.x, r.min.y-dy, r.max.x, r.max.y));
        pl_rtdraw(b, Rect(r.min.x, r.max.y+dy, r.max.x, r.max.y),
            t, yoffs+size+dy);
    }
    else if(dy>0){
        pl_reposition(t, b, Pt(r.min.x, r.min.y+dy),
            Rect(r.min.x, r.min.y, r.max.x, r.max.y-dy));
        pl_rtdraw(b, Rect(r.min.x, r.min.y, r.max.x, r.min.y+dy), t, yoffs);
    }
}
```

12.5.3 Reacting

⟨*function* pl_rthit 81a⟩≡ (107)

```
Rtext *pl_rthit(Rtext *t, int yoffs, Point p, Point ul){
    Rectangle r;
    Point lp;
    if(t==0) return 0;
    p.x-=ul.x;
    p.y+=yoffs-ul.y;
    while(t->nextline && t->nextline->topy<=p.y) t=t->nextline;
    lp=ZP;
    for(;t!=0;t=t->next){
        if(t->topy>p.y) return 0;
        r = t->r;
        if((t->flags&PL_HOT) != 0 && t->b == nil && t->p == nil){
            if(lp.y == r.max.y && lp.x < r.min.x)
                r.min.x=lp.x;
            lp=r.max;
        } else
            lp=ZP;
        if(ptinrect(p, r)) return t;
    }
    return 0;
}
```

12.5.4 Packing methods

12.5.5 Other methods

⟨*function* plrtseltext 81b⟩≡ (107)

```
void plrtseltext(Rtext *t, Rtext *s, Rtext *e){
    while(t){
        t->flags &= ~PL_SEL;
        t = t->next;
    }
    if(s==0 || e==0)
        return;
    for(t=s; t!=0 && t!=e; t=t->next)
        ;
    if(t==e){
        for(t=s; t!=e; t=t->next)
            t->flags |= PL_SEL;
    }else{
        for(t=e; t!=s; t=t->next)
            t->flags |= PL_SEL;
    }
    t->flags |= PL_SEL;
}
```

Chapter 13

Advanced Topics TODO

13.1 copy/paste

⟨Panel *other methods* 82a⟩+≡ (11a) ◁49g

```
char* (*snarf)(Panel *);   /* snarf text from panel */
void (*paste)(Panel *, char *);   /* paste text into panel */
```

⟨pl_newpanel() *set default methods* 82b⟩+≡ (11b) ◁49h

```
v->snarf=nil;
v->paste=nil;
```

⟨*function* plputsnarf 82c⟩≡ (108e)

```
void plputsnarf(char *s){
    int fd;

    if(s==0 || *s=='\0')
        return;
    if((fd=open("/dev/snarf", OWRITE|OTRUNC))>=0){
        write(fd, s, strlen(s));
        close(fd);
    }
}
```

⟨*function* plgetsnarf 82d⟩≡ (108e)

```
char *plgetsnarf(void){
    int fd, n, r;
    char *s;

    if((fd=open("/dev/snarf", OREAD))<0)
        return nil;
    n=0;
    s=nil;
    for(;;){
        s=pl_erealloc(s, n+1024);
        if((r = read(fd, s+n, 1024)) <= 0)
            break;
        n += r;
    }
    close(fd);
    if(n <= 0){
        free(s);
        return nil;
    }
    s[n] = '\0';
    return s;
}
```

⟨*function* plsnarf 83a⟩≡ (108e)

```
void plsnarf(Panel *p){
    char *s;

    if(p==0 || p->snarf==0)
        return;
    s=p->snarf(p);
    plputsnarf(s);
    free(s);
}
```

⟨*function* plpaste 83b⟩≡ (108e)

```
void plpaste(Panel *p){
    char *s;

    if(p==0 || p->paste==0)
        return;
    if(s=plgetsnarf()){
        p->paste(p, s);
        free(s);
    }
}
```

13.1.1 Widgets hooks

⟨plinitentry() *set snarf methods* 83c⟩≡ (30d)

```
v->snarf=pl_snarfentry;
v->paste=pl_pasteentry;
```

⟨*function* pl_snarfentry 83d⟩≡ (100b)

```
char *pl_snarfentry(Panel *p){
    Entry *ep;
    int n;

    if(p->flags&USERFL) /* no snarfing from password entry */
        return nil;
    ep=p->data;
    n=ep->entp-ep->entry;
    if(n<=0) return nil;
    return smprint("%.*s", n, ep->entry);
}
```

⟨*function* pl_pasteentry 83e⟩≡ (100b)

```
void pl_pasteentry(Panel *p, char *s){
    Entry *ep;
    char *e;
    int n, m;

    ep=p->data;
    n=ep->entp-ep->entry;
    m=strlen(s);
    e=pl_erealloc(ep->entry,n+m+SLACK);
    ep->entry=e;
    e+=n;
    strncpy(e, s, m);
    e+=m;
    *e='\0';
    ep->entp=ep->eent=e;
    pldraw(p, p->b);
}
```

⟨*constant* SLACK 84a⟩≡ (100b)

```
#define SLACK 7 /* enough for one extra rune and < and a nul */
```

⟨pl_hitentry() *handle copy/paste when middle or right click* 84b⟩≡ (32d)

```
if((old&7)==CLICK_LEFT){
    if((m->buttons&7)==CLICK_LEFT|CLICK_MIDDLE){
        Entry *ep;

        plsnarf(p);

        /* cut */
        ep=p->data;
        ep->entp=ep->entry;
        *ep->entp='\0';
        pldraw(p, p->b);
    }
    if((m->buttons&7)==CLICK_LEFT|CLICK_RIGHT)
        plpaste(p);
}
```

⟨*function* pl_snarfedit 84c⟩≡ (101a)

```
char *pl_snarfedit(Panel *p){
    int s0, s1;
    Rune *t;
    t=pleget(p);
    plegetsel(p, &s0, &s1);
    if(t==0 || s0>=s1)
        return nil;
    return smprint("%.*S", s1-s0, t+s0);
}
```

⟨*function* pl_pasteedit 84d⟩≡ (101a)

```
void pl_pasteedit(Panel *p, char *s){
    Rune *t;
    if(t=runesmprint("%s", s)){
        plepaste(p, t, runestrlen(t));
        free(t);
    }
}
```

13.2 Rich text

13.3 Hooks

⟨Panel *user-specific fields* 84e⟩≡ (11a)

```
int user;     /* available for user */
void *userp;     /* available for user */
```

13.4 Advanced layout

13.4.1 FIXEDX, FIXEDY

⟨*constant* FIXEDX 84f⟩≡ (92)

```
#define FIXEDX 0x0400
```

⟨*constant* FIXEDY 85a⟩≡ (92)

```
#define FIXEDY 0x0800
```

⟨Panel *fixed size field* 85b⟩≡ (11a)

```
Point fixedsize;    /* size of Panel, if FIXED */
```

⟨pl_sizereq() *if* FIXEDX *or* FIXEDY 85c⟩≡ (22f)

```
if(p->flags&FIXEDX) p->sizereq.x=p->fixedsize.x;
if(p->flags&FIXEDY) p->sizereq.y=p->fixedsize.y;
```

⟨*constant* FIXED 85d⟩≡ (92)

```
#define FIXED 0x0c00  /* don't pass children's size requests through to parent */
```

13.4.2 MAXX, MAXY

⟨*constant* MAXX 85e⟩≡ (92)

```
#define MAXX 0x1000  /* make x size as big as biggest sibling's */
```

⟨*constant* MAXY 85f⟩≡ (92)

```
#define MAXY 0x2000  /* make y size as big as biggest sibling's */
```

⟨pl_sizeref() *other locals* 85g⟩≡ (22f)

```
Vector maxsize = Pt(0,0);
```

⟨pl_sizeref() *when looping over children, adjust* maxsize 85h⟩≡ (22f)

```
if(cp->sizereq.x>maxsize.x) maxsize.x=cp->sizereq.x;
if(cp->sizereq.y>maxsize.y) maxsize.y=cp->sizereq.y;
```

⟨pl_sizereq() *if* MAXX *or* MAXY 85i⟩≡ (22f)

```
for(cp=p->child;cp;cp=cp->next){
    if(cp->flags&MAXX) cp->sizereq.x=maxsize.x;
    if(cp->flags&MAXY) cp->sizereq.y=maxsize.y;
}
```

Chapter 14

Advanced Widgets TODO

Chapter 15

Conclusion

Appendix A

Debugging

⟨Panel *debugging fields* 88a⟩≡ (11a)

```
char *kind;     /* what kind of panel? */
```

⟨*function* pl_print 88b⟩≡ (99b)

```
void pl_print(Panel *p){
    pl_ipprint(p, 0);
}
```

⟨*function* pl_iprint 88c⟩≡ (99b)

```
void pl_iprint(int indent, char *fmt, ...){
    char buf[8192];
    va_list arg;

    memset(buf, '\t', indent);
    va_start(arg, fmt);
    write(1, buf, vsnprint(buf+indent, sizeof(buf)-indent, fmt, arg));
    va_end(arg);
}
```

⟨*function* pl_ipprint 88d⟩≡ (99b)

```
void pl_ipprint(Panel *p, int n){
    Panel *c;
    char *place, *stick;

    pl_iprint(n, "%s (0x%.8x)\n", p->kind, p);
    pl_iprint(n, "  r=(%d %d, %d %d)\n",
        p->r.min.x, p->r.min.y, p->r.max.x, p->r.max.y);
    switch(p->flags&PACK){
    default: SET(place); break;
    case PACKN: place="n"; break;
    case PACKE: place="e"; break;
    case PACKS: place="s"; break;
    case PACKW: place="w"; break;
    }
    switch(p->flags&PLACE){
    default: SET(stick); break;
    case PLACECEN: stick=""; break;
    case PLACES: stick=" stick s"; break;
    case PLACEE: stick=" stick e"; break;
    case PLACEW: stick=" stick w"; break;
    case PLACEN: stick=" stick n"; break;
    case PLACENE: stick=" stick ne"; break;
    case PLACENW: stick=" stick nw"; break;
    case PLACESE: stick=" stick se"; break;
    case PLACESW: stick=" stick sw"; break;
```

```
    }
    pl_iprint(n, "  place %s%s%s%s%s%s\n",
        place,
        p->flags&FILLX?" fill x":"",
        p->flags&FILLY?" fill y":"",
        stick,
        p->flags&EXPAND?" expand":"",
        p->flags&FIXED?" fixed":"");
    if(!eqpt(p->pad, Pt(0, 0))) pl_iprint(n, "  pad=%d,%d)\n", p->pad.x, p->pad.y);
    if(!eqpt(p->ipad, Pt(0, 0))) pl_iprint(n, "  ipad=%d,%d)\n", p->ipad.x, p->ipad.y);
    pl_iprint(n, "  size=(%d,%d), sizereq=(%d,%d)\n",
        p->size.x, p->size.y, p->sizereq.x, p->sizereq.y);
    for(c=p->child;c;c=c->next)
        pl_ipprint(c, n+1);
}
```

Appendix B

Error Management

⟨*function* pl_unexpected 90a⟩≡ (98b)

```
void pl_unexpected(Panel *g, char *rou){
    fprint(STDERR, "%s called unexpectedly (%s %lux)\n",
           rou, g->kind, (ulong)g);
    abort();
}
```

⟨*function* pl_drawerror 90b⟩≡ (98b)

```
void pl_drawerror(Panel *g){
    pl_unexpected(g, "draw");
}
```

⟨*function* pl_hiterror 90c⟩≡ (98b)

```
int pl_hiterror(Panel *g, Mouse *m){
    USED(m);
    pl_unexpected(g, "hit");
    return 0;
}
```

⟨*function* pl_typeerror 90d⟩≡ (98b)

```
void pl_typeerror(Panel *g, Rune c){
    USED(c);
    pl_unexpected(g, "type");
}
```

⟨*function* pl_getsizeerror 90e⟩≡ (98b)

```
Point pl_getsizeerror(Panel *g, Point childsize){
    pl_unexpected(g, "getsize");
    return childsize;
}
```

⟨*function* pl_childspaceerror 90f⟩≡ (98b)

```
void pl_childspaceerror(Panel *g, Point *ul, Vector *size){
    USED(ul, size);
    pl_unexpected(g, "childspace");
}
```

⟨*function* pl_scrollerror 90g⟩≡ (98b)

```
void pl_scrollerror(Panel *g, int dir, buttons button, int num, int den){
    USED(dir, button, num, den);
    pl_unexpected(g, "scroll");
}
```

⟨*function* pl_setscrollbarerror 90h⟩≡ (98b)

```
void pl_setscrollbarerror(Panel *g, int top, int bot, int den){
    USED(top, bot, den);
    pl_unexpected(g, "setscrollbar");
}
```

Appendix C

Utilities

C.1 Memory management

⟨*function* pl_emalloc 91a⟩≡ (98b)

```
void *pl_emalloc(int n){
    void *v;

    v=mallocz(n, 1);
    if(v==nil){
        fprint(STDERR, "Can't malloc!\n");
        exits("no mem");
    }
    setmalloctag(v, getcallerpc(&n));
    return v;
}
```

⟨*function* pl_erealloc 91b⟩≡ (98b)

```
void *pl_erealloc(void *v, int n)
{
    v=realloc(v, n);
    if(v==nil){
        fprint(STDERR, "Can't realloc!\n");
        exits("no mem");
    }
    setrealloctag(v, getcallerpc(&v));
    return v;
}
```

Appendix D

Extra Code

D.1 include/gui/

D.1.1 include/gui/panel.h

⟨include/gui/panel.h 92⟩≡

```
#pragma src "/sys/src/libpanel"
#pragma lib "libpanel.a"

⟨type Icon 14c⟩

// forward decls
typedef struct Panel Panel;  /* a Graphical User Interface element */
typedef struct Scroll Scroll;
typedef struct Rtext Rtext;  /* formattable text */
typedef struct Idol Idol;  /* A picture/text combo */

⟨struct Scroll 49e⟩
⟨struct Rtext 78c⟩
⟨struct Panel 11a⟩

/*
 * Panel flags
 */
⟨constant PACK 14n⟩
⟨constant PACKN 15a⟩
⟨constant PACKE 15b⟩
⟨constant PACKS 15c⟩
⟨constant PACKW 15d⟩

⟨constant PACKCEN 15e⟩

⟨constant FILLX 15f⟩
⟨constant FILLY 15g⟩

⟨constant PLACE 14d⟩
⟨constant PLACECEN 14e⟩

⟨constant PLACES 14f⟩
⟨constant PLACEE 14g⟩
⟨constant PLACEW 14h⟩
⟨constant PLACEN 14i⟩
⟨constant PLACENE 14j⟩
⟨constant PLACENW 14k⟩
⟨constant PLACESE 14l⟩
```

⟨*constant* PLACESW 14m⟩

⟨*constant* EXPAND 15h⟩

⟨*constant* FIXED 85d⟩
⟨*constant* FIXEDX 84f⟩
⟨*constant* FIXEDY 85a⟩

⟨*constant* MAXX 85e⟩
⟨*constant* MAXY 85f⟩

⟨*constant* BITMAP 27b⟩
⟨*constant* USERFL 32a⟩

⟨*constant* OUT 20a⟩

```
/*
 * Priorities
 */
```
⟨*constant* PRI_NORMAL 20g⟩
⟨*constant* PRI_POPUP 20h⟩
⟨*constant* PRI_SCROLLBAR 20i⟩

```
/* Rtext.flags */
```
⟨*constant* PL_HOT 79e⟩
⟨*constant* PL_SEL 79f⟩

⟨*global* plkbfocus 21c⟩

```
//pad's stuff
```
⟨*constant* NOFLAG 12d⟩

```
// Initialization
int plinit(int);   /* initialization */

// Memory
void plfree(Panel *);   /* give back space */

// Drawing
void pldraw(Panel *, Image *);  /* display the panel on the bitmap */

// Events
void plkeyboard(Rune);   /* send a keyboard event to the appropriate Panel */
void plmouse(Panel *, Mouse *);  /* send a Mouse event to a Panel tree */

// Packing
void plpack(Panel *, Rectangle); /* figure out where to put the Panel & children */
void plmove(Panel *, Point);  /* move an already-packed panel to a new location */


//XXX
void plgrabkb(Panel *);   /* this Panel should receive keyboard events */
void plscroll(Panel *, Panel *, Panel *); /* link up scroll bars */
void plepaste(Panel *, Rune *, int); /* paste in an edit window */

/*
 * Panel creation & reinitialization functions
 */
Panel *pllabel(Panel *pl, int, Icon *);
```

```
Panel *plbutton(Panel *pl, int, Icon *, void (*)(Panel *pl, int));
Panel *plcheckbutton(Panel *pl, int, Icon *, void (*)(Panel *pl, int, int));
Panel *plradiobutton(Panel *pl, int, Icon *, void (*)(Panel *pl, int, int));

Panel *plentry(Panel *pl, int, int, char *, void (*)(Panel *pl, char *));
Panel *pledit(Panel *, int, Point, Rune *, int, void (*)(Panel *));
Panel *plmessage(Panel *pl, int, int, char *);

Panel *plslider(Panel *pl, int, Point, void(*)(Panel *pl, int, int, int));

Panel *plcanvas(Panel *pl, int, void (*)(Panel *), void (*)(Panel *pl, Mouse *));

Panel *plgroup(Panel *pl, int);
Panel *plframe(Panel *pl, int);
Panel *pllist(Panel *pl, int, char *(*)(Panel *, int), int, void(*)(Panel *pl, int, int));
Panel *plidollist(Panel*, int, Point, Font*, Idol*, void (*)(Panel*, int, void*));

Panel *plmenu(Panel *pl, int, Icon **, int, void (*)(int, int));
Panel *plmenubar(Panel *pl, int, int, Icon *, Panel *pl, Icon *, ...);
Panel *plpopup(Panel *pl, int, Panel *pl, Panel *pl, Panel *pl);
Panel *plpulldown(Panel *pl, int, Icon *, Panel *pl, int);

Panel *plscrollbar(Panel *plparent, int flags);
Panel *pltextview(Panel *, int, Point, Rtext *, void (*)(Panel *, int, Rtext *));


// setters
void plplacelabel(Panel *, int); /* label placement */
void plsetbutton(Panel *, int);  /* set or clear the mark on a button */
void plsetslider(Panel *, int, int); /* set the value of a slider */
void plesel(Panel *, int, int);  /* set the selection in an edit window */
void plescroll(Panel *, int);  /* scroll an edit window */
void plsetscroll(Panel *, Scroll); /* set scrolling information */

// getters
char *plentryval(Panel *);  /* entry delivers its value */
Rune *pleget(Panel *);   /* get the text from an edit window */
int plelen(Panel *);   /* get the length of the text from an edit window */
void plegetsel(Panel *, int *, int *); /* get the selection from an edit window */
Scroll plgetscroll(Panel *);  /* get scrolling information from panel */


// plinitxxx()
void plinitlabel(Panel *, int, Icon *);

void plinitbutton(Panel *, int, Icon *, void (*)(Panel *, int));
void plinitcheckbutton(Panel *, int, Icon *, void (*)(Panel *, int, int));
void plinitradiobutton(Panel *, int, Icon *, void (*)(Panel *, int, int));

void plinitcanvas(Panel *, int, void (*)(Panel *), void (*)(Panel *, Mouse *));
void plinitedit(Panel *, int, Point, Rune *, int, void (*)(Panel *));
void plinitentry(Panel *, int, int, char *, void (*)(Panel *, char *));
void plinitframe(Panel *, int);
void plinitgroup(Panel *, int);
void plinitidollist(Panel*, int, Point, Font*, Idol*, void (*)(Panel*, int, void*));
void plinitlist(Panel *, int, char *(*)(Panel *, int), int, void(*)(Panel *, int, int));
void plinitmenu(Panel *, int, Icon **, int, void (*)(int, int));
void plinitmessage(Panel *, int, int, char *);
void plinitpopup(Panel *, int, Panel *, Panel *, Panel *);
void plinitpulldown(Panel *, int, Icon *, Panel *, int);
```

```
void plinitscrollbar(Panel *parent, int flags);
void plinitslider(Panel *, int, Point, void(*)(Panel *, int, int, int));
void plinittextview(Panel *, int, Point, Rtext *, void (*)(Panel *, int, Rtext *));

/*
 * Rtext constructors & destructor
 */
Rtext *plrtstr(Rtext **, int, int, Font *, char *, int, void *);
Rtext *plrtbitmap(Rtext **, int, int, Image *, int, void *);
Rtext *plrtpanel(Rtext **, int, int, Panel *, void *);
void plrtfree(Rtext *);
void plrtseltext(Rtext *, Rtext *, Rtext *);
char *plrtsnarftext(Rtext *);

int plgetpostextview(Panel *);
void plsetpostextview(Panel *, int);

/*
 * Idols
 */
Idol *plmkidol(Idol**, Image*, Image*, char*, void*);
void plfreeidol(Idol*);
Point plidolsize(Idol*, Font*, int);
void *plidollistgetsel(Panel*);

/*
 * Snarf
 */
void plputsnarf(char *);
char *plgetsnarf(void);
void plsnarf(Panel *);   /* snarf a panel */
void plpaste(Panel *);   /* paste a panel */
```

D.1.2 include/gui/rtext.h

⟨*constant* PL_NOPBIT 95a⟩≡ (96a)

```
#define PL_NOPBIT 4
```

⟨*constant* PL_NARGBIT 95b⟩≡ (96a)

```
#define PL_NARGBIT 12
```

⟨*constant* PL_ARGMASK 95c⟩≡ (96a)

```
#define PL_ARGMASK ((1<<PL_NARGBIT)-1)
```

⟨*function* PL_SPECIAL 95d⟩≡ (96a)

```
#define PL_SPECIAL(op) (((-1<<PL_NOPBIT)|op)<<PL_NARGBIT)
```

⟨*function* PL_OP 95e⟩≡ (96a)

```
#define PL_OP(t) ((t)&~PL_ARGMASK)
```

⟨*function* PL_ARG 95f⟩≡ (96a)

```
#define PL_ARG(t) ((t)&PL_ARGMASK)
```

⟨*constant* PL_TAB 95g⟩≡ (96a)

```
#define PL_TAB  PL_SPECIAL(0)  /* # of tab stops before text */
```

⟨include/gui/rtext.h 96a⟩≡

```
/*
 * Rtext definitions
 */
⟨constant PL_NOPBIT 95a⟩
⟨constant PL_NARGBIT 95b⟩
⟨constant PL_ARGMASK 95c⟩
⟨function PL_SPECIAL 95d⟩

⟨function PL_OP 95e⟩
⟨function PL_ARG 95f⟩
⟨constant PL_TAB 95g⟩

void pltabsize(int, int);   /* set min tab and tab size */
```

D.2 lib_gui/libpanel/

D.2.1 lib_gui/libpanel/pldefs.h

⟨enum _anon_ (lib_gui/libpanel/pldefs.h)2 96b⟩≡ (96d)

```
/*
 * Scroll flags
 */
enum{
    SCROLLUP,
    SCROLLDOWN,
    SCROLLABSY,
    SCROLLLEFT,
    SCROLLRIGHT,
    SCROLLABSX,
};
```

⟨struct Textwin 96c⟩≡ (96d)

```
struct Textwin{
    Rune *text, *etext, *eslack; /* text, with some slack off the end */
    int top, bot;   /* range of runes visible on screen */
    int sel0, sel1;   /* selection */
    Point *loc, *eloc;  /* ul corners of visible runes (+1 more at end!) */
    Image *b;   /* bitmap the text is drawn in */
    Rectangle r;   /* rectangle the text is drawn in */
    Font *font;   /* font text is drawn in */
    int hgt;   /* same as font->height */
    int tabstop;   /* tab settings are every tabstop pixels */
    int mintab;   /* the minimum size of a tab */
};
```

⟨lib_gui/libpanel/pldefs.h 96d⟩≡

```
/*
 * Definitions for internal use only
 */

/*
 * Variable-font text routines
 * These could make a separate library.
 */
int pl_rtfmt(Rtext *, int);
void pl_rtdraw(Image *, Rectangle, Rtext *, int);
```

```
void pl_rtredraw(Image *, Rectangle, Rtext *, int, int);
Rtext *pl_rthit(Rtext *, int, Point, Point);

⟨constant LEAF 13c⟩
⟨constant INVIS 63b⟩
⟨constant REMOUSE 20c⟩

⟨enum Style 13j⟩
⟨enum _anon_ (lib_gui/libpanel/pldefs.h)2 96b⟩
⟨enum Direction 15k⟩

Panel *pl_newpanel(Panel *, int); /* make a new Panel, given parent & data size */
void *pl_emalloc(int);   /* allocate some space, exit on error */
void *pl_erealloc(void*,int);  /* reallocate some space, exit on error */

void pl_print(Panel *);   /* print a Panel tree */
Panel *pl_ptinpanel(Point, Panel *); /* highest-priority subpanel containing point */

/*
 * Drawing primitives
 */
int pl_drawinit(int);
Rectangle pl_box(Image *, Rectangle, int);
Rectangle pl_outline(Image *, Rectangle, int);
Point pl_boxsize(Point, int);
void pl_interior(int, Point *, Point *);
void pl_drawicon(Image *, Rectangle, int, int, Icon *);
Rectangle pl_check(Image *, Rectangle, int);
Rectangle pl_radio(Image *, Rectangle, int);
int pl_ckwid(void);
void pl_sliderupd(Image *, Rectangle, int, int, int);
void pl_invis(Panel *, bool);
Point pl_iconsize(int, Icon *);
void pl_highlight(Image *, Rectangle);
void pl_clr(Image *, Rectangle);
void pl_fill(Image *, Rectangle);
void pl_cpy(Image *, Point, Rectangle);

/*
 * Rune mangling functions
 */
int pl_idchar(int);
int pl_rune1st(int);
char *pl_nextrune(char *);
int pl_runewidth(Font *, char *);

/*
 * Fixed-font Text-window routines
 * These could be separated out into a separate library.
 */
typedef struct Textwin Textwin;
⟨struct Textwin 96c⟩

Textwin *twnew(Image *, Font *, Rune *, int);
void twfree(Textwin *);
void twhilite(Textwin *, int, int, int);
void twselect(Textwin *, Mouse *);
void twreplace(Textwin *, int, int, Rune *, int);
void twscroll(Textwin *, int);
int twpt2rune(Textwin *, Point);
```

```
void twreshape(Textwin *, Rectangle);
void twmove(Textwin *, Point);
void plemove(Panel *, Point);
```

D.2.2 lib_gui/libpanel/init.c

⟨lib_gui/libpanel/init.c 98a⟩≡
⟨libpanel *includes* 10e⟩

⟨*function* plinit 16a⟩

D.2.3 lib_gui/libpanel/mem.c

⟨lib_gui/libpanel/mem.c 98b⟩≡
⟨libpanel *includes* 10e⟩

⟨*function* pl_emalloc 91a⟩
⟨*function* pl_erealloc 91b⟩

⟨*function* pl_unexpected 90a⟩
⟨*function* pl_drawerror 90b⟩
⟨*function* pl_hiterror 90c⟩
⟨*function* pl_typeerror 90d⟩
⟨*function* pl_getsizeerror 90e⟩
⟨*function* pl_childspaceerror 90f⟩
⟨*function* pl_scrollerror 90g⟩
⟨*function* pl_setscrollbarerror 90h⟩

⟨*function* pl_prinormal 20k⟩

⟨*function* pl_newpanel 11b⟩
⟨*function* plfree 12b⟩

D.2.4 lib_gui/libpanel/draw.c

⟨lib_gui/libpanel/draw.c 98c⟩≡
⟨libpanel *includes* 10e⟩

⟨*constant* PWID 28e⟩
⟨*constant* BWID 31f⟩
⟨*constant* FWID 47a⟩
⟨*constant* SPACE 18d⟩
⟨*constant* CKSIZE 39b⟩
⟨*constant* CKSPACE 40a⟩
⟨*constant* CKWID 38c⟩
⟨*constant* CKINSET 38b⟩
⟨*constant* CKBORDER 38d⟩

⟨*global* plldepth 16b⟩

⟨*globals* pl_xxx 16c⟩

⟨*function* pl_drawinit 16d⟩
⟨*function* pl_relief 17d⟩
⟨*function* pl_boxoutline 18c⟩
⟨*function* pl_outline 18a⟩
⟨*function* pl_box 18b⟩

⟨*function* pl_boxsize 18e⟩
⟨*function* pl_interior 18f⟩

⟨*function* pl_drawicon 28f⟩
⟨*function* pl_radio 39g⟩
⟨*function* pl_check 38a⟩
⟨*function* pl_ckwid 39a⟩
⟨*function* pl_sliderupd 41d⟩

```
void pl_draw1(Panel *p, Image *b);
```

⟨*function* pl_drawall 17c⟩
⟨*function* pl_draw1 17b⟩
⟨*function* pldraw 17a⟩
⟨*function* pl_invis 63a⟩
⟨*function* pl_iconsize 28g⟩
⟨*function* pl_highlight 52b⟩
⟨*function* pl_clr 80a⟩
⟨*function* pl_fill 52c⟩
⟨*function* pl_cpy 54c⟩

D.2.5 lib_gui/libpanel/event.c

⟨lib_gui/libpanel/event.c 99a⟩≡
⟨libpanel *includes* 10e⟩

⟨*function* plgrabkb 21d⟩
⟨*function* plkeyboard 21e⟩

⟨*function* pl_ptinpanel 19d⟩
⟨*function* plmouse 19a⟩

D.2.6 lib_gui/libpanel/print.c

⟨lib_gui/libpanel/print.c 99b⟩≡
⟨libpanel *includes* 10e⟩

⟨*function* pl_iprint 88c⟩
⟨*function* pl_ipprint 88d⟩
⟨*function* pl_print 88b⟩

D.2.7 lib_gui/libpanel/label.c

⟨lib_gui/libpanel/label.c 99c⟩≡
⟨libpanel *includes* 10e⟩

```
typedef struct Label Label;
```

⟨*struct* Label 27a⟩

⟨*function* pl_drawlabel 28b⟩
⟨*function* pl_hitlabel 29e⟩
⟨*function* pl_typelabel 29f⟩
⟨*function* pl_getsizelabel 29g⟩
⟨*function* pl_childspacelabel 29i⟩
⟨*function* plinitlabel 27d⟩
⟨*function* pllabel 27c⟩
⟨*function* plplacelabel 28a⟩

D.2.8 lib_gui/libpanel/button.c

```
⟨lib_gui/libpanel/button.c 100a⟩≡
  ⟨libpanel includes 10e⟩

  typedef struct Button Button;

  ⟨struct Button 34e⟩

  /*
   * Button types
   */
  ⟨constant BUTTON 34f⟩
  ⟨constant CHECK 34g⟩
  ⟨constant RADIO 35a⟩

  ⟨function pl_drawbutton 36b⟩
  ⟨function pl_hitbutton 36d⟩
  ⟨function pl_typebutton 37a⟩
  ⟨function pl_getsizebutton 37b⟩
  ⟨function pl_childspacebutton 37c⟩
  ⟨function pl_initbtype 35d⟩
  ⟨function pl_buttonhit 36a⟩
  ⟨function plinitbutton 35f⟩
  ⟨function plinitcheckbutton 37e⟩
  ⟨function plinitradiobutton 39e⟩
  ⟨function plbutton 35e⟩
  ⟨function plcheckbutton 37d⟩
  ⟨function plradiobutton 39d⟩

  ⟨function pl_hitmenu 60a⟩
  ⟨function plinitmenu 59d⟩
  ⟨function plmenu 59c⟩
  ⟨function plsetbutton 39c⟩
```

D.2.9 lib_gui/libpanel/entry.c

```
⟨lib_gui/libpanel/entry.c 100b⟩≡
  ⟨libpanel includes 10e⟩
  #include <keyboard.h>

  typedef struct Entry Entry;

  ⟨struct Entry 30a⟩
  ⟨constant SLACK 84a⟩

  ⟨function pl_snarfentry 83d⟩
  ⟨function pl_pasteentry 83e⟩
  ⟨function pl_drawentry 31d⟩
  ⟨function pl_hitentry 32d⟩
  ⟨function pl_typeentry 32e⟩
  ⟨function pl_getsizeentry 34a⟩
  ⟨function pl_childspaceentry 34c⟩
  ⟨function pl_freeentry 30e⟩
  ⟨function plinitentry 30d⟩
  ⟨function plentry 30c⟩
  ⟨function plentryval 34d⟩
```

D.2.10 lib_gui/libpanel/edit.c

⟨lib_gui/libpanel/edit.c 101a⟩≡

```
/*
 * Interface includes:
 * void plescroll(Panel *p, int top);
 *  move the given character position onto the top line
 * void plegetsel(Panel *p, int *sel0, int *sel1);
 *  read the selection back
 * int plelen(Panel *p);
 *  read the length of the text back
 * Rune *pleget(Panel *p);
 *  get a pointer to the text
 * void plesel(Panel *p, int sel0, int sel1);
 *  set the selection -- adjusts hiliting
 * void plepaste(Panel *p, Rune *text, int ntext);
 *  replace the selection with the given text
 */
⟨libpanel includes 10e⟩
#include <keyboard.h>

typedef struct Edit Edit;

⟨struct Edit 70c⟩
⟨function pl_drawedit 71c⟩

⟨function pl_snarfedit 84c⟩
⟨function pl_pasteedit 84d⟩

⟨function pl_hitedit 72a⟩
⟨function pl_scrolledit 73c⟩
⟨function pl_typeedit 72b⟩
⟨function pl_getsizeedit 73a⟩
⟨function pl_childspaceedit 73b⟩
⟨function pl_freeedit 71b⟩
⟨function plinitedit 71a⟩
⟨function pledit 70d⟩
⟨function plescroll 74a⟩
⟨function plegetsel 74b⟩
⟨function plelen 74c⟩
⟨function pleget 75a⟩
⟨function plesel 75b⟩
⟨function plepaste 75c⟩
⟨function plemove 75d⟩
```

D.2.11 lib_gui/libpanel/slider.c

⟨lib_gui/libpanel/slider.c 101b⟩≡

```
⟨libpanel includes 10e⟩

typedef struct Slider Slider;

⟨struct Slider 40d⟩

⟨function pl_drawslider 41c⟩
⟨function pl_hitslider 42b⟩
⟨function pl_typeslider 43c⟩
⟨function pl_getsizeslider 43d⟩
⟨function pl_childspaceslider 43e⟩
```

⟨*function* plinitslider 41b⟩
⟨*function* plslider 41a⟩
⟨*function* plsetslider 43f⟩

D.2.12 lib_gui/libpanel/canvas.c

⟨lib_gui/libpanel/canvas.c 102a⟩≡
⟨libpanel *includes* 10e⟩

```
typedef struct Canvas Canvas;
```

⟨*struct* Canvas 43h⟩

⟨*function* pl_drawcanvas 44c⟩
⟨*function* pl_hitcanvas 44d⟩
⟨*function* pl_typecanvas 44e⟩
⟨*function* pl_getsizecanvas 45a⟩
⟨*function* pl_childspacecanvas 45b⟩
⟨*function* plinitcanvas 44b⟩
⟨*function* plcanvas 44a⟩

D.2.13 lib_gui/libpanel/frame.c

⟨lib_gui/libpanel/frame.c 102b⟩≡
⟨libpanel *includes* 10e⟩

⟨*function* pl_drawframe 46c⟩
⟨*function* pl_hitframe 47b⟩
⟨*function* pl_typeframe 47c⟩
⟨*function* pl_getsizeframe 47d⟩
⟨*function* pl_childspaceframe 47f⟩
⟨*function* plinitframe 46b⟩
⟨*function* plframe 46a⟩

D.2.14 lib_gui/libpanel/group.c

⟨lib_gui/libpanel/group.c 102c⟩≡
⟨libpanel *includes* 10e⟩

⟨*function* pl_drawgroup 48b⟩
⟨*function* pl_hitgroup 48c⟩
⟨*function* pl_typegroup 48d⟩
⟨*function* pl_getsizegroup 48e⟩
⟨*function* pl_childspacegroup 48f⟩
⟨*function* plinitgroup 48a⟩
⟨*function* plgroup 47h⟩

D.2.15 lib_gui/libpanel/list.c

⟨lib_gui/libpanel/list.c 102d⟩≡
⟨libpanel *includes* 10e⟩

```
typedef struct List List;
```

⟨*struct* List 50a⟩

⟨*function* pl_listsel 52a⟩
⟨*function* pl_liststrings 51f⟩
⟨*function* pl_drawlist 51e⟩
⟨*function* pl_hitlist 52f⟩
⟨*function* pl_scrolllist 53e⟩
⟨*function* pl_typelist 53a⟩
⟨*function* pl_getsizelist 53b⟩
⟨*function* pl_childspacelist 53c⟩
⟨*function* plinitlist 50d⟩
⟨*function* pllist 50c⟩

D.2.16 lib_gui/libpanel/message.c

⟨lib_gui/libpanel/message.c 103a⟩≡

```
⟨libpanel includes 10e⟩

typedef struct Message Message;

⟨struct Message 68a⟩

⟨function pl_textmsg 69b⟩
⟨function pl_foldsize 70a⟩
⟨function pl_drawmessage 69a⟩
⟨function pl_hitmessage 69c⟩
⟨function pl_typemessage 69d⟩
⟨function pl_getsizemessage 69e⟩
⟨function pl_childspacemessage 70b⟩
⟨function plinitmessage 68c⟩
⟨function plmessage 68b⟩
```

D.2.17 lib_gui/libpanel/pack.c

⟨lib_gui/libpanel/pack.c 103b⟩≡

```
⟨libpanel includes 10e⟩

⟨function pl_max 23g⟩
⟨function pl_sizesibs 23c⟩
⟨function pl_sizereq 22f⟩
⟨function pl_getshare 26d⟩
⟨function pl_setrect 24a⟩
⟨function plpack 22a⟩
⟨function plmove 62d⟩
```

D.2.18 lib_gui/libpanel/popup.c

⟨lib_gui/libpanel/popup.c 103c⟩≡

```
/*
 * popup
 * looks like a group, except diverts hits on certain buttons to
 * panels that it temporarily pops up.
 */
⟨libpanel includes 10e⟩

typedef struct Popup Popup;

⟨struct Popup 60b⟩
```

⟨*function* pl_drawpopup 62a⟩
⟨*function* pl_hitpopup 61b⟩
⟨*function* pl_typepopup 61f⟩
⟨*function* pl_getsizepopup 63d⟩
⟨*function* pl_childspacepopup 63e⟩
⟨*function* pl_pripopup 21a⟩
⟨*function* plinitpopup 60d⟩
⟨*function* plpopup 60c⟩

D.2.19 lib_gui/libpanel/pulldown.c

⟨lib_gui/libpanel/pulldown.c 104a⟩≡

```
/*
 * pulldown
 * makes a button that pops up a panel when hit
 */
⟨libpanel includes 10e⟩

typedef struct Pulldown Pulldown;

⟨struct Pulldown 63f⟩

⟨function pl_drawpulldown 64b⟩
⟨function pl_hitpulldown 64c⟩
⟨function pl_typepulldown 66b⟩
⟨function pl_getsizepulldown 66c⟩
⟨function pl_childspacepulldown 66d⟩
⟨function plinitpulldown 64a⟩
⟨function plpulldown 63g⟩
⟨function plmenubar 66e⟩
```

D.2.20 lib_gui/libpanel/rtext.c

⟨*constant* LEAD 104b⟩≡ (107)

```
#define LEAD 4 /* extra space between lines */
```

⟨*constant* BORD 104c⟩≡ (107)

```
#define BORD 2 /* extra border for images */
```

⟨*function* pltabsize 104d⟩≡ (107)

```
void pltabsize(int min, int size){
    pl_tabmin=min;
    pl_tabsize=size;
}
```

⟨*function* pl_space 104e⟩≡ (107)

```
int pl_space(int space, int pos, int indent){
    if(space>=0) return space;
    switch(PL_OP(space)){
    default:
        return 0;
    case PL_TAB:
        return ((pos-indent+pl_tabmin)/pl_tabsize+PL_ARG(space))*pl_tabsize+indent-pos;
    }
}
```

⟨*function* pl_rtfmt 105⟩≡ (107)

```
/*
 * initialize rectangles & nextlines of text starting at t,
 * galley width is wid.  Returns the total length of the text
 */
int pl_rtfmt(Rtext *t, int wid){
    Rtext *tp, *eline;
    int ascent, descent, x, space, a, d, w, topy, indent;
    Point p;
    p=Pt(0,0);
    eline=t;
    while(t){
        ascent=0;
        descent=0;
        indent=space=pl_space(t->indent, 0, 0);
        x=0;
        tp=t;
        for(;;){
            if(tp->b){
                a=tp->b->r.max.y-tp->b->r.min.y+BORD;
                d=BORD;
                w=tp->b->r.max.x-tp->b->r.min.x+BORD*2;
            }
            else if(tp->p){
                /* what if plpack fails? */
                plpack(tp->p, Rect(0,0,wid,wid));
                plmove(tp->p, subpt(Pt(0,0), tp->p->r.min));
                a=tp->p->r.max.y-tp->p->r.min.y;
                d=0;
                w=tp->p->r.max.x-tp->p->r.min.x;
            }
            else{
                a=tp->font->ascent;
                d=tp->font->height-a;
                w=tp->wid=stringwidth(tp->font, tp->text);
            }
            if(x+w+space>wid) break;
            if(a>ascent) ascent=a;
            if(d>descent) descent=d;
            x+=w+space;
            tp=tp->next;
            if(tp==0){
                eline=0;
                break;
            }
            space=pl_space(tp->space, x, indent);
            if(space) eline=tp;
        }
        if(eline==t){ /* No progress!  Force fit the first block! */
            if(tp==t){
                if(a>ascent) ascent=a;
                if(d>descent) descent=d;
                eline=tp->next;
            }else
                eline=tp;
        }
        topy=p.y;
        p.y+=ascent;
        p.x=indent=pl_space(t->indent, 0, 0);
        for(;;){
```

```
            t->topy=topy;
            t->r.min.x=p.x;
            if(t->b){
                t->r.max.y=p.y+BORD;
                t->r.min.y=p.y-(t->b->r.max.y-t->b->r.min.y)-BORD;
                p.x+=(t->b->r.max.x-t->b->r.min.x)+BORD*2;
            }
            else if(t->p){
                t->r.max.y=p.y;
                t->r.min.y=p.y-t->p->r.max.y;
                p.x+=t->p->r.max.x;
            }
            else{
                t->r.min.y=p.y-t->font->ascent;
                t->r.max.y=t->r.min.y+t->font->height;
                p.x+=t->wid;
            }
            t->r.max.x=p.x;
            t->nextline=eline;
            t=t->next;
            if(t==eline) break;
            p.x+=pl_space(t->space, p.x, indent);
        }
        p.y+=descent+LEAD;
    }
    return p.y;
}
```

⟨*function* pl_stuffbitmap 106a⟩≡ (107)

```
/*
 * If we draw the text in a backup bitmap and copy it onto the screen,
 * the bitmap pointers in all the subpanels point to the wrong bitmap.
 * This code fixes them.
 */
void pl_stuffbitmap(Panel *p, Image *b){
    p->b=b;
    for(p=p->child;p;p=p->next)
        pl_stuffbitmap(p, b);
}
```

⟨*function* pl_reposition 106b⟩≡ (107)

```
/*
 * Reposition text already drawn in the window.
 * We just move the pixels and update the positions of any
 * enclosed panels
 */
void pl_reposition(Rtext *t, Image *b, Point p, Rectangle r){
    Point offs;
    pl_cpy(b, p, r);
    offs=subpt(p, r.min);
    for(;t;t=t->next)
        if(!eqrect(t->r, Rect(0,0,0,0)) && !t->b && t->p)
            plmove(t->p, offs);
}
```

⟨*function* plrtsnarftext 106c⟩≡ (107)

```
char *plrtsnarftext(Rtext *w){
    char *b, *p, *e, *t;
    int n;
```

```
    b=p=e=0;
    for(; w; w = w->next){
        if((w->flags&PL_SEL)==0 || w->text==0)
            continue;
        n = strlen(w->text)+64;
        if(p+n >= e){
            n = (p+n+64)-b;
            t = pl_erealloc(b, n);
            p = t+(p-b);
            e = t+n;
            b = t;
        }
        if(w->space == 0)
            p += sprint(p, "%s", w->text);
        else if(w->space > 0)
            p += sprint(p, " %s", w->text);
        else if(PL_OP(w->space) == PL_TAB)
            p += sprint(p, "\t%s", w->text);
        if(w->nextline == w->next)
            p += sprint(p, "\n");
    }
    return b;
}
```

⟨lib_gui/libpanel/rtext.c 107⟩≡

```
/*
 * Rich text with images.
 * Should there be an offset field, to do subscripts & kerning?
 */
⟨libpanel includes 10e⟩
#include "rtext.h"

⟨constant LEAD 104b⟩
⟨constant BORD 104c⟩

⟨function pl_rtnew 78d⟩
⟨function plrtpanel 79a⟩
⟨function plrtstr 79b⟩
⟨function plrtbitmap 79c⟩
⟨function plrtfree 79d⟩
int pl_tabmin;
int pl_tabsize;
⟨function pltabsize 104d⟩
⟨function pl_space 104e⟩
⟨function pl_rtfmt 105⟩

⟨function pl_stuffbitmap 106a⟩

⟨function pl_rtdraw 79g⟩
⟨function pl_reposition 106b⟩
⟨function pl_rtredraw 80b⟩
⟨function pl_rthit 81a⟩

⟨function plrtseltext 81b⟩

⟨function plrtsnarftext 106c⟩
```

D.2.21 lib_gui/libpanel/scroll.c

⟨*function* plgetscroll 108a⟩≡ (108c)

```
Scroll plgetscroll(Panel *p){
    return p->scr;
}
```

⟨*function* plsetscroll 108b⟩≡ (108c)

```
void plsetscroll(Panel *p, Scroll s){
    if(p->scroll){
        if(s.size.x) p->scroll(p, HORIZ, 2, s.pos.x, s.size.x);
        if(s.size.y) p->scroll(p, VERT, 2, s.pos.y, s.size.y);
    }
}
```

⟨lib_gui/libpanel/scroll.c 108c⟩≡

```
⟨libpanel includes 10e⟩

⟨function plscroll 49c⟩
⟨function plgetscroll 108a⟩
⟨function plsetscroll 108b⟩
```

D.2.22 lib_gui/libpanel/scrollbar.c

⟨lib_gui/libpanel/scrollbar.c 108d⟩≡

```
⟨libpanel includes 10e⟩

typedef struct Scrollbar Scrollbar;

⟨struct Scrollbar 55a⟩
⟨constant SBWID 56a⟩

⟨function pl_drawscrollbar 56d⟩
⟨function pl_hitscrollbar 56e⟩
⟨function pl_typescrollbar 57c⟩
⟨function pl_getsizescrollbar 57d⟩
⟨function pl_childspacescrollbar 57e⟩
⟨function pl_setscrollbarscrollbar 58b⟩
⟨function pl_priscrollbar 21b⟩
⟨function plinitscrollbar 55d⟩
⟨function plscrollbar 55c⟩
```

D.2.23 lib_gui/libpanel/snarf.c

⟨lib_gui/libpanel/snarf.c 108e⟩≡

```
⟨libpanel includes 10e⟩

⟨function plputsnarf 82c⟩
⟨function plgetsnarf 82d⟩
⟨function plsnarf 83a⟩
⟨function plpaste 83b⟩
```

D.2.24 lib_gui/libpanel/textview.c

⟨*function* pl_setscrpos 109a⟩≡ (110d)

```
void pl_setscrpos(Panel *p, Textview *tp, Rectangle r){
    Panel *sb;
    int lo, hi;
    lo=tp->yoffs;
    hi=lo+r.max.y-r.min.y; /* wrong? */
    sb=p->yscroller;
    if(sb && sb->setscrollbar) sb->setscrollbar(sb, lo, hi, tp->thgt);
}
```

⟨*function* pl_passon 109b⟩≡ (110d)

```
/*
 * If t is a panel word, pass the mouse event on to it
 */
void pl_passon(Rtext *t, Mouse *m){
    if(t && t->b==0 && t->p!=0)
        plmouse(t->p, m);
}
```

⟨*function* pl_scrolltextview 109c⟩≡ (110d)

```
void pl_scrolltextview(Panel *p, int dir, int buttons, int num, int den){
    int yoffs;
    Point ul, size;
    Textview *tp;
    Rectangle r;
    if(dir!=VERT) return;

    tp=p->data;
    ul=p->r.min;
    size=subpt(p->r.max, p->r.min);
    pl_interior(p->state, &ul, &size);
    switch(buttons){
    default:
        SET(yoffs);
        break;
    case 1:  /* left -- top moves to pointer */
        yoffs=(vlong)tp->yoffs-num*size.y/den;
        if(yoffs<0) yoffs=0;
        break;
    case 2:  /* middle -- absolute index of file */
        yoffs=(vlong)tp->thgt*num/den;
        break;
    case 4:  /* right -- line pointed at moves to top */
        yoffs=tp->yoffs+(vlong)num*size.y/den;
        if(yoffs>tp->thgt) yoffs=tp->thgt;
        break;
    }
    if(yoffs!=tp->yoffs){
        r=pl_outline(p->b, p->r, p->state);
        pl_rtredraw(p->b, r, tp->text, yoffs, tp->yoffs);
        p->scr.pos.y=tp->yoffs=yoffs;
        pl_setscrpos(p, tp, r);
    }
}
```

⟨*function* pl_pritextview 109d⟩≡ (110d)

```
/*
 * Priority depends on what thing inside the panel we're pointing at.
```

```
 */
int pl_pritextview(Panel *p, Point xy){
    Point ul, size;
    Textview *tp;
    Rtext *h;
    tp=p->data;
    ul=p->r.min;
    size=subpt(p->r.max, p->r.min);
    pl_interior(p->state, &ul, &size);
    h=pl_rthit(tp->text, tp->yoffs, xy, ul);
    if(h && h->b==0 && h->p!=0){
        p=pl_ptinpanel(xy, h->p);
        if(p) return p->pri(p, xy);
    }
    return PRI_NORMAL;
}
```

⟨*function* pl_snarftextview 110a⟩≡ (110d)

```
char* pl_snarftextview(Panel *p){
    return plrtsnarftext(((Textview *)p->data)->text);
}
```

⟨*function* plgetpostextview 110b⟩≡ (110d)

```
int plgetpostextview(Panel *p){
    return ((Textview *)p->data)->yoffs;
}
```

⟨*function* plsetpostextview 110c⟩≡ (110d)

```
void plsetpostextview(Panel *p, int yoffs){
    ((Textview *)p->data)->yoffs=yoffs;
    pldraw(p, p->b);
}
```

⟨lib_gui/libpanel/textview.c 110d⟩≡

```
/*
 * Fonted text viewer, calls out to code in rtext.c
 *
 * Should redo this to copy the already-visible parts on scrolling & only
 * update the newly appearing stuff -- then the offscreen assembly bitmap can go away.
 */
⟨libpanel includes 10e⟩

typedef struct Textview Textview;
⟨struct Textview 75e⟩

⟨function pl_setscrpos 109a⟩
⟨function pl_drawtextview 76c⟩
⟨function pl_passon 109b⟩
⟨function pl_hittextview 76d⟩
⟨function pl_scrolltextview 109c⟩
⟨function pl_typetextview 77⟩
⟨function pl_getsizetextview 78a⟩
⟨function pl_childspacetextview 78b⟩
⟨function pl_pritextview 109d⟩

⟨function pl_snarftextview 110a⟩

⟨function plinittextview 76b⟩
⟨function pltextview 76a⟩
⟨function plgetpostextview 110b⟩
⟨function plsetpostextview 110c⟩
```

D.2.25 lib_gui/libpanel/textwin.c

⟨*function* tw_before 111a⟩≡ (119a)

```
/*
 * Is text at point a before or after that at point b?
 */
int tw_before(Textwin *t, Point a, Point b){
    return a.y<b.y || a.y<b.y+t->hgt && a.x<b.x;
}
```

⟨*function* twpt2rune 111b⟩≡ (119a)

```
/*
 * Return the character index indicated by point p, or -1
 * if its off-screen.  The screen must be up-to-date.
 *
 * Linear search should be binary search.
 */
int twpt2rune(Textwin *t, Point p){
    Point *el, *lp;
    el=t->loc+(t->bot-t->top);
    for(lp=t->loc;lp!=el;lp++)
        if(tw_before(t, p, *lp)){
            if(lp==t->loc) return t->top;
            return lp-t->loc+t->top-1;
        }
    return t->bot;
}
```

⟨*function* tw_rune2pt 111c⟩≡ (119a)

```
/*
 * Return ul corner of the character with the given index
 */
Point tw_rune2pt(Textwin *t, int i){
    if(i<t->top) return t->r.min;
    if(i>t->bot) return t->r.max;
    return t->loc[i-t->top];
}
```

⟨*function* tw_storeloc 111d⟩≡ (119a)

```
/*
 * Store p at t->loc[l], extending t->loc if necessary
 */
void tw_storeloc(Textwin *t, int l, Point p){
    int nloc;
    if(l>=t->eloc-t->loc){
        nloc=l+SLACK;
        t->loc=pl_erealloc(t->loc, nloc*sizeof(Point));
        t->eloc=t->loc+nloc;
    }
    t->loc[l]=p;
}
```

⟨*function* tw_setloc 111e⟩≡ (119a)

```
/*
 * Set the locations at which the given runes should appear.
 * Returns the index of the first rune not set, which might not
 * be last because we reached the bottom of the window.
 *
 * N.B. this zaps the loc of r[last], so that value should be saved first,
 * if it's important.
 */
```

```
int tw_setloc(Textwin *t, int first, int last, Point ul){
    Rune *r, *er;
    int x, dt, lp;
    char buf[UTFmax+1];
    er=t->text+last;
    for(r=t->text+first,lp=first-t->top;r!=er && ul.y+t->hgt<=t->r.max.y;r++,lp++){
        tw_storeloc(t, lp, ul);
        switch(*r){
        case '\n':
            ul.x=t->r.min.x;
            ul.y+=t->hgt;
            break;
        case '\t':
            x=ul.x-t->r.min.x+t->mintab+t->tabstop;
            x-=x%t->tabstop;
            ul.x=x+t->r.min.x;
            if(ul.x>t->r.max.x){
                ul.x=t->r.min.x;
                ul.y+=t->hgt;
                tw_storeloc(t, lp, ul);
                if(ul.y+t->hgt>t->r.max.y) return r-t->text;
                ul.x+=+t->tabstop;
            }
            break;
        default:
            buf[runetochar(buf, r)]='\0';
            dt=stringwidth(t->font, buf);
            ul.x+=dt;
            if(ul.x>t->r.max.x){
                ul.x=t->r.min.x;
                ul.y+=t->hgt;
                tw_storeloc(t, lp, ul);
                if(ul.y+t->hgt>t->r.max.y) return r-t->text;
                ul.x+=dt;
            }
            break;
        }
    }
    tw_storeloc(t, lp, ul);
    return r-t->text;
}
```

⟨*function* tw_draw 112⟩≡ (119a)

```
/*
 * Draw the given runes at their locations.
 * Bug -- saving up multiple characters would
 * reduce the number of calls to string,
 * and probably make this a lot faster.
 */
void tw_draw(Textwin *t, int first, int last){
    Rune *r, *er;
    Point *lp, ul, ur;
    char buf[UTFmax+1];
    if(first<t->top) first=t->top;
    if(last>t->bot) last=t->bot;
    if(last<=first) return;
    er=t->text+last;
    for(r=t->text+first,lp=t->loc+(first-t->top);r!=er;r++,lp++){
        if(lp->y+t->hgt>t->r.max.y){
            fprint(2, "chr %C, index %ld of %d, loc %d %d, off bottom\n",
```

```
                *r, lp-t->loc, t->bot-t->top, lp->x, lp->y);
            return;
        }
        switch(*r){
        case '\n':
            ur=*lp;
            break;
        case '\t':
            ur=*lp;
            if(lp[1].y!=lp[0].y)
                ul=Pt(t->r.min.x, lp[1].y);
            else
                ul=*lp;
            pl_clr(t->b, Rpt(ul, Pt(lp[1].x, ul.y+t->hgt)));
            break;
        default:
            buf[runetochar(buf, r)]='\0';
    /***/  pl_clr(t->b, Rpt(*lp, addpt(*lp, stringsize(t->font, buf))));
            ur=string(t->b, *lp, display->black, ZP, t->font, buf);
            break;
        }
        if(lp[1].y!=lp[0].y)
    /***/  pl_clr(t->b, Rpt(ur, Pt(t->r.max.x, ur.y+t->hgt)));
    }
}
```

⟨*function* tw_hilitep 113a⟩≡ (119a)

```
/*
 * Hilight the characters with tops between ul and ur
 */
void tw_hilitep(Textwin *t, Point ul, Point ur){
    Point swap;
    int y;
    if(tw_before(t, ur, ul)){ swap=ul; ul=ur; ur=swap;}
    y=ul.y+t->hgt;
    if(y>t->r.max.y) y=t->r.max.y;
    if(ul.y==ur.y)
        pl_highlight(t->b, Rpt(ul, Pt(ur.x, y)));
    else{
        pl_highlight(t->b, Rpt(ul, Pt(t->r.max.x, y)));
        ul=Pt(t->r.min.x, y);
        pl_highlight(t->b, Rpt(ul, Pt(t->r.max.x, ur.y)));
        ul=Pt(t->r.min.x, ur.y);
        y=ur.y+t->hgt;
        if(y>t->r.max.y) y=t->r.max.y;
        pl_highlight(t->b, Rpt(ul, Pt(ur.x, y)));
    }
}
```

⟨*function* twhilite 113b⟩≡ (119a)

```
/*
 * Hilite/unhilite the given range of characters
 */
void twhilite(Textwin *t, int sel0, int sel1, int on){
    Point ul, ur;
    int swap, y;
    if(sel1<sel0){ swap=sel0; sel0=sel1; sel1=swap; }
    if(sel1<t->top || t->bot<sel0) return;
    if(sel0<t->top) sel0=t->top;
    if(sel1>t->bot) sel1=t->bot;
```

```
    if(!on){
        if(sel1==sel0){
            ul=t->loc[sel0-t->top];
            y=ul.y+t->hgt;
            if(y>t->r.max.y) y=t->r.max.y;
            pl_clr(t->b, Rpt(ul, Pt(ul.x+1, y)));
        }else
            tw_draw(t, sel0, sel1);
        return;
    }
    ul=t->loc[sel0-t->top];
    if(sel1==sel0)
        ur=addpt(ul, Pt(1, 0));
    else
        ur=t->loc[sel1-t->top];
    tw_hilitep(t, ul, ur);
}
```

⟨*function* twselect 114⟩≡ (119a)

```
/*
 * Set t->sel[01] from mouse input.
 * Also hilites the selection.
 * Caller should unhilite the previous
 * selection before calling this.
 */
void twselect(Textwin *t, Mouse *m){
    int sel0, sel1, newsel;
    Point p0, p1, newp;
    sel0=sel1=twpt2rune(t, m->xy);
    p0=tw_rune2pt(t, sel0);
    p1=addpt(p0, Pt(1, 0));
    twhilite(t, sel0, sel1, 1);
    for(;;){
        flushimage(display, 1);
        *m=emouse();
        if((m->buttons&7)!=1) break;
        newsel=twpt2rune(t, m->xy);
        newp=tw_rune2pt(t, newsel);
        if(eqpt(newp, p0)) newp=addpt(newp, Pt(1, 0));
        if(!eqpt(newp, p1)){
            if((sel0<=sel1 && sel1<newsel) || (newsel<sel1 && sel1<sel0))
                tw_hilitep(t, p1, newp);
            else if((sel0<=newsel && newsel<sel1) || (sel1<newsel && newsel<=sel0)){
                twhilite(t, sel1, newsel, 0);
                if(newsel==sel0)
                    tw_hilitep(t, p0, newp);
            }else if((newsel<sel0 && sel0<=sel1) || (sel1<sel0 && sel0<=newsel)){
                twhilite(t, sel0, sel1, 0);
                tw_hilitep(t, p0, newp);
            }
            sel1=newsel;
            p1=newp;
        }
    }
    if(sel0<=sel1){
        t->sel0=sel0;
        t->sel1=sel1;
    }
    else{
        t->sel0=sel1;
```

```
        t->sel1=sel0;
    }
}
```

⟨*function* tw_clrend 115a⟩≡ (119a)

```
/*
 * Clear the area following the last displayed character
 */
void tw_clrend(Textwin *t){
    Point ul;
    int y;
    ul=t->loc[t->bot-t->top];
    y=ul.y+t->hgt;
    if(y>t->r.max.y) y=t->r.max.y;
    pl_clr(t->b, Rpt(ul, Pt(t->r.max.x, y)));
    ul=Pt(t->r.min.x, y);
    pl_clr(t->b, Rpt(ul, t->r.max));
}
```

⟨*function* tw_moverect 115b⟩≡ (119a)

```
/*
 * Move part of a line of text, truncating the source or padding
 * the destination on the right if necessary.
 */
void tw_moverect(Textwin *t, Point uld, Point urd, Point uls, Point urs){
    int sw, dw, d;
    if(urs.y!=uls.y) urs=Pt(t->r.max.x, uls.y);
    if(urd.y!=uld.y) urd=Pt(t->r.max.x, uld.y);
    sw=uls.x-urs.x;
    dw=uld.x-urd.x;
    if(dw>sw){
        d=dw-sw;
        pl_clr(t->b, Rect(urd.x-d, urd.y, urd.x, urd.y+t->hgt));
        dw=sw;
    }
    pl_cpy(t->b, uld, Rpt(uls, Pt(uls.x+dw, uls.y+t->hgt)));
}
```

⟨*function* tw_moveup 115c⟩≡ (119a)

```
/*
 * Move a block of characters up or to the left:
 * Identify contiguous runs of characters whose width doesn't change, and
 * move them in one bitblt per run.
 * If we get to a point where source and destination are x-aligned,
 * they will remain x-aligned for the rest of the block.
 * Then, if they are y-aligned, they're already in the right place.
 * Otherwise, we can move them in three bitblts; one if all the
 * remaining characters are on one line.
 */
void tw_moveup(Textwin *t, Point *dp, Point *sp, Point *esp){
    Point uld, uls;   /* upper left of destination/source */
    int y;
    while(sp!=esp && sp->x!=dp->x){
        uld=*dp;
        uls=*sp;
        while(sp!=esp && sp->y==uls.y && dp->y==uld.y && sp->x-uls.x==dp->x-uld.x){
            sp++;
            dp++;
        }
        tw_moverect(t, uld, *dp, uls, *sp);
```

```
    }
    if(sp==esp || esp->y==dp->y) return;
    if(esp->y==sp->y){ /* one line only */
        pl_cpy(t->b, *dp, Rpt(*sp, Pt(esp->x, sp->y+t->hgt)));
        return;
    }
    y=sp->y+t->hgt;
    pl_cpy(t->b, *dp, Rpt(*sp, Pt(t->r.max.x, y)));
    pl_cpy(t->b, Pt(t->r.min.x, dp->y+t->hgt),
        Rect(t->r.min.x, y, t->r.max.x, esp->y));
    y=dp->y+esp->y-sp->y;
    pl_cpy(t->b, Pt(t->r.min.x, y),
        Rect(t->r.min.x, esp->y, esp->x, esp->y+t->hgt));
}
```

⟨*function* tw_movedn 116a⟩≡ (119a)

```
/*
 * Same as above, but moving down and in reverse order, so as not to overwrite stuff
 * not moved yet.
 */
void tw_movedn(Textwin *t, Point *dp, Point *bsp, Point *esp){
    Point *sp, urs, urd;
    int dy;
    dp+=esp-bsp;
    sp=esp;
    dy=dp->y-sp->y;
    while(sp!=bsp && dp[-1].x==sp[-1].x){
        --dp;
        --sp;
    }
    if(dy!=0){
        if(sp->y==esp->y)
            pl_cpy(t->b, *dp, Rect(sp->x, sp->y, esp->x, esp->y+t->hgt));
        else{
            pl_cpy(t->b, Pt(t->r.min.x, sp->x+dy),
                Rect(t->r.min.x, sp->y, esp->x, esp->y+t->hgt));
            pl_cpy(t->b, Pt(t->r.min.x, dp->y+t->hgt),
                Rect(t->r.min.x, sp->y+t->hgt, t->r.max.x, esp->y));
            pl_cpy(t->b, *dp,
                Rect(sp->x, sp->y, t->r.max.x, sp->y+t->hgt));
        }
    }
    while(sp!=bsp){
        urd=*dp;
        urs=*sp;
        while(sp!=bsp && sp[-1].y==sp[0].y && dp[-1].y==dp[0].y
           && sp[-1].x-sp[0].x==dp[-1].x-dp[0].x){
            --sp;
            --dp;
        }
        tw_moverect(t, *dp, urd, *sp, urs);
    }
}
```

⟨*function* tw_relocate 116b⟩≡ (119a)

```
/*
 * Move the given range of characters, already drawn on
 * the given textwin, to the given location.
 * Start and end must both index characters that are initially on-screen.
 */
```

```
void tw_relocate(Textwin *t, int first, int last, Point dst){
    Point *srcloc;
    int nbyte;
    if(first<t->top || last<first || t->bot<last) return;
    nbyte=(last-first+1)*sizeof(Point);
    srcloc=pl_emalloc(nbyte);
    memmove(srcloc, &t->loc[first-t->top], nbyte);
    tw_setloc(t, first, last, dst);
    if(tw_before(t, dst, srcloc[0]))
        tw_moveup(t, t->loc+first-t->top, srcloc, srcloc+(last-first));
    else
        tw_movedn(t, t->loc+first-t->top, srcloc, srcloc+(last-first));
}
```

⟨*function* twreplace 117⟩≡ (119a)

```
/*
 * Replace the runes with indices from r0 to r1-1 with the text
 * pointed to by text, and with length ntext.
 * Open up a hole in t->text, t->loc.
 * Insert new text, calculate their locs (save the extra loc that's overwritten first)
 * (swap saved & overwritten locs)
 * move tail.
 * calc locs and draw new text after tail, if necessary.
 * draw new text, if necessary
 */
void twreplace(Textwin *t, int r0, int r1, Rune *ins, int nins){
    int olen, nlen, tlen, dtop;
    olen=t->etext-t->text;
    nlen=olen+nins-(r1-r0);
    tlen=t->eslack-t->text;
    if(nlen>tlen){
        tlen=nlen+SLACK;
        t->text=pl_erealloc(t->text, tlen*sizeof(Rune));
        t->eslack=t->text+tlen;
    }
    if(olen!=nlen)
        memmove(t->text+r0+nins, t->text+r1, (olen-r1)*sizeof(Rune));
    if(nins!=0) /* ins can be 0 if nins==0 */
        memmove(t->text+r0, ins, nins*sizeof(Rune));
    t->etext=t->text+nlen;
    if(r0>t->bot)  /* insertion is completely below visible text */
        return;
    if(r1<t->top){  /* insertion is completely above visible text */
        dtop=nlen-olen;
        t->top+=dtop;
        t->bot+=dtop;
        return;
    }
    if(1 || t->bot<=r0+nins){ /* no useful text on screen below r0 */
        if(r0<=t->top) /* no useful text above, either */
            t->top=r0;
        t->bot=tw_setloc(t, r0, nlen, t->loc[r0-t->top]);
        tw_draw(t, r0, t->bot);
        tw_clrend(t);
        return;
    }
    /*
     * code for case where there is useful text below is missing (see `1 ||' above)
     */
}
```

⟨*function* twscroll 118a⟩≡ (119a)

```
/*
 * This works but is stupid.
 */
void twscroll(Textwin *t, int top){
    while(top!=0 && t->text[top-1]!='\n') --top;
    t->top=top;
    t->bot=tw_setloc(t, top, t->etext-t->text, t->r.min);
    tw_draw(t, t->top, t->bot);
    tw_clrend(t);
}
```

⟨*function* twreshape 118b⟩≡ (119a)

```
void twreshape(Textwin *t, Rectangle r){
    t->r=r;
    t->bot=tw_setloc(t, t->top, t->etext-t->text, t->r.min);
    tw_draw(t, t->top, t->bot);
    tw_clrend(t);
}
```

⟨*function* twnew 118c⟩≡ (119a)

```
Textwin *twnew(Image *b, Font *f, Rune *text, int ntext){
    Textwin *t;
    t=pl_emalloc(sizeof(Textwin));
    t->text=pl_emalloc((ntext+SLACK)*sizeof(Rune));
    t->loc=pl_emalloc(SLACK*sizeof(Point));
    t->eloc=t->loc+SLACK;
    t->etext=t->text+ntext;
    t->eslack=t->etext+SLACK;
    if(ntext) memmove(t->text, text, ntext*sizeof(Rune));
    t->top=0;
    t->bot=0;
    t->sel0=0;
    t->sel1=0;
    t->b=b;
    t->font=f;
    t->hgt=f->height;
    t->mintab=stringwidth(f, "0");
    t->tabstop=8*t->mintab;
    return t;
}
```

⟨*function* twfree 118d⟩≡ (119a)

```
void twfree(Textwin *t){
    free(t->loc);
    free(t->text);
    free(t);
}
```

⟨*function* twmove 118e⟩≡ (119a)

```
/*
 * Correct the character locations in a textwin after the panel is moved.
 * This horrid hack would not be necessary if loc values were relative
 * to the panel, rather than absolute.
 */
void twmove(Textwin *t, Point d){
    Point *lp;
    t->r = rectaddpt(t->r, d);
    for(lp=t->loc; lp<t->eloc; lp++)
        *lp = addpt(*lp, d);
}
```

⟨lib_gui/libpanel/textwin.c 119a⟩≡

```
/*
 * Text windows
 * void twhilite(Textwin *t, int sel0, int sel1, int on)
 *  hilite (on=1) or unhilite (on=0) a range of characters
 * void twselect(Textwin *t, Mouse *m)
 *  set t->sel0, t->sel1 from mouse input.
 *  Also hilites selection.
 *  Caller should first unhilite previous selection.
 * void twreplace(Textwin *t, int r0, int r1, Rune *ins, int nins)
 *  Replace the given range of characters with the given insertion.
 *  Caller should unhilite selection while this is called.
 * void twscroll(Textwin *t, int top)
 *  Character with index top moves to the top line of the screen.
 * int twpt2rune(Textwin *t, Point p)
 *  which character is displayed at point p?
 * void twreshape(Textwin *t, Rectangle r)
 *  save r and redraw the text
 * Textwin *twnew(Bitmap *b, Font *f, Rune *text, int ntext)
 *  create a new text window
 * void twfree(Textwin *t)
 *  get rid of a surplus Textwin
 */
```

⟨libpanel *includes* 10e⟩

⟨*constant* SLACK*(*(lib_gui/libpanel/textwin.c)*)* 31b⟩

⟨*function* tw_before 111a⟩
⟨*function* twpt2rune 111b⟩
⟨*function* tw_rune2pt 111c⟩
⟨*function* tw_storeloc 111d⟩
⟨*function* tw_setloc 111e⟩
⟨*function* tw_draw 112⟩
⟨*function* tw_hilitep 113a⟩
⟨*function* twhilite 113b⟩
⟨*function* twselect 114⟩
⟨*function* tw_clrend 115a⟩
⟨*function* tw_moverect 115b⟩
⟨*function* tw_moveup 115c⟩
⟨*function* tw_movedn 116a⟩
⟨*function* tw_relocate 116b⟩
⟨*function* twreplace 117⟩
⟨*function* twscroll 118a⟩
⟨*function* twreshape 118b⟩
⟨*function* twnew 118c⟩
⟨*function* twfree 118d⟩
⟨*function* twmove 118e⟩

D.2.26 lib_gui/libpanel/utf.c

⟨*function* pl_idchar 119b⟩≡ (120d)

```
/*
 * This is the same definition that 8½ uses
 */
int pl_idchar(int c){
    if(c<=' '
    || 0x7F<=c && c<=0xA0
    || utfrune("!\"#$%&'()*+,-./:;<=>?@`[\\]^{|}~", c))
        return 0;
```

```
    return 1;
}
```

⟨*function* pl_rune1st 120a⟩≡ (120d)

```
int pl_rune1st(int c){
    return (c&0xc0)!=0x80;
}
```

⟨*function* pl_nextrune 120b⟩≡ (120d)

```
char *pl_nextrune(char *s){
    do s++; while(!pl_rune1st(*s));
    return s;
}
```

⟨*function* pl_runewidth 120c⟩≡ (120d)

```
int pl_runewidth(Font *f, char *s){
    char r[4], *t;
    t=r;
    do *t++=*s++; while(!pl_rune1st(*s));
    *t='\0';
    return stringwidth(f, r);
}
```

⟨lib_gui/libpanel/utf.c 120d⟩≡

```
⟨libpanel includes 10e⟩

⟨function pl_idchar 119b⟩
⟨function pl_rune1st 120a⟩
⟨function pl_nextrune 120b⟩
⟨function pl_runewidth 120c⟩
```

Glossary

LOC = Lines Of Code

Indexes

Here is a list of the identifiers used, and where they appear. Underlined entries indicate the place of definition. This index is generated automatically.

Bibliography

[Knu92] Donald E. Knuth. *Literate Programming*. Center for the Study of Language and Information, 1992. cited page(s) 7

[Pad09] Yoann Padioleau. Syncweb, literate programming meets unison, 2009. https://github.com/aryx/syncweb. cited page(s) 7